计算机和编程语言

翁恺

计算机是生活必需品

- 如果想要自己设计一些功能和行为，让计算机按照你的意图做事情，就需要写程序了
- 本课程的主要的目的是通过学习编程，来理解计算机是如何解决问题的，理解计算机的能与不能，擅长与非擅长

计算机如何解决问题

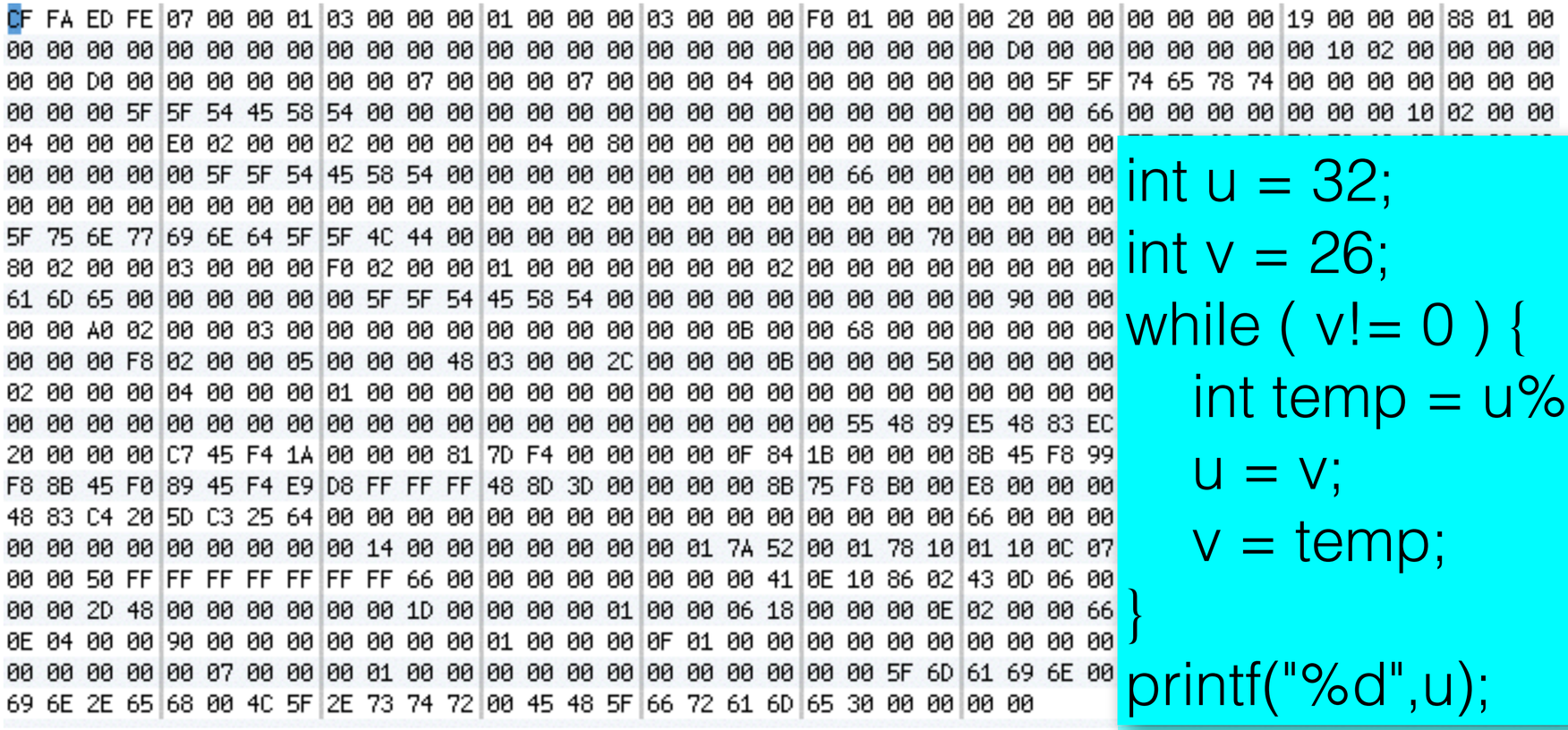
- “请给我一杯水”
  1. 转身走到厨房；
  2. 找到一个杯子；
  3. 找到一个水壶；
  4. 在杯子中倒入一些水；
  5. 拿着杯子走回桌子。

人：What to do
计算机：How to do

计算机语言

- 程序是用特殊的编程语言写出来表达如何解决问题的
- 不是用编程语言来和计算机交谈，而是描述要求它如何做事情的过程或方法

计算机的语言



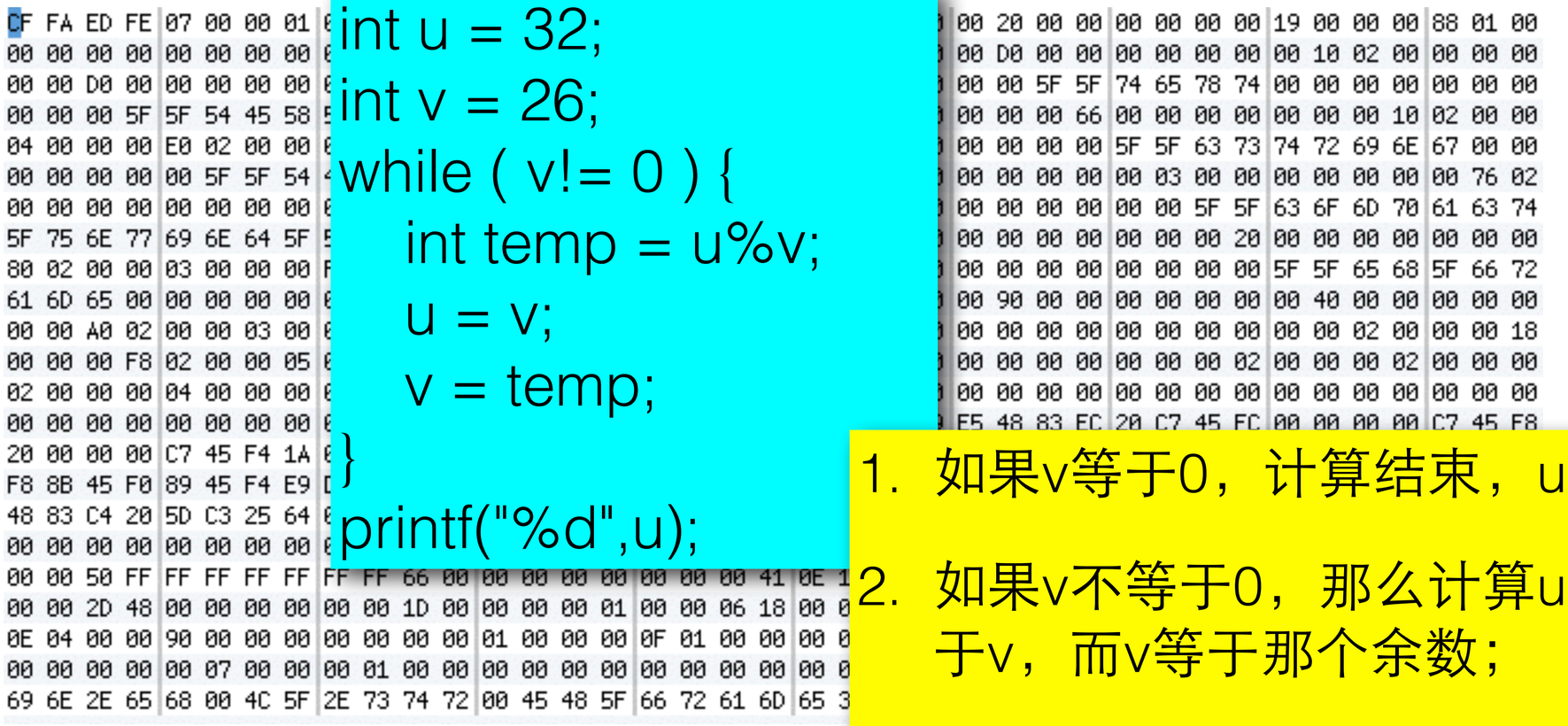

```
int u = 32;
int v = 26;
while ( v!= 0 ) {
    int temp = u%v;
    u = v;
    v = temp;
}
printf("%d",u);
```

辗转相除法

```
int u = 32;
int v = 26;
while ( v!= 0 ) {
    int temp = u%v;
    u = v;
    v = temp;
}
printf("%d",u);
```

1. 如果v等于0，计算结束，u就是最大公约数；
2. 如果v不等于0，那么计算u除以v的余数，让u等于v，而v等于那个余数；
3. 回到第一步。

计算机-程序-算法



```
int u = 32;
int v = 26;
while ( v!= 0 ) {
    int temp = u%v;
    u = v;
    v = temp;
}
printf("%d",u);
```

1. 如果v等于0，计算结束，u就是最大公约数；
2. 如果v不等于0，那么计算u除以v的余数，让u等于v，而v等于那个余数；
3. 回到第一步。

算法

- 我们要让计算机做计算,就需要像这样找出计算的步骤,然后用编程语言写出来
- 计算机做的所有的事情都叫做计算

计算机的思维

- 2x+6=20
  - 解方程
  - 枚举
  - 二分搜索

枚举求最大公约数

1. 设t为2；
2. 如果u和v都能被t整除，则记下这个t
3. t加1后重复第2步，直到t等于u或v；
4. 那么，曾经记下的最大的可以同时整除u和v的t就是gcd

程序的执行

- 解释：借助一个程序，那个程序能试图理解你的程序，然后按照你的要求执行
- 编译：借助一个程序，就像一个翻译，把你的程序翻译成计算机真正能懂的语言——机器语言——写的程序，然后，这个机器语言写的程序就能直接执行了

解释语言vs编译语言

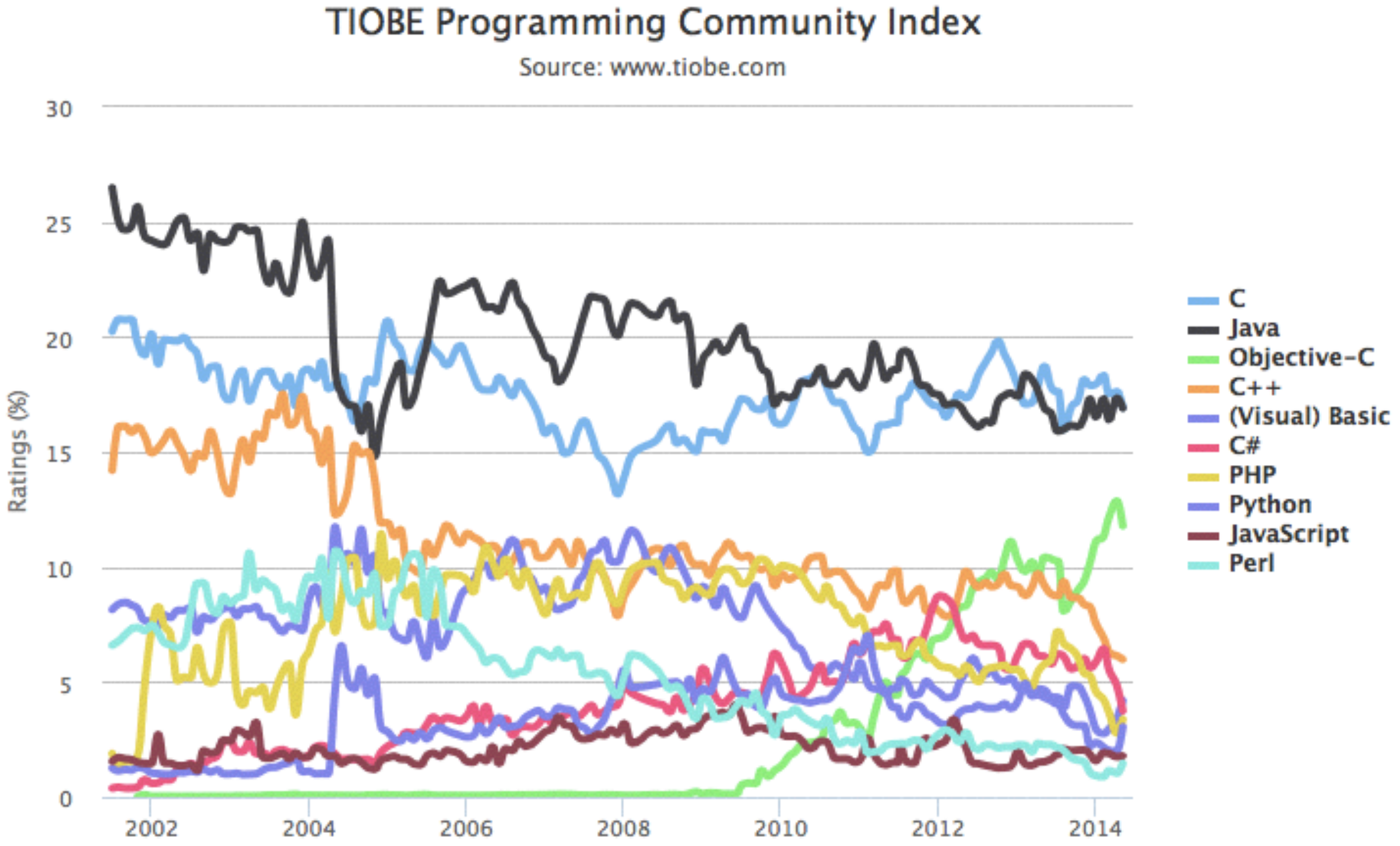
- 语言本无编译/解释之分
- 常用的执行方式而已
- 解释型语言有特殊的计算能力
- 编译型语言有确定的运算性能

C语言

翁恺

为什么是C?

May 2014	May 2013	Change	Programm
1	1		C
2	2		Java
3	3		Objective-C
4	4		C++
5	7	^	(Visual) Basi
6	5	v	C#
7	6	v	PHP
8	8		Python
9	11	^	JavaScript
10	9	v	Perl



其他语言?

- 现代的编程语言在语法上的差异很小
  - 几乎都是C-like语言
- 语言的能力/适用领域主要是由
  - 库和
  - 传统所决定的

```c
#include <stdio.h>

int main()
{
    printf("Hello World!\n");

    return 0;
}
```

```java
class Hello {
    public static void main(String[] arg) {
        System.out.println("Hello World!");
    }
}
```

```
print "Hello World!\n"
```

“it was the summer of ‘69”

PDP-7

图片来源http://www.soetrom.org

Thompson y Ritchie, trabajando en el PDP-11

PDP-11

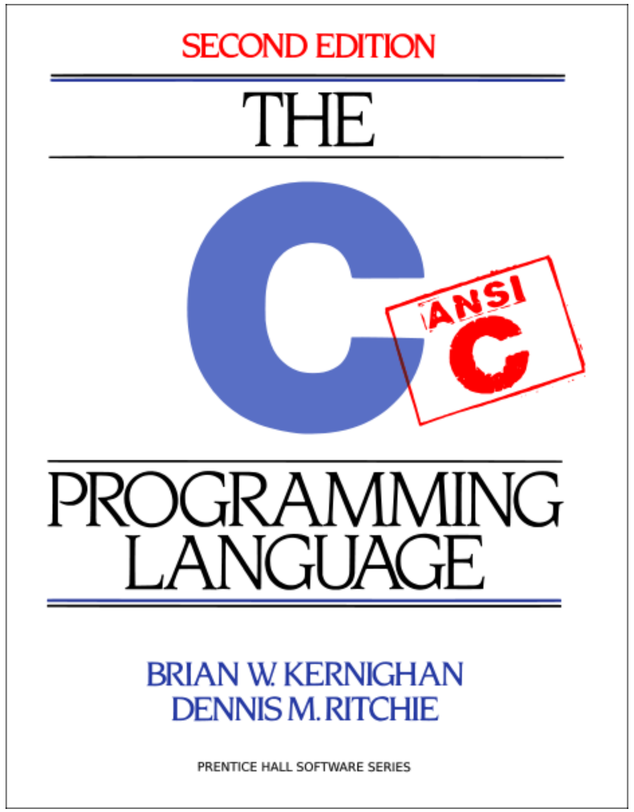
图片来源http://www.ionlitio.com/

C语言

- C语言是从B语言发展而来的，B语言是从BCPL发展而来的，BCPL是从 FORTRAN发展而来的
- BCPL和B都支持指针间接方式，所以C也支持了
- C语言还受到了PL/1的影响，还和PDP-11的机器语言有很大的关系
- 1973年3月，第三版的Unix上出现了C语言的编译器
- 1973年11月，第四版的Unix（System Four）发布了，这个版本是完全用C语言重新写的

C的发展与版本-K&R

- 经典 C
  - 又被叫做 “K&R the C”
  - The C Programming Language, by Brian Kernighan and Dennis Ritchie, 2nd Edition, Prentice Hall



图片来源：http://en.wikipedia.org

C的发展与版本-标准

- 1989年ANSI发布了一个标准——ANSI C
- 1990年ISO接受了ANSI的标准——C89
- C的标准在1995年和1999年两次更新——C95和C99
- 所有的当代编译器都支持C99了

C语言用在哪里?

- 操作系统
- 嵌入式系统
- 驱动程序
- 底层驱动
  - 图形引擎、图像处理、声音效果

C是一种工业语言

- 操作系统
- 嵌入式系统
- 驱动程序
- 底层驱动
  - 图形引擎、图像处理、声音效果

- 开发效率>>学习过程
- 开发效率>>开发乐趣
- 日常应用很少直接用C语言编写
- 学习C的过程主要是写练习代码
  - 而非真实软件

编译—>运行

- C需要被编译才能运行，所以你需要
  - 编辑器
  - 编译器
- 或者，IDE（集成开发环境）

推荐的编程软件

- Dev C++（4.9 for Win7, 5.0 for Win8）
- 免费
- 安装简单
- 不用建工程

其他选择

- MS Visual Studio Express（Windows）
- Xcode（Mac OS X）
- Eclipse-CDT
- Geany（和MinGW一起）
- Sublime（和MinGW一起）
- vim/emacs（和MinGW一起）

第一个C程序

翁恺

初学C的困惑

- 用什么软件好?
  - 选择太多有时候是个麻烦
  - 当代语言往往只有一个编程软件可用
- 那就Dev C++ 5.6.3吧!

第一个C程序

```
#include <stdio.h>

int main()
{
   printf("Hello World!\n");

   return 0;
}
```

在Dev C++里

hello.c

```c
#include <stdio.h>

int main()
{
    printf("Hello World!\n");
    return 0;
}
```

程序框架

```c
#include <stdio.h>

int main()
{

    return 0;
}
```

- 本课程中所有的程序都需要这一段
- 直到学函数之前，我们的代码都只是在这个框架中间

输出

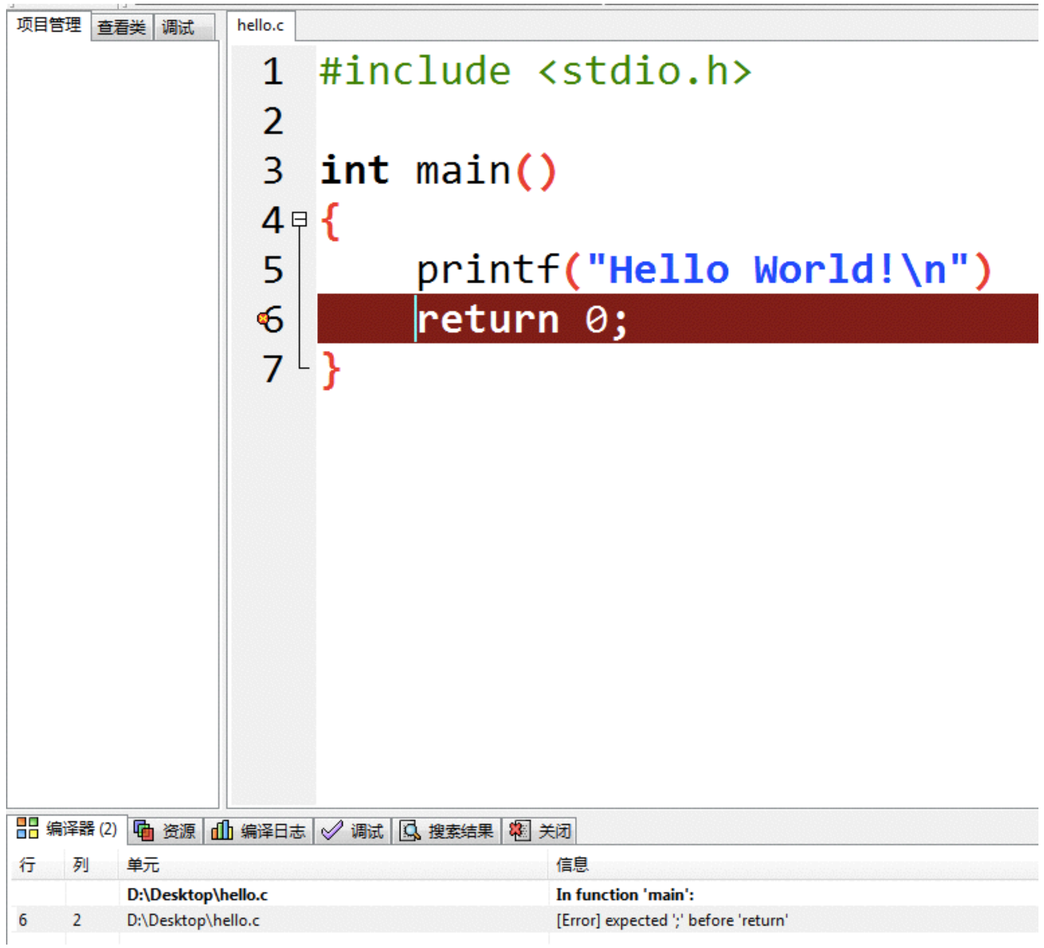
- printf("Hello World!\n");

- ""里面的内容叫做“字符串”，printf会把其中的内容原封不动地输出
- \n表示需要在输出的结果后面换一行

暂停

- 如果你在使用Dev C++ 4.9.9.2
  - system("pause");
- 让程序运行完成后，窗口还能留下
  - 不是Dev C++ 4.9.9.2就不需要这个了

程序中的错误



- 编译的时候发现的错误所在的地方会以红色的底表示出来
- 具体的错误原因列在下方的窗口里（是英文的）
- C的编译器给出的错误提示往往不那么好“猜”

不要用中文！！！

- 中国学生还有一个极其常见的低级错误，就是用了中文输入法来输入程序。那些标点符号，在中文和英文可能看上去相似，但是对于计算机是完全不同的符号，如果你还开了全角标点的话，问题就更严重了

做计算

- printf("%d\n", 23+43);
  - %d说明后面有一个整数要输出在这个位置上

- printf("23+43=%d\n", 23+43);

四则运算

四则运算	C符号	意义
+	+	加
-	-	减
×	*	乘
÷	/	除
	%	取余
()	()	括号

- %表示取两个数相除以后的余数

第一周的练习

- 在自己的计算机上安装好编程的软件
- 依次下载/拷贝课程网站上的四个程序
- 编译并运行
  - 结果如何
  - 有没有问题—>来讨论区讨论吧！
  - 尝试理解

变量

翁恺

做计算

```
printf("23+43=%d\n", 23+43);
```

算找零

```
printf("23+43=%d\n", 23+43);

printf("100-23=%d\n", 100-23);
```

- 如何能在程序运行时输入那个数字23，然后计算输出结果?

算找零

```
printf("23+43=%d\n", 23+43);
```

```
printf("100-23=%d\n", 100-23);
```

需要：
1. 有地方放输入的数字；
2. 有办法输入数字；
3. 输入的数字能参与计算。

- 如何能在程序运行时输入那个数字23，然后计算输出结果?

change.c

```c
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

今后课件上的程序都只有main的{}里面的部分

如何输入

- 输入也在终端窗口中
- 输入是以行为单位进行的，行的结束标志就是你按下了回车键。在你按下回车之前，你的程序不会读到任何东西

变量

```c
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

- int price = 0;
- 这一行，定义了一个变量。变量的名字是price，类型是int，初始值是0。
- 变量是一个保存数据的地方，当我们需要在程序里保存数据时，比如上面的例子中要记录用户输入的价格，就需要一个变量来保存它。用一个变量保存了数据，它才能参加到后面的计算中，比如计算找零。

变量定义

- 变量定义的一般形式就是：
  - <类型名称> <变量名称>;
- int price;
- int amount;
- int price, amount;

变量的名字

- 变量需要一个名字，变量的名字是一种“标识符”，意思是它是用来识别这个和那个的不同的名字。
- 标识符有标识符的构造规则。基本的原则是：标识符只能由字母、数字和下划线组成，数字不可以出现在第一个位置上，C语言的关键字（有的地方叫它们保留字）不可以用做标识符。

C语言的保留字

auto,break,case,char,const,
continue,default,do,double,
else,enum,extern,float,for,
goto,if,int,long,register,return,
short,signed,sizeof,static,
struct,switch,typedef,union,
unsigned,void,volatile,while,
inline,restrict

不需要背诵!

赋值和初始化

```c
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

- int price = 0;
- 这一行，定义了一个变量。变量的名字是price，类型是int，初始值是0。
- price=0是一个式子，这里的“=”是一个赋值运算符，表示将“=”右边的值赋给左边的变量。

赋值

- 和数学不同，a=b在数学中表示关系，即a和b的值一样；而在程序设计中，a=b表示要求计算机做一个动作：将b的值赋给a。关系是静态的，而动作是动态的。在数学中，a=b和b=a是等价的，而在程序设计中，两者的意思完全相反

初始化

- 当赋值发生在定义变量的时候，就像程序1中的第7行那样，就是变量的初始化。虽然C语言并没有强制要求所有的变量都在定义的地方做初始化，但是所有的变量在第一次被使用（出现在赋值运算符的右边）之前被应该赋值一次
- 如果没有初始化?

变量初始化

- <类型名称> <变量名称> = <初始值>;
  - int price = 0;
  - int amount = 100;
- 组合变量定义的时候，也可以在这个定义中单独给单个变量赋初值，如：
  - int price = 0, amount = 100;

读整数

```c
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

- scanf("%d", &price);
- 要求scnaf这个函数读入下一个整数，读到的结果赋值给变量price
- 小心price前面的&

如果输入的不是整数会怎样?

表达式

- “=”是赋值运算符，有运算符的式子就叫做表达式。
  - price=0;
  - change=100-price;

变量类型

```c
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

- int price = 0;
- 这一行，定义了一个变量。变量的名字是price，类型是int，初始值是0。
- C是一种有类型的语言，所有的变量在使用之前必须定义或声明，所有的变量必须具有确定的数据类型。数据类型表示在变量中可以存放什么样的数据，变量中只能存放指定类型的数据，程序运行过程中也不能改变变量的类型。

第二个变量

```c
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

- int change = 100 - price;
- 定义了第二个变量change
- 并且做了计算

C99！

ANSI C

- 只能在代码开头的地方定义变量

```c
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

C99

```c
int price = 0;
int change = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

change = 100 - price;

printf("找您%d元。\n", change);
```

ANSI C

常量

```c
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

- int change = 100 - price;
- 固定不变的数，是常数。直接写在程序里，我们称作直接量（literal）。
- 更好的方式，是定义一个常量：
  - const int AMOUNT = 100;

C99!

常量

```c
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

```c
const int AMOUNT = 100;
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = AMOUNT - price;

printf("找您%d元。\n", change);
```

- int change = 100 - price;
- 100这个固定不变的数，叫做常数。直接写在程序里，我们称作直接量（literal，字面量）。
- 更好的方式，是定义一个常量变量：
  - const int AMOUNT = 100;

C99！

const

- const是一个修饰符，加在int的前面，用来给这个变量加上一个const（不变的）的属性。这个const的属性表示这个变量的值一旦初始化，就不能再修改了。
  - int change = AMOUNT - price;
- 如果你试图对常量做修改，把它放在赋值运算符的左边，就会被编译器发现，指出为一个错误。

C99！

try

- 让用户输入变量AMOUNT的值，而不是使用固定的初始值
  - 这个变量在哪里定义合适呢?

tips

- 程序要求读入多个数字时，可以在一行输入，中间用空格分开，也可以在多行输入
- 在scanf的格式字符串中有几个%d，它就等待用户输入一个整数，当然，字符串后面也需要对应有那么多个整数

plus.c

```c
int a;
int b;

printf("请输入两个整数: ");
scanf("%d %d", &a, &b);
printf("%d + %d = %d\n", a, b, a + b);
```

身高5尺7寸?

- 美国人固执地使用英制计量单位，他们习惯用几尺几寸（英尺英寸）来报自己的身高。如果遇到一个美国人告诉你他是5英尺7，他的身高应该是一米几呢?
  - (5 + 7 ÷ 12) × 0.3048 = 1.7018米

计算身高的程序

```
printf("请分别输入身高的英尺和英寸，"
    "如输入\"5 7\"表示5英尺7英寸: ");

int foot;
int inch;

scanf("%d %d", &foot, &inch);

printf("身高是%f米。\n",
    ((foot + inch / 12) * 0.3048));
```

运行一下

- 为什么总是1.524米?

因为...

- 因为两个整数的运算的结果只能是整数
  - 10/3*3=>?
- 10和10.0在C中是完全不同的数
- 10.0是浮点数

浮点数

- 带小数点的数值。浮点这个词的本意就是指小数点是浮动的，是计算机内部表达非整数（包含分数和无理数）的一种方式。另一种方式叫做定点数，不过在C语言中你不会遇到定点数。人们借用浮点数这个词来表达所有的带小数点的数。

改进

- (foot + inch / 12) * 0.3048;

- (foot + inch / 12.0) * 0.3048;

- 当浮点数和整数放到一起运算时，C会将整数转换成浮点数，然后进行浮点数的运算。

double

- inch是定义为int类型的变量，如果把int换成double，我们就把它改为double类型的浮点数变量了。
- double的意思是“双”，它本来是“双精度浮点数”的第一个单词，人们用来表示浮点数类型。除了double，还有float（意思就是浮点！）表示单精度浮点数。

另一个版本

```
printf("请分别输入身高的英尺和英寸，"
    "如输入\"5 7\"表示5英尺7英寸: ");

double foot;
double inch;

scanf("%lf %lf", &foot, &inch);

printf("身高是%f米。\n",
    ((foot + inch / 12) * 0.3048));
```

数据类型

- 整数
  - int
  - printf("%d",…)
  - scanf("%d",…)
- 带小数点的数
  - double
  - printf("%f",…)
  - scanf("%lf",…)

还有更多的，以后再仔细研究！

整数

- 整数类型不能表达有小数部分的数，整数和整数的运算结果还是整数。计算机里会有纯粹的整数这种奇怪的东西，是因为整数的运算比较快，而且占地方也小。其实人们日常生活中大量做的还是纯粹整数的计算，所以整数的用处还是很大的。

表达式计算

翁恺

表达式

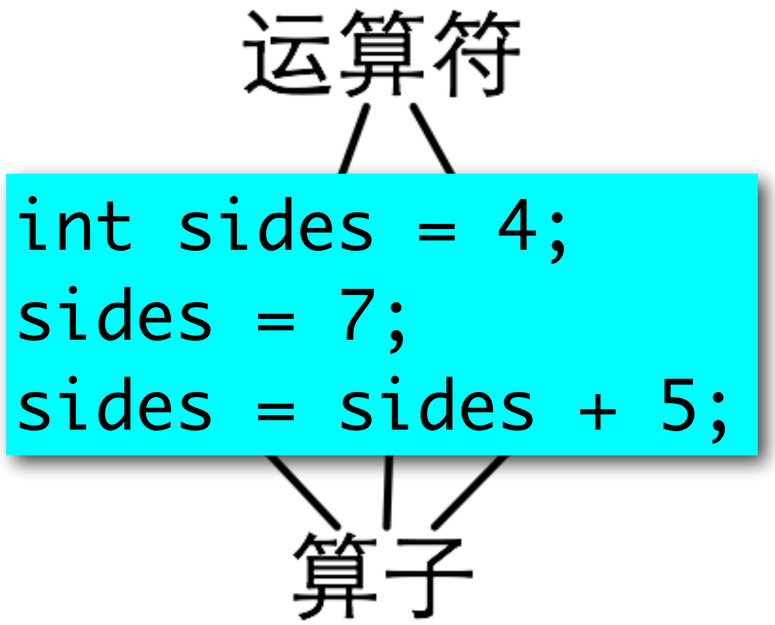
- 一个表达式是一系列运算符和算子的组合，用来计算一个值

```
amount = x * (1 + 0.033) * (1
+ 0.033) * (1 + 0.033);
total = 57;
count = count + 1;
value = (min / 2) * lastValue;
```

运算符

- 运算符（operator）是指进行运算的动作，比如加法运算符“+”，减法运算符“-”。
- 算子（operand）是指参与运算的值，这个值可能是常数，也可能是变量，还可能是一个方法的返回值

运算符



计算

- result = 12 + 6 / 2;
- result = (12 + 6) / 2;
- result = 4 * ((12 – 4) / 2);

四则运算

四则运算	C符号	意义
+	+	加
-	-	减
×	*	乘
÷	/	除
	%	取余
()	()	括号

- %表示取两个数相除以后的余数

计算时间差

- 输入两个时间，每个时间分别输入小时和分钟的值，然后输出两个时间之间的差，也以几小时几分表示

```
int hour1, minute1;
int hour2, minute2;

scanf("%d %d", &hour1, &minute1);
scanf("%d %d", &hour2, &minute2);
```

如果直接分别减，会出现分钟借位的情况：1点40分和2点10分的差?

计算时间差

```c
int hour1, minute1;
int hour2, minute2;

scanf("%d %d", &hour1, &minute1);
scanf("%d %d", &hour2, &minute2);

int t1 = hour1 * 60 + minute1;
int t2 = hour2 * 60 + minute2;

int t = t2-t1;

printf("时间差是%d小时%d分。", t/60, t%60);
```

- hour1*60+minute1—>转换为分钟为单位
- t/60—>小时部分；t%60—>分钟

求平均值

- 写一个程序，输入两个整数，输出它们的平均值

```
int a,b;

scanf("%d %d", &a, &b);

double c = (a+b)/2.0;

printf("%d和%d的平均值=%f\n", a, b, c);
```

运算符优先级

优先级	运算符	运算	结合关系	举例
1	+	单目不变	自右向左	a*+b
1	–	单目取负	自右向左	a*-b
2	*	乘	自左向右	a*b
2	/	除	自左向右	a/b
2	%	取余	自左向右	a%b
3	+	加	自左向右	a+b
3	-	减	自左向右	a-b
4	=	赋值	自右向左	a=b

单目运算符

- 只有一个算子的运算符：+、-

```
int a = 10;
int b = -20;
printf("%d", a * - b);
```

赋值运算符

- 赋值也是运算，也有结果
- a=6的结果是a被赋予的值，也就是6
- a=b=6 —> a=(b=6)

“嵌入式赋值”

```
int a = 6;
int b;
int c = 1+(b=a);
```

- 不利于阅读
- 容易产生错误

结合关系

- 一般自左向右
- 单目+和赋值=自右向左

这样的表达式太复杂，不容易阅读和理解，容易造成读程序时的误解。所以，要避免写出这样的复杂表达式来的。这个表达式应该被拆成若干个表达式，然后以明显的正确的顺序来进行计算。

```
result = a + b + 3 + c;
result = 2;
result = (result = result * 2) * 6 * (result = 3 + result);
```

计算复利

- 在银行存定期的时候，可以选择到期后自动转存，并将到期的利息计入本金合并转存。如果1年期的定期利率是3.3%，那么连续自动转存3年后，最初存入的x元定期会得到多少本息余额?
- 本息合计 = $x(1+3.3\%)^3$

计算复利

```
int x;
scanf("%d", &x);
double amount = x * (1 + 0.033) * (1 + 0.033) *
(1 + 0.033);
printf("%f", amount);
```

要计算任意年以后的本息金额，
就需要做(1+0.033)^n的计算

交换两个变量

- 如果已经有：

```
int a = 6;
int b = 5;
```

如何交换a、b两个变量的值?

程序是按步执行的

- 程序表达的是顺序执行的动作，而不是关系

```
a=b;
b=a;
```

是依次执行的，结果使得a和b都得到b原来的值

交换

```
int t = a;
a = b;
b = t;
```

复合赋值

- 5个算术运算符，+ - * / %，可以和赋值运算符“=”结合起来，形成复合赋值运算符：“+=”、“-=”、“*=”、“/=”和“%=”
  - total += 5;
  - total = total + 5;
- 注意两个运算符中间不要有空格

复合赋值

- total += (sum+100)/2;
  - total = total + (sum+100)/2;
- total * = sum+12;
  - total = total*(sum+12);
- total /= 12+6;
  - total = total / (12+6);

递增递减运算符

- “++”和“--”是两个很特殊的运算符，它们是单目运算符，这个算子还必须是变量。这两个运算符分别叫做递增和递减运算符，他们的作用就是给这个变量+1或者-1。
  - count++;
  - count += 1;
  - count = count + 1;

前缀后缀

- ++和--可以放在变量的前面，叫做前缀形式，也可以放在变量的后面，叫做后缀形式。
- a++的值是a加1以前的值，而++a的值是加了1以后的值，无论哪个，a自己的值都加了1了。

前缀后缀

表达式	运算	表达式的值
count++	给count加1	count原来的值
++count	给count加1	count+1以后的值
count--	给count减1	count原来的值
--count	给count减1	count-1以后的值

++--

- 这两个运算符有其历史来源
- 可以单独使用，但是不要组合进表达式
  - ++i++ -->?
  - i++++ —>?
- a = b+=c++-d+--e/-f

判断

翁恺

做判断

计算时间差

- 输入两个时间，每个时间分别输入小时和分钟的值，然后输出两个时间之间的差，也以几小时几分表示

```
int hour1, minute1;
int hour2, minute2;

scanf("%d %d", &hour1, &minute1);
scanf("%d %d", &hour2, &minute2);
```

如果直接分别减，会出现分钟借位的情况：1点40分和2点10分的差?

如果

- 就用分别减的方案，然后判断有没有出现借位行不行?
- 借位的表现是，分钟减的结果小于0

```
int im = minute2 - minute1;
if ( im <0 ) {
    …
}
```

```c
int hour1, minute1;
int hour2, minute2;

scanf("%d %d", &hour1, &minute1);
scanf("%d %d", &hour2, &minute2);

int ih = hour2 - hour1;
int im = minute2 - minute1;
if ( im <0 ) {
    im = 60 + im;
    ih --;
}

printf("时间差是%d小时%d分。\n", ih, im);
```

如果 （ im 小于 0 ），那么

如果

```
if ( 条件成立 ) {
    …
}
```

判断的条件

条件

- 计算两个值之间的关系，所以叫做关系运算

运算符	意义
= =	相等
!=	不相等
>	大于
>=	大于或等于
<	小于
<=	小于或等于

关系运算的结果

- 当两个值的关系符合关系运算符的预期时，关系运算的结果为整数1，否则为整数0
  - printf("%d\n", 5==3);
  - printf("%d\n", 5>3);
  - printf("%d\n", 5<=3);

优先级

- 所有的关系运算符的优先级比算术运算的低，但是比赋值运算的高
  - 7 >= 3 + 4
  - int r = a>0;

优先级

- 判断是否相等的==和!= 的优先级比其他的低，而连续的关系运算是从左到右进行的
  - 5 > 3 == 6 > 4
  - 6 > 5 > 4
  - a == b == 6
  - a == b > 0

找零计算器

找零计算

这个应该是类似超市之类用来进行找零计算的一个小程序，输入商品价格，收一百元，计算应该找零多少。

该程序有可以改进的地方，超市收入不可能一直是100，所以把收到的金额也写进程序，这样就可以进行任意计算了。

改进如下：

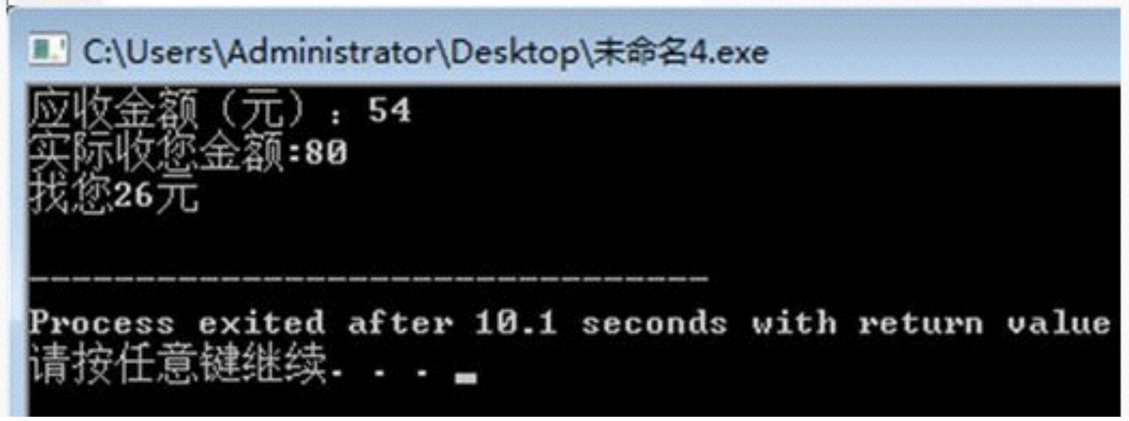
```
#include <stdio.h>

int main()

{

int price=0;

int get=0;

printf("请输入金额（元）：");

scanf("%d",&price);

printf("请输入收入（元）：");

scanf("%d",&get);

int change=get-price;

printf("找您%d元。\n",change);

return 0;

}
```

socovo 5月22日 来自课件"change.c"

关注 回复 4 | 举报 | 管理

小小修改了下，更使用了，呵呵

```
1  #include<stdio.h>
2  int main()
3  {
4      int price=0;
5      int total=0;
6      printf("应收金额（元）：");
7      scanf("%d",&price);
8      printf("实际收您金额:");
9      scanf("%d",&total);
10     int change=total-price;
11     printf("找您%d元\n",change);
12     return 0;
13 }
```



找零计算器

- 找零计算器需要用户做两个操作：输入购买的金额，输入支付的票面，而找零计算器则根据用户的输入做出相应的动作：计算并打印找零，或告知用户余额不足以购买。
- 从计算机程序的角度看，这就是意味着程序需要读用户的两个输入，然后进行一些计算和判断，最后输出结果。

找零计算器

```c
//  初始化
int price = 0;
int bill = 0;
//  读入金额和票面
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
//  计算找零
printf("应该找您: %d\n", bill - price);
```

注释

- 以两个斜杠“//”开头的语句把程序分成了三个部分：
  1. 初
  2. 读
  3. 计

注释（comment）插入在程序代码中，用来向读者提供解释信息。它们对于程序的功能没有任何影响，但是往往能使得程序更容易被人类读者理解。

/* */注释

- 延续数行的注释，要用多行注释的格式来写。多行注释由一对字符序列“/*”开始，而以“*/”结束。
- 也可以用于一行内的注释
  - int ak=47 /* 36*/, y=9;

比较

```c
int price = 0;
int bill = 0;
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
printf("应该找您: %d\n", bill - price);
```

```c
//  初始化
int price = 0;
int bill = 0;
//  读入金额和票面
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
//  计算找零
printf("应该找您: %d\n", bill - price);
```

高下立见

判断票面够不够

```
//   初始化
int price = 0;
int bill = 0;
//   读入金额和票面
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
//   计算找零
if ( bill >= price ) {
    printf("应该找您: %d\n", bill - price);
}
```

如果 （ bill 大于等于 price ），那么

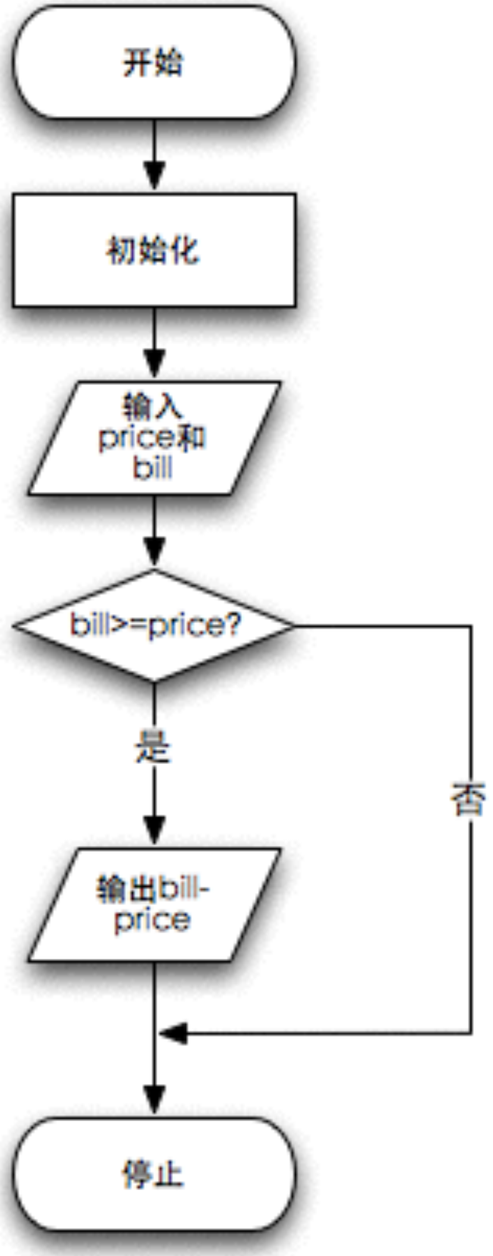
不够怎么办?

```
//  初始化
int price = 0;
int bill = 0;
//  读入金额和票面
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
//  计算找零
if ( bill >= price ) {
    printf("应该找您: %d\n", bill - price);
}
```

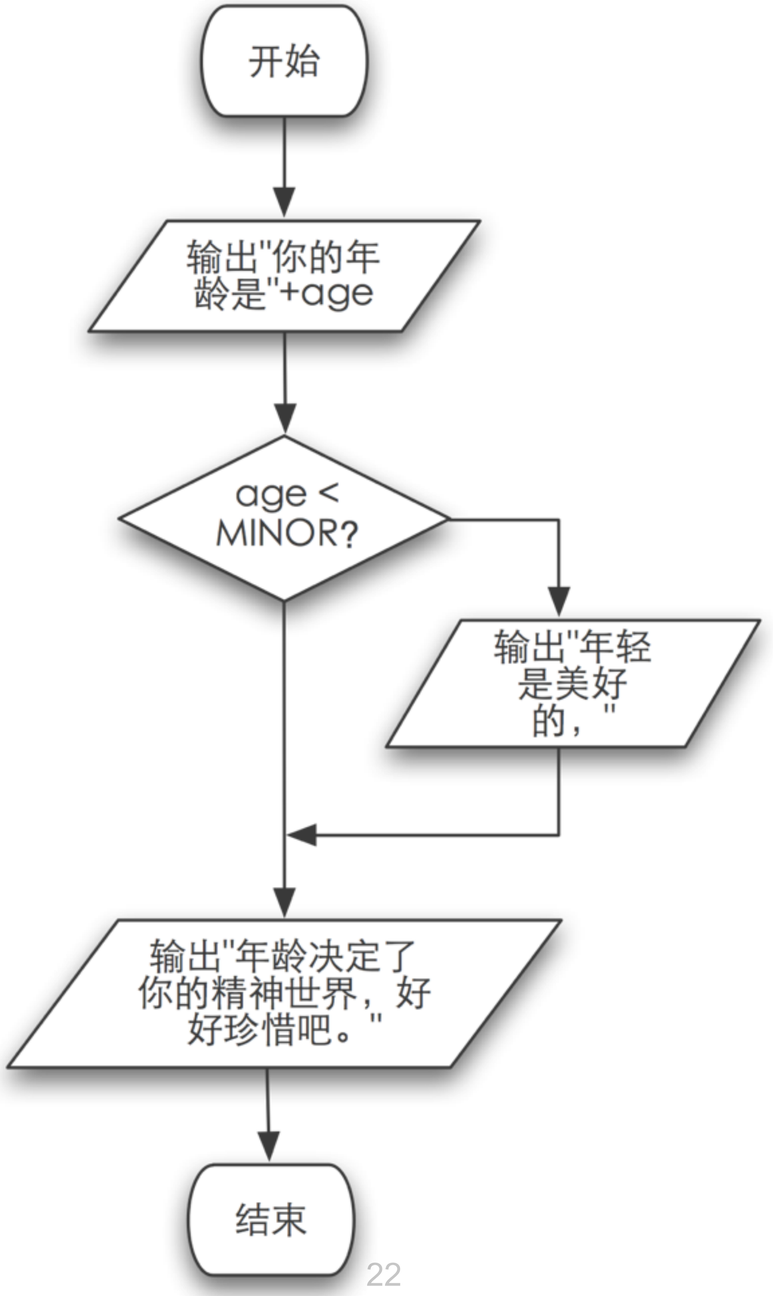
- 画流程图来看看

流程图

```c
//  初始化
int price = 0;
int bill = 0;
//  读入金额和票面
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
//  计算找零
if ( bill >= price ) {
    printf("应该找您: %d\n", bill - price);
}
```



```c
const int MINOR = 35;

int age = 0;

printf("请输入你的年龄: ");
scanf("%d", &age);

printf("你的年龄是%d岁。\n", age);

if ( age < MINOR ) {
    printf("年轻是美好的，");
}

printf("年龄决定了你的精神世界，好好珍惜吧。\n");
```



否则的话

不够怎么办?

```
//   初始化
int price = 0;
int bill = 0;
//   读入金额和票面
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
//   计算找零
if ( bill >= price ) {
    printf("应该找您: %d\n", bill - price);
}
```

- 如果在if后面还有语句，它们在if结束后会执行，无论条件如何

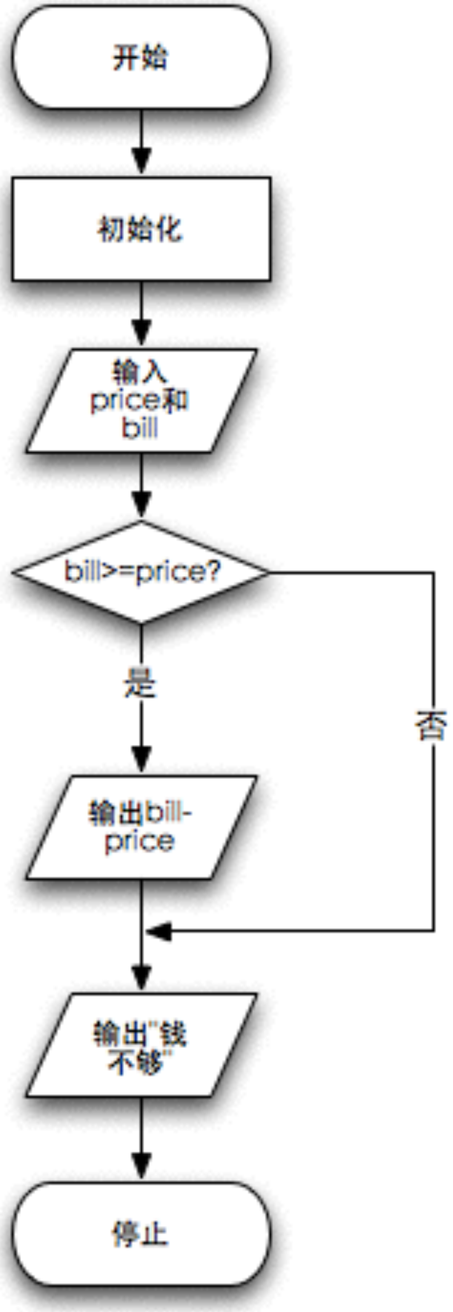
不够怎么办?

```
//  初始化
int price = 0;
int bill = 0;
//  读入金额和票面
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
//  计算找零
if ( bill >= price ) {
    printf("应该找您: %d\n", bill - price);
}
printf("你的钱不够\n");
```

- 如果在if后面还有语句，它们在if结束后会执行，无论条件如何

不够怎么办?

```
//  初始化
int price = 0;
int bill = 0;
//  读入金额和票面
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
//  计算找零
if ( bill >= price ) {
    printf("应该找您: %d\n", bill - price);
}
printf("你的钱不够\n");
```



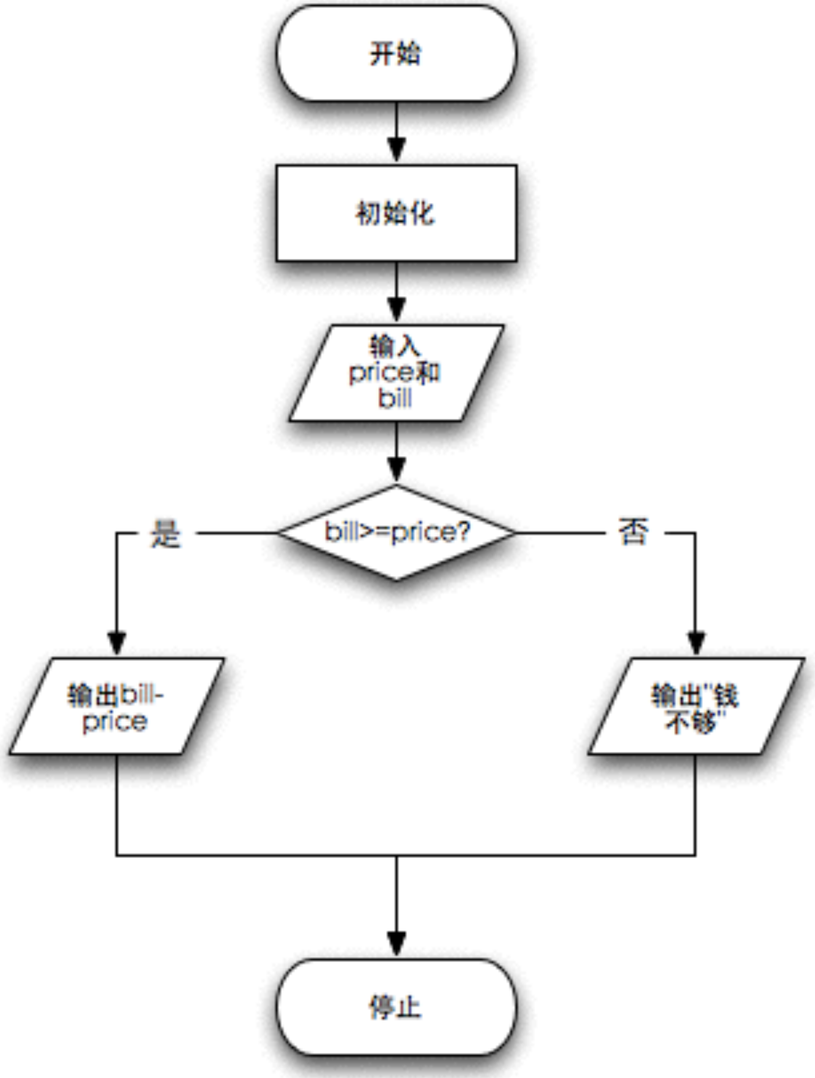
else

```
//  初始化
int price = 0;
int bill = 0;
//  读入金额和票面
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
//  计算找零
if ( bill >= price ) {
    printf("应该找您: %d\n", bill - price);
} else {
    printf("你的钱不够\n");
}
```

- else = 否则的话

else

```
//  初始化
int price = 0;
int bill = 0;
//  读入金额和票面
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
//  计算找零
if ( bill >= price ) {
    printf("应该找您: %d\n", bill - price);
} else {
    printf("你的钱不够\n");
}
```



比较数的大小

```
printf("请输入两个整数: ");
scanf("%d %d", &a, &b);

int max = 0;
if ( a > b )
    max = a;
}

printf("大的那个是%d\n", max);
```

它没有解决b大于a的问题，当a>b的条件不成立时，程序就结束了，max没有得到值

方案I

```c
int a,b;

printf("请输入两个整数: ");
scanf("%d %d", &a, &b);

int max = 0;
if ( a > b ) {
    max = a;
}
if ( b > a ) {
    max = b;
}

printf("大的那个是%d\n", max);
```

方案2

```c
int a,b;

printf("请输入两个整数: ");
scanf("%d %d", &a, &b);

int max = 0;
if ( a > b ) {
    max = a;
} else {
    max = b;
}

printf("大的那个是%d\n", max);
```

方案3

```c
int a,b;

printf("请输入两个整数: ");
scanf("%d %d", &a, &b);

int max = b;
if ( a > b ) {
    max = a;
}

printf("大的那个是%d\n", max);
```

* 比较一下

```c
int a,b;

printf("请输入两个整数: ");
scanf("%d %d", &a, &b);

int max = 0;
if ( a > b ) {
    max = a;
} else {
    max = b;
}

printf("大的那个是%d\n", max);
```

```c
int a,b;

printf("请输入两个整数: ");
scanf("%d %d", &a, &b);

int max = b;
if ( a > b ) {
    max = a;
}

printf("大的那个是%d\n", max);
```

if语句

if语句

- 一个基本的if语句由一个关键字if
上在括号里的一个表示条件的逻
式，然后是一对大括号“{}”之间
语句。如果表示条件的逻辑表达
果不是零，那么就执行后面跟着
大括号中的语句，否则就跳过这
不执行，而继续下面的其他语句

```
if ( total > amount )
        total += amount+10;
```

if语句这一行结束的时候并没有表示语句结束的“;”，而后面的赋值语句写在if的下一行，并且缩进了，在这一行结束的时候有一个表示语句结束的“;”。这表明这条赋值语句是if语句的一部分，if语句拥有和控制这条赋值语句，决定它是否要被执行。

计算薪水

```
const double RATE = 8.25;
const int STANDARD = 40;
double pay = 0.0;
int hours;

printf("请输入工作的小时数: ");
scanf("%d", &hours);
printf("\n");
if (hours > STANDARD)
    pay = STANDARD * RATE +
        (hours-STANDARD) * (RATE * 1.5);
else
    pay = hours * RATE;
printf("应付工资: %f\n", pay);
```

判断成绩

```
const int PASS=60;
int score;

printf("请输入成绩: ");
scanf("%d", &score);

printf("你输入的成绩是%d.\n", score);
if ( score < PASS )
    printf("很遗憾，这个成绩没有及格。");
else
    printf("祝贺你，这个成绩及格了。");
printf("再见\n");
```

判断成绩

```c
const int PASS=60;
int score;

printf("请输入成绩: ");
scanf("%d", &score);

printf("你输入的成绩是%d.\n", score);
if ( score < PASS )
    printf("很遗憾，这个成绩没有及格。");
else
    printf("祝贺你，这个成绩及格了。");
printf("再见\n");
```

```c
const int PASS=60;
int score;

printf("请输入成绩: ");
scanf("%d", &score);

printf("你输入的成绩是%d.\n", score);
if ( score < PASS )
    printf("很遗憾，这个成绩没有及格。");
else {
    printf("祝贺你，这个成绩及格了。");
    printf("再见\n");
}
```

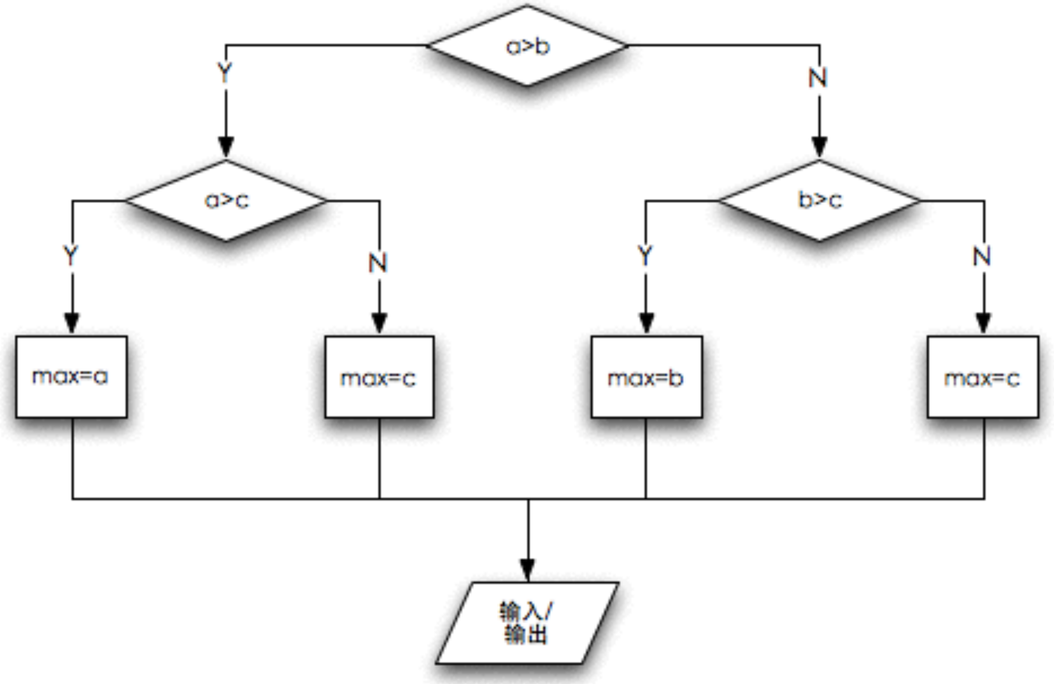
嵌套的if

找三个数中的最大

```c
int a,b;

printf("请输入两个整数: ");
scanf("%d %d", &a, &b);

int max = 0;
if ( a > b ) {
    max = a;
} else {
    max = b;
}

printf("大的那个是%d\n", max);
```

- 如果要求输入三个整数，输出其中最大的，应该怎么做?

```c
int a,b,c;
scanf("%d %d %d", &a, &b, &c);

int max = 0;

if ( a>b ) {
    if ( a>c ) {
        max = a;
    } else {
        max = c;
    }
} else {
    if ( b>c ) {
        max = b;
    } else {
        max = c;
    }
}

printf("The max is %d\n", max);
```



```c
int a,b,c;
scanf("%d %d %d", &a, &b, &c);

int max = 0;

if ( a>b ) {
    if ( a>c ) {
        max = a;
    } else {
        max = c;
    }
} else {
    if ( b>c ) {
        max = b;
    } else {
        max = c;
    }
}

printf("The max is %d\n", max);
```

```c
int a,b,c;
scanf("%d %d %d", &a, &b, &c);

int max = 0;

if ( a>b )
    if ( a>c )
        max = a;
    else
        max = c;
else
    if ( b>c )
        max = b;
    else
        max = c;

printf("The max is %d\n", max);
```

嵌套的判断

- 当if的条件满足或者不满足的时候要执行的语句也可以是一条if或if-else语句，这就是嵌套的if语句

```
if ( code == READY )
   if ( count <20 )
      printf("一切正常\n");
   else
      printf("继续等待\n");
```

else的匹配

- else总是和最近的那个if匹配

```
if ( code == READY ) {
    if ( count <20 )
       printf("一切正常\n");
} else
   printf("继续等待\n");
```

缩进

- 缩进格式不能暗示else的匹配

```
if ( code == READY )
    if ( count <20 )
       printf(“一切正常\n");
else
      printf(“继续等待\n");
```

嵌套的if

```
if ( gameover == 0 )
  if ( player2move == 2 )
printf("Your turn\n");
```

```
if ( gameover == 0 )
  if ( player2move == 2 )
    printf("Your turn\n");
  else
    printf("My turn\n");
```

```
if ( gameover == 0 )
  if ( player2move == 2 )
    printf("Your turn\n");
  else
    printf("My turn\n");
else
  printf("GAME OVER\n");
```

```
if ( gameover == 0 )
  if ( player2move == 2 )
    printf("Your turn\n");
else
  printf("GAME OVER\n");
```

```
if ( gameover == 0 ) {
   if ( player2move == 2 )
      printf("Your turn\n");
} else
   printf("GAME OVER\n");
```

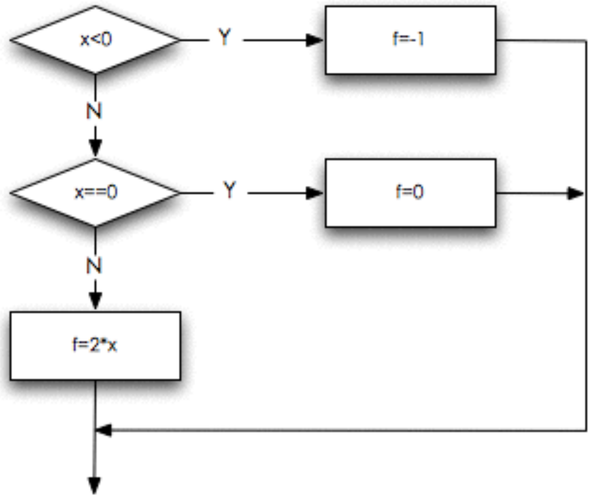
tips

- 在if或else后面总是用{}
- 即使只有一条语句的时候

级联的if-else if

分段函数

```
if ( x < 0 ) {
  f = -1;
} else if ( x == 0 ) {
  f = 0;
} else {
  f = 2 * x;
}
```



级联的if-else if

```
if ( exp1 )
  st1;
else if ( exp2 )
  st2;
else
  st3;
```

```
int f;
if ( x < 0 ) {
  f = -1;
} else if ( x == 0 ) {
  f = 0;
} else {
  f = 2 * x;
}
printf("%d", f);
```

```
if ( x < 0 ) {
  printf("%d", -1);
} else if ( x == 0 ) {
  printf("%d", 0);
} else {
  printf("%d", 2 * x);
}
```

单一出口

if语句常见错误

if语句常见的错误

- 忘了大括号
- if后面的分号
- 错误使用==和=
- 使人困惑的else

忘了大括号

```
if ( age > 60 )
  salary = salary * 1.2;
  printf("%f",sal
```

```
if ( age > 60 )
{
  salary = salary * 1.2;
  printf("%f",salay);
}
```

- 永远在if和else后面加上大括号，即使当时后面只有一条语句

if后面的分号

```
if ( age > 60 );
{
  salary = salary * 1.2;
  printf("%f",salay);
}
```

错误使用==和=

- if只要求()里的值是零或非零

```
if ( a = b )
{
  printf("A=B");
}
```

代码风格

- 在if和else之后必须加上大括号形成语句块；
- 大括号内的语句缩进一个tab的位置；

风格是三观...

```
if ( x < 0 ) {
    f = -1;
} else if ( x == 0 ) {
    f = 0;
} else {
    f = 2 * x;
}
```

```
if ( x < 0 )
{
    f = -1;
} else if ( x == 0 )
{
    f = 0;
} else
{
    f = 2 * x;
}
```

```
if ( x < 0 )
{
    f = -1;
}
else if ( x == 0 )
{
    f = 0;
}
else
{
    f = 2 * x;
}
```

- 在if和else之后必须加上大括号形成语句块；
- 大括号内的语句缩进一个tab的位置；

多路分支

switch-case

```
if ( type==1 )
   printf("你好");
else if ( type==2)
   printf("早上好");
else if ( type==3 )
   printf("晚上好");
else if ( type==4 )
   printf("再见");
else
   printf("啊，什么啊？");
```

```
switch ( type ) {
 case 1:
  printf("你好");
  break;
 case 2:
   printf("早上好");
   break;
 case 3:
   printf("晚上好");
   break;
 case 4:
   printf("再见");
   break;
 default:
   printf("啊，什么啊？");
}
```

switch-case

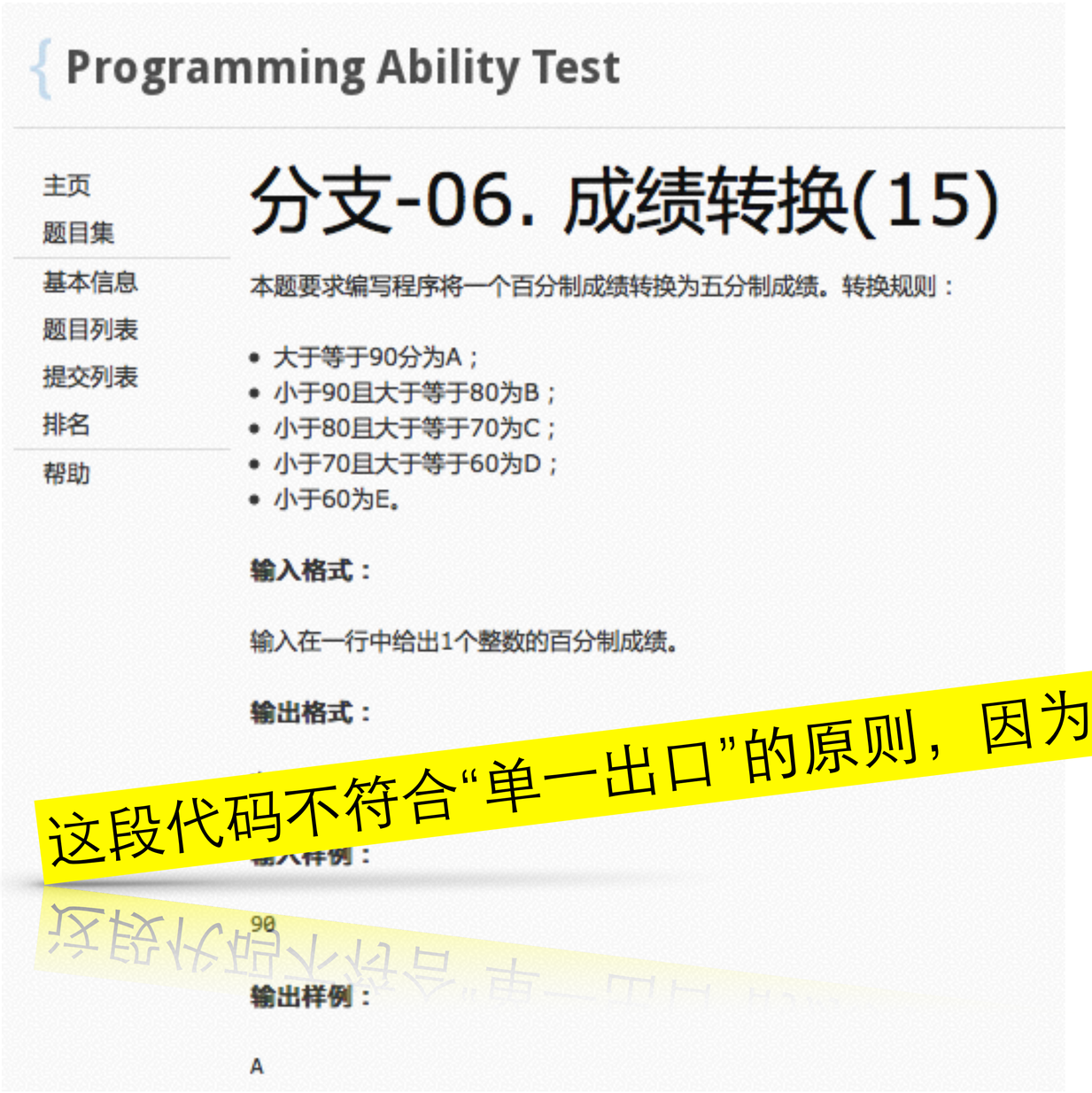
```
switch ( 控制表达式 ) {
case 常量:
   语句
   ……
case 常量:
   语句
   ……
default:
   语句
   ……
}
```

- 控制表达式只能是整数型的结果
- 常量可以是常数，也可以是常数计算的表达式
- 根据表达式的结果，寻找匹配的case，并执行case后面的语句，一直到break为止
- 如果所有的case都不匹配，那么就执行default后面的语句；如果没有default，那么就什么都不做

break

```
switch ( type )
{
case 1:
case 2:
    printf("你好\n");
    break;
case 3:
    printf("晚上好\n");
case 4:
    printf("再见\n");
    break;
default:
    printf("啊，什么啊？\n");
    break;
}
```

- switch语句可以看作是一种基于计算的跳转，计算控制表达式的值后，程序会跳转到相匹配的case（分支标号）处。分支标号只是说明switch内部位置的路标，在执行完分支中的最后一条语句后，如果后面没有break，就会顺序执行到下面的case里去，直到遇到一个break，或者switch结束为止。



Programming Ability Test

主页
题目集
基本信息
题目列表
提交列表
排名
帮助

分支-06. 成绩转换(15)

本题要求编写程序将一个百分制成绩转换为五分制成绩。转换规则：

- 大于等于90分为A；
- 小于90且大于等于80为B；
- 小于80且大于等于70为C；
- 小于70且大于等于60为D；
- 小于60为E。

输入格式：

输入在一行中给出1个整数的百分制成绩。

输出格式：

输入样例：

90

输出样例：

A

这段代码不符合“单一出口”的原则，因为我们还没学过字符或字符串数据的处理

```
//printf("输入成绩（0-100）");
int grade;
scanf("%d", &grade);
grade /=10;
switch ( grade ) {
  case 10:
  case 9:
    printf("A\n");
    break;
  case 8:
    printf("B\n");
    break;
  case 7:
    printf("C\n");
    break;
  case 6:
    printf("D\n");
    break;
  default:
    printf("F\n");
    break;
}
```

```
printf("请输入月份: ");
int month;
scanf("%d", &month);
switch ( month ) {
      case 1: printf("January\n"); break;
      case 2: printf("February\n"); break;
      case 3: printf("March\n"); break;
      case 4: printf("April\n"); break;
      case 5: printf("May\n"); break;
      case 6: printf("June\n"); break;
      case 7: printf("July\n"); break;
      case 8: printf("August\n"); break;
      case 9: printf("September\n"); break;
      case 10:printf("October\n"); break;
      case 11:printf("November\n"); break;
      case 12:printf("December\n"); break;
}
```

今后可以用数组来做

try

$$f(x) = \begin{cases} -1; & x<0 \\ 0; & x=0 \\ 2x; & x>0 \end{cases}$$

- 这个分段函数能否用switch-case?

循环

数数几位数

- 程序要读入一个4位以下（含4位）的正整数，然后输出这个整数的位数。如：
- 输入：352，输出：3

人vs计算机

- 人的方式：眼睛一看就知道了
  - 352 —> 3位！
- 计算机的方式：判断数的范围来决定它的位数
  - 352∈[100,999] —> 3位
  - 人不擅长，因为人对数字的计算能力比文字弱

程序实现

```
int x;
int n =1;

scanf("%d", &x);

if ( x > 999 ) {
    n = 4;
} else if ( x > 99 ) {
    n = 3;
} else if ( x > 9 ) {
    n = 2;
}

printf("%d\n", n);
```

- 因为题目明确了4位数及以下的正整数，所以可以简化一些判断
- 因为从高处往下判断，所以不需要判断上限了
  - 反过来不行
- 问题：任意范围的正整数怎么办?

```c
int x;
int n =0;

scanf("%d", &x);

if ( x > 999 ) {
    n = 4;
} else if ( x > 99 ) {
    n = 3;
} else if ( x > 9 ) {
    n = 2;
} else if ( x > 0 ) {
    n = 1;
}

printf("%d\n", n);
```

```c
int x;
int n =0;

scanf("%d", &x);

if ( x > 9999 ) {
    n = 5;
} else if ( x > 999 ) {
    n = 4;
} else if ( x > 99 ) {
    n = 3;
} else if ( x > 9 ) {
    n = 2;
} else if ( x > 0 ) {
    n = 1;
}

printf("%d\n", n);
```

如何才是个头啊！！！

```c
int x;
int n =0;

scanf("%d", &x);

if ( x > 99999 ) {
    n = 6;
} else if ( x > 9999 ) {
    n = 5;
} else if ( x > 999 ) {
    n = 4;
} else if ( x > 99 ) {
    n = 3;
} else if ( x > 9 ) {
    n = 2;
} else if ( x > 0 ) {
    n = 1;
}

printf("%d\n", n);
```

换个方式想

- 352 —> 3 很快，
  12381284326751827361827361267531 7是
  几位?
- 数数！

数数

- 1238128432675182736182736126753l7
- 人怎么数？从左往右数，一次划掉一个数字
- 计算机怎么划掉那个数字？

三位数逆序的题

- 352
  - 352 %100 —> 52
- 那么，
  12381284326751827361827361267531 7%
  10000000000000000000000000000000-> 2381284326751827361827361267531 7
- 怎么得到那个
  10000000000000000000000000000000?

人vs计算机

- 如果换一下，从右边开始划
- 123812843267518273618273612675317/10->
  12381284326751827361827361267531
- 去掉最右边的数。然后?
- 不断地划，直到没数可以划...
  - 在这个过程中计数

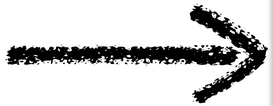
试试代码

```c
int x;
int n =0;

scanf("%d", &x);

n++;
x /= 10;
if ( x > 0 ) {
    n++;
    x /= 10;
    if ( x > 0 ) {
        n++;
        x /= 10;
        if ...
    }
}

printf("%d\n", n);
```

- 这事儿还是没完没了…

试试代码

```c
int x;
int n =0;

scanf("%d", &x);

n++;
x /= 10;
if ( x > 0 ) {
    n++;
    x /= 10;
    if ( x > 0 ) {
        n++;
        x /= 10;
    }
}

printf("%d\n", n);
```



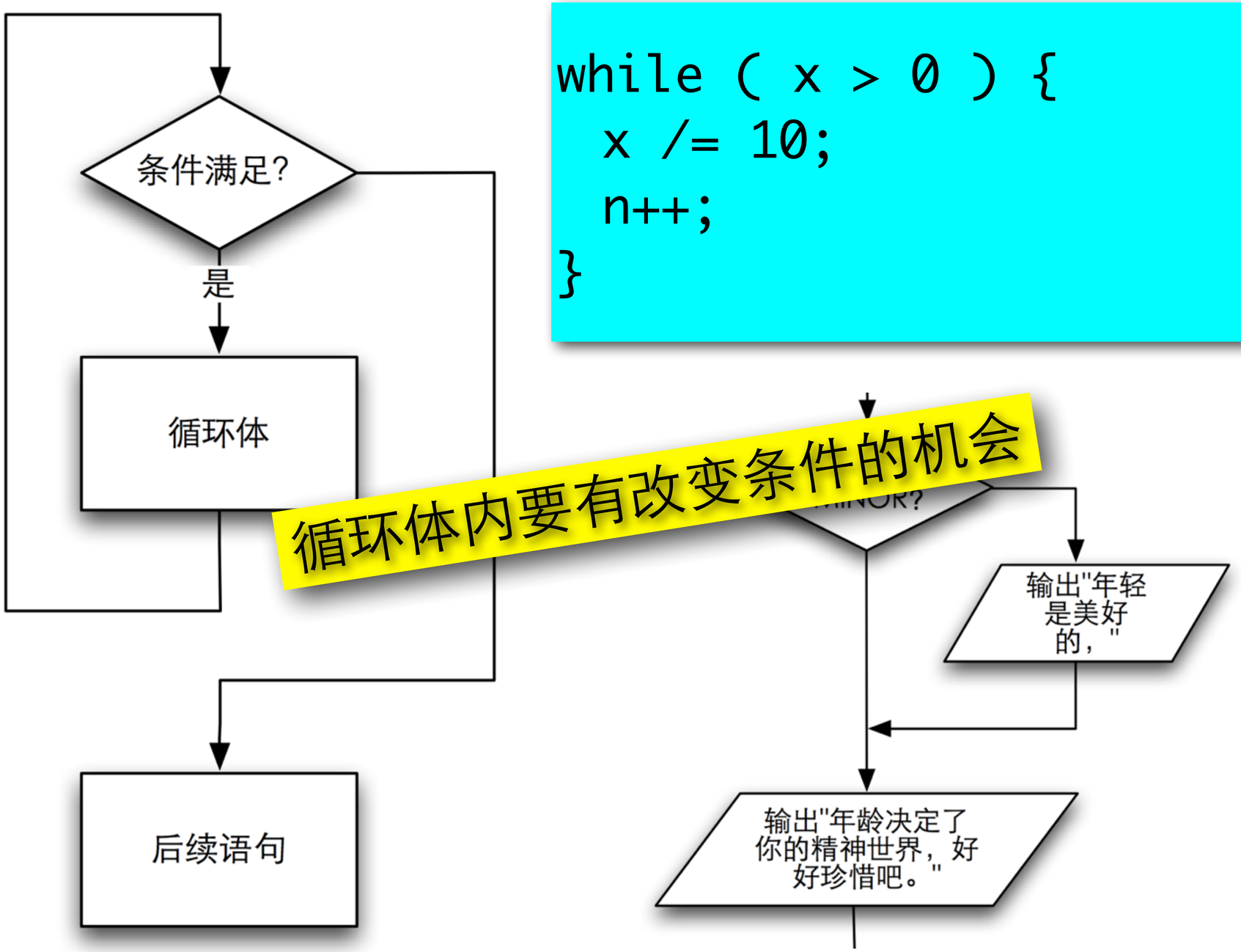
```c
int x;
int n =0;

scanf("%d", &x);

n++;
x /= 10;
while ( x > 0 ) {
    n++;

printf("%d\n", n);
```

别拿123812843267518273618273612675317去试！

while循环

```
if ( x > 0 ) {
  x /= 10;
  n++;
}
```

```
while ( x > 0 ) {
  x /= 10;
  n++;
}
```




```
while ( x > 0 ) {
  x /= 10;
  n++;
}
```

循环体内要有改变条件的机会

while循环

- 如果我们把while翻译作“当”，那么一个while循环的意思就是：当条件满足时，不断地重复循环体内的语句。

- 循环执行之前判断是否继续循环，所以有可能循环一次也没有被执行；
- 条件成立是循环继续的条件。

再想想

```c
int x;
int n =0;

scanf("%d", &x);

n++;
x /= 10;
while ( x > 0 ) {
    n++;
    x /= 10;
}

printf("%d\n", n);
```

```c
int x;
int n =0;

scanf("%d", &x);

while ( x > 0 ) {
    n++;
    x /= 10;
}

printf("%d\n", n);
```

- 如果没有外面的运算?

看程序运行结果

- 人脑模拟计算机的运行，在纸上列出所有的变量，随着程序的进展不断重新计算变量的值。当程序运行结束时，留在表格最下面的就是程序的最终结果

验证

- 测试程序常使用边界数据，如有效范围两端的数据、特殊的倍数等
  - 个位数；
  - 10；
  - 0；
  - 负数。

调试

```c
int x;
int n =0;

scanf("%d", &x);

n++;
x /= 10;
while ( x > 0 ) {
    printf("x=%d, n=%d\n", x, n);
    n++;
    x /= 10;
}

printf("%d\n", n);
```

- 在程序适当的地方插入printf来输出变量的内容

do-while循环

数位数的算法

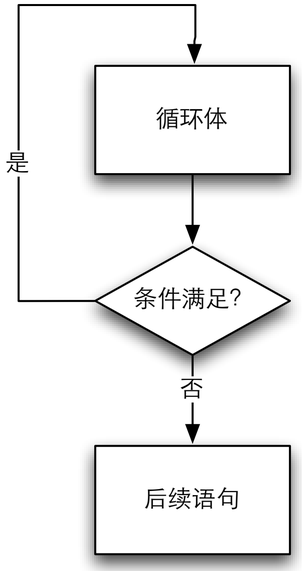
1. 用户输入x；
2. 初始化n为0；
3. x = x / 10，去掉个位；
4. n ++；
5. 如果x>0，回到3；
6. 否则n就是结果。

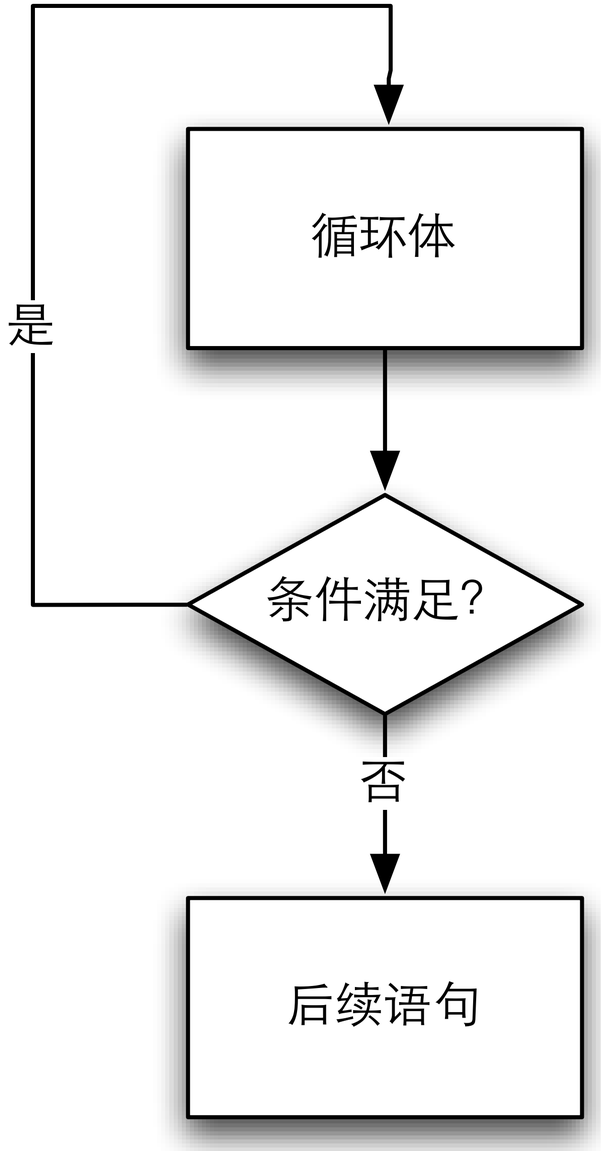
```c
int x;
int n =0;

scanf("%d", &x);

n++;
x /= 10;
while ( x > 0 ) {
    n++;
    x /= 10;
}

printf("%d\n", n);
```

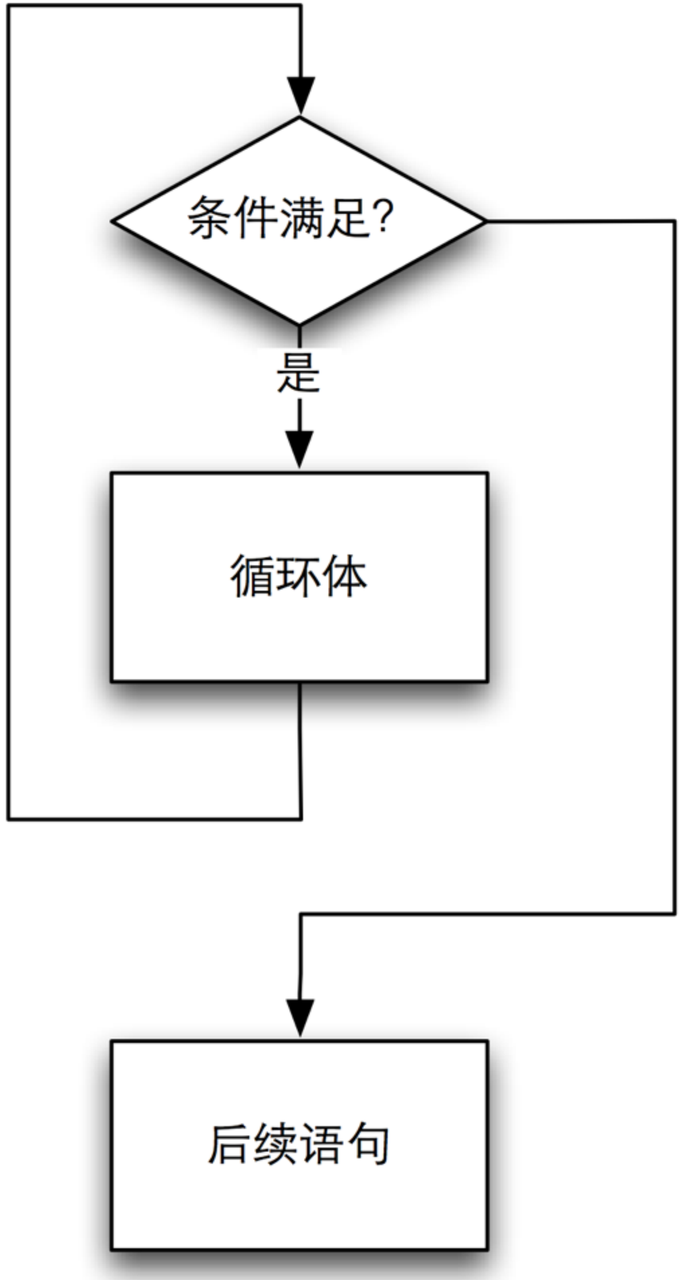
do-while循环

- 在进入循环的时候不做检查，而是在执行完一轮循环体的代码之后，再来检查循环的条件是否满足，如果满足则继续下一轮循环，不满足则结束循环

```
do
{
    <循环体语句>
} while ( <循环条件> );
```



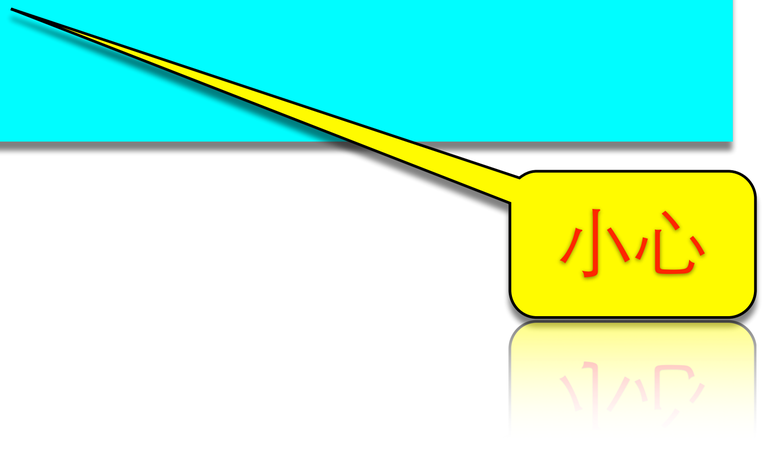

循环体
是
条件满足?
否
后续语句


条件满足?
是
循环体
后续语句

两种循环

- do-while循环和while循环很像，区别是在循环体执行结束的时候才来判断条件。也就是说，无论如何，循环都会执行至少一遍，然后再来判断条件。与while循环相同的是，条件满足时执行循环，条件不满足时结束循环。

```
int x;
scanf("%d", &x);
int n = 0;
do
{
  x /= 10;
  n ++;
} while ( x > 0 );
printf("%d", n);
```



循环计算

$\log_2 x$

```
int x;
int ret = 0;

scanf("%d", &x);
while ( x > 1 ) {
    x /= 2;
    ret ++;
}
printf("log2 of %d is %d.", x, ret);
```

这个log2x错哪儿了？

小套路

```
int x;
int ret = 0;

scanf("%d", &x);
int t = x;
while ( x > 1 ) {
    x /= 2;
    ret ++;
}
printf("log2 of %d is %d.", t, ret);
```

- 计算之前先保存原始的值，后面可能有用

这些值是怎么定的?

```c
int x;
int ret = 0;

scanf("%d", &x);
int t = x;
while ( x > 1 ) {
    x /= 2;
    ret ++;
}
printf("log2 of %d is %d.", t, ret);
```

为什么从0开始

为什么是>1

计数循环

```
int count =100;
while ( count >=0 ) {
    count --;
    printf("%d\n",count);
}
printf("发射！\n");
```

小套路：如果要模拟运行一个很大次数的循环，可以模拟较少的循环次数，然后作出推断。

- 这个循环需要执行多少次?
- 循环停下来的时候，有没有输出最后的0?
- 循环结束后，count的值是多少?

```
int count =10;
do {
    printf("%d ", count);
    count --;
} while ( count >0 );
printf("发射！\n");
```

循环应用

猜数游戏

- 让计算机来想一个数，然后让用户来猜，用户每输入一个数，就告诉它是大了还是小了，直到用户猜中为止，最后还要告诉用户它猜了多少次。
- 因为需要不断重复让用户猜，所以需要用到循环
- 在实际写出程序之前，我们可以先用文字描述程序的思路
- 核心重点是循环的条件
  - 人们往往会考虑循环终止的条件

1. 计算机随机想一个数，记在变量number里；
2. 一个负责计次数的变量count初始化为0；
3. 让用户输入一个数字a；
4. count递增（加一）；
5. 判断a和number的大小关系，如果a大就输出“大”；如果a小就输出“小”；
6. 如果a和number是不相等的（无论大还是小），程序转回到第3步；
7. 否则，程序输出“猜中”和次数，然后结束。

循环的条件是a和number不相等

```
srand(time(0));
int number = rand()%100+1;
int count = 0;
int a = 0;
printf("我已经想好了一个1到100之间的数。");
do {
  printf("请猜这个1到100之间数：");
  scanf("%d", &a);
  count ++;
  if ( a > number ) {
    printf("你猜的数大了。");
  } else if ( a < number ) {
    printf("你猜的数小了。");
  }
} while (a != number);
printf("太好了，你用了%d次就猜到了答案。\n", count);
```

随机数

- 每次召唤rand()就得到一个随机的整数

%100

- x % n 的结果是[0, n-1]的一个整数

```
srand(time(0));
int number = rand()%100+1;
int count = 0;
int a = 0;
printf("我已经想好了一个1到100之间的数。");
do {
  printf("请猜这个1到100之间数：");
  scanf("%d", &a);
  count ++;
  if ( a > number ) {
    printf("你猜的数大了。");
  } else if ( a < number ) {
    printf("你猜的数小了。");
  }
} while (a != number);
printf("太好了，你用了%d次就猜到了答案。\n", count);
```

算平均数

$$sum = \frac{\sum_{i=1}^{n} x_i}{n}$$

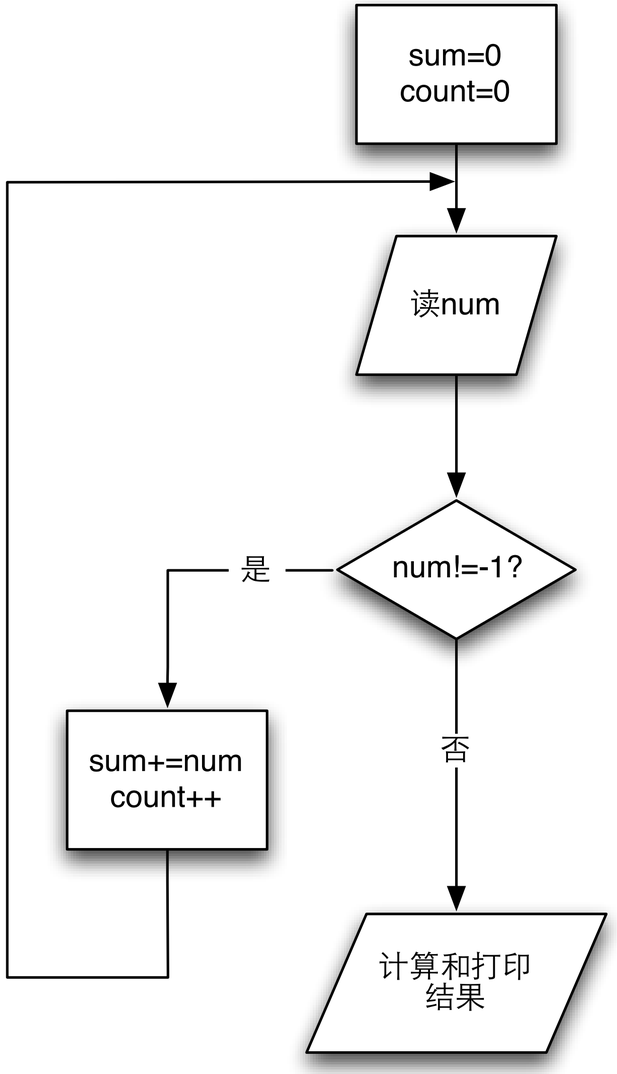
- 让用户输入一系列的正整数，最后输入-1表示输入结束，然后程序计算出这些数字的平均数，输出输入的数字的个数和平均数
- 变量->算法->流程图->程序

变量

- 一个记录读到的整数的变量
- 平均数要怎么算?
  - 只需要每读到一个数，就把它加到一个累加的变量里，到全部数据读完，再拿它去除读到的数的个数就可以了
- 一个变量记录累加的结果，一个变量记录读到的数的个数

算法

1. 初始化变量sum和count为0；
2. 读入number；
3. 如果number不是-1，则将number加入sum，并将count加1，回到2；
4. 如果number是-1，则计算和打印出sum / count（注意换成浮点来计算）。



```
number = 0;
count = 0;
while ( number != -1 ) {
   scanf("%d", &number);
   if ( number != -1 ) {
      sum += number;
      count ++;
   }
}
```

```
int sum = 0;
int count = -1;
int number = 0;
while ( number != -1 )
{
  sum += number;
  count ++;
  scanf("%d", &number);
}
```

```
int sum = 0;
int count = -1;
int number = 0;
do
{
    sum += number;
    count ++;
    scanf("%d", &number);
} while ( number != -1 );
```

整数的分解

- 一个整数是由1至多位数字组成的，如何分解出整数的各个位上的数字，然后加以计算
  - 对一个整数做%10的操作，就得到它的个位数；
  - 对一个整数做/10的操作，就去掉了它的个位数；
  - 然后再对2的结果做%10，就得到原来数的十位数了；
  - 依此类推。

数的逆序

- 输入一个正整数，输出逆序的数
  - 结尾的0的处理

for循环

阶乘

- n! = 1×2×3×4×…×n
- 写一个程序，让用户输入n，然后计算输出n!
- 变量：
  - 显然读用户的输入需要一个int的n，然后计算的结果需要用一个变量保存，可以是int的factor，在计算中需要有一个变量不断地从1递增到n，那可以是int的i

```c
int n;

scanf("%d", &n);
int fact = 1;

int i = 1;
while ( i <= n ) {
    fact *= i;
    i++;
}

printf("%d!=%d\n", n, fact);
```

```c
int n;

scanf("%d", &n);
int fact = 1;

int i = 1;
for ( i=1; i<=n; i++ ) {
    fact *= i;
}

printf("%d!=%d\n", n, fact);
```

for循环

for循环像一个计数循环：设定一个计数器，初始化它，然后在计数器到达某值之前，重复执行循环体，而每执行一轮循环，计数器值以一定步进进行调整，比如加1或者减1

```
for ( i=0; i<5; i=i+1 ) {
    printf("%d", i);
}
```

for = 对于

- for (count=10; count>0; count--)

- 就读成：“对于一开始的count=10，当count>0时，重复做循环体，每一轮循环在做完循环体内语句后，使得count--。”

小套路

- 做求和的程序时，记录结果的变量应该初始化为0，而做求积的变量时，记录结果的变量应该初始化为1

- 循环控制变量i只在循环里被使用了，在循环外面它没有任何用处。因此，我们可以把变量i的定义写到for语句里面去

```
int n;

scanf("%d", &n);
int fact = 1;
for ( int i=1; i<=n; i++ ) {
    fact *= i;
}

printf("%d!=%d\n", n, fact);
```

C99 ONLY!

try

- 1×1还是1，所以程序的循环不需要从1开始，那么改成从多少开始合适呢？这样修改之后，程序对所有的n都正确吗？这样的改动有价值吗?
- 除了可以从1乘到n来计算n!，还可以从n乘到1来计算吧？试试把循环换个方向来计算n。这时候，还需要循环控制变量i吗?

```c
int n;

scanf("%d", &n);
int fact = 1;

int i = 1;
while ( i <= n ) {
    fact *= i;
    i++;
}

printf("%d!=%d\n", n, fact);
```

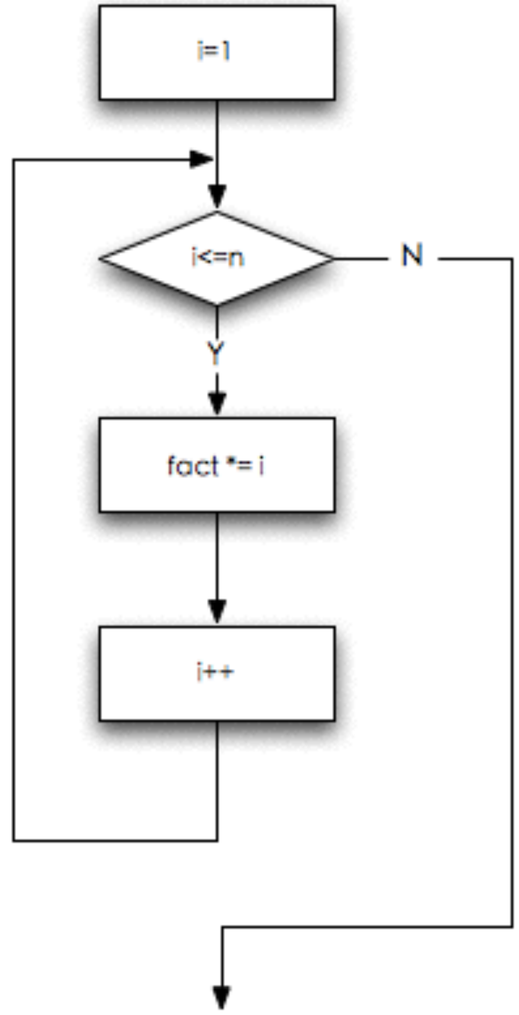
```c
int n;

scanf("%d", &n);
int fact = 1;

int i = 1;
for ( i=1; i<=n; i++ ) {
    fact *= i;
}

printf("%d!=%d\n", n, fact);
```

for == while

```
for ( int i=1; i<=n; i++ ) {
    fact *= i;
}
```

=

```
int i=1;
while (i<=n) {
    fact *=i;
    i++;
}
```



for循环

for (初始动作; 条件; 每轮的动作) {

}

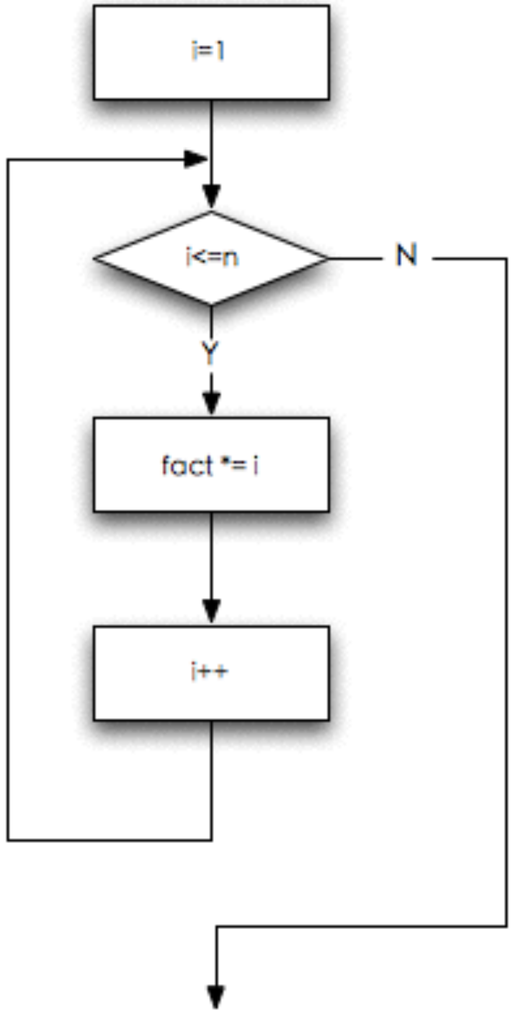
- for中的每一个表达式都是可以省略的

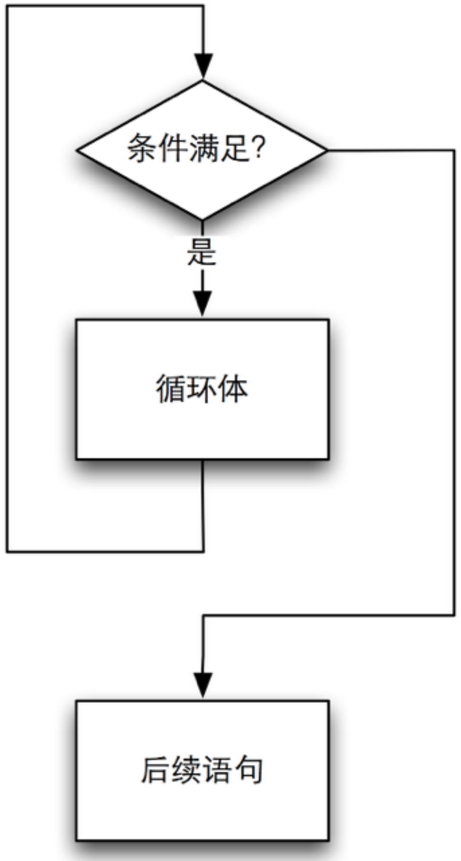
for (; 条件;) == while (条件)

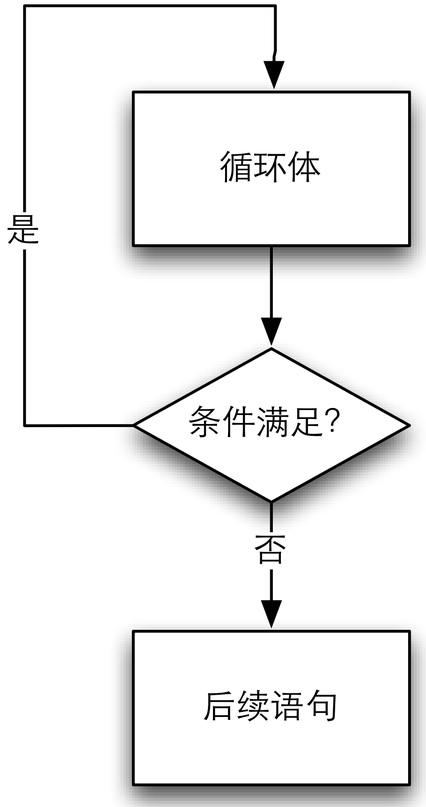
循环次数

- for (i=0; i<n; i++)
- 则循环的次数是n，而循环结束以后，i的值是n。循环的控制变量i，是选择从0开始还是从I开始，是判断i<n还是判断i<=n，对循环的次数，循环结束后变量的值都有影响

三种循环







Tips for loops

- 如果有固定次数，用for
- 如果必须执行一次，用do_while
- 其他情况用while

循环控制

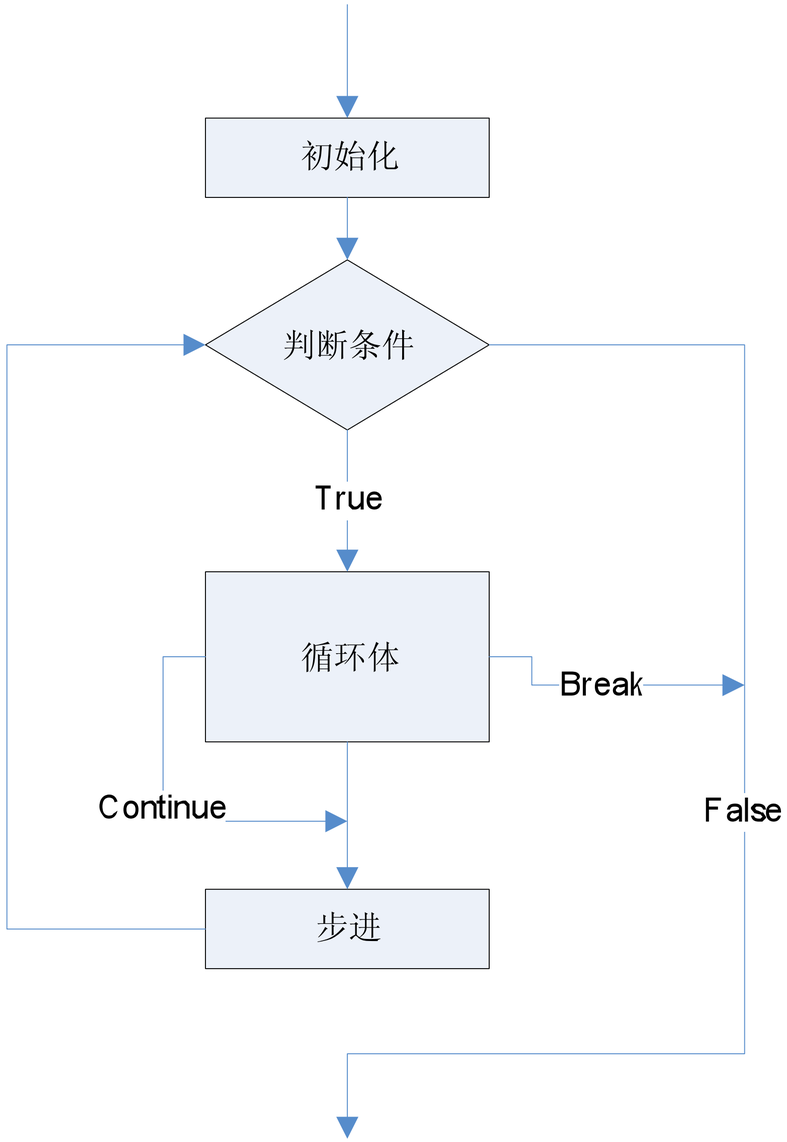
素数

- 只能被1和自己整除的数，不包括1
  - 2, 3, 5, 7, 11,13,17, 19….

break vs continue

- break: 跳出循环
- continue: 跳过循环这一轮剩下的语句进入下一轮

break vs continue



嵌套的循环

100以内的素数

- 如何写程序输出100以内的素数?

嵌套的循环

- 循环里面还是循环

前50个素数

- 如何写程序输出前50个素数

嵌套循环时的break

凑硬币

- 如何用1角、2角和5角的硬币凑出10元以下的金额呢?

```c
int x;
int one, two, five;

scanf("%d", &x);
for ( one = 1; one < x*10; one++ ) {
    for ( two = 1; two < x*10/2; two++ ) {
        for ( five = 1; five < x*10/5; five++ ) {
            if ( one + two*2 + five*5 == x*10 ) {
                printf("可以用%d个1角加%d个2角加%d个5角得到%d元\n",
                    one, two, five, x);
            }
        }
    }
}
```

break和continue

- 只能对它所在的那层循环做

接力break

```c
int x;
int one, two, five;
int exit = 0;

scanf("%d", &x);
for ( one = 1; one < x*10; one++ ) {
    for ( two = 1; two < x*10/2; two++ ) {
        for ( five = 1; five < x*10/5; five++ ) {
            if ( one + two*2 + five*5 == x*10 ) {
                printf("可以用%d个1角加%d个2角加%d个5角得到%d元\n",
                    one, two, five, x);
                exit = 1;
                break;
            }
        }
        if ( exit == 1 ) break;
    }
    if ( exit == 1 ) break;
}
```

goto

```
    int x;
    int one, two, five;

    scanf("%d", &x);
    for ( one = 1; one < x*10; one++ ) {
        for ( two = 1; two < x*10/2; two++ ) {
            for ( five = 1; five < x*10/5; five++ ) {
                if ( one + two*2 + five*5 == x*10 ) {
                    printf("可以用%d个1角加%d个2角加%d个5角得到%d元\n",
                        one, two, five, x);
                    goto out;
                }
            }
        }
    }
out:
    return 0;
```

循环应用

$$f(n) = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \ldots + + \frac{1}{n}$$

```c
int n;
int i;
double ret=0.0;

scanf("%d", &n);
for ( i=1; i<=n; i++ ) {
    ret += 1.0/i;
}
printf("%f\n", ret);
```

$$f(n) = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \ldots + \frac{1}{n}$$

```c
int n;
int i;
double ret=0.0;
int sign = 1;

scanf("%d", &n);
for ( i=1; i<=n; i++ ) {
    ret += 1.0*sign/i;
    sign = -sign;
}
printf("%f\n", ret);
```

正序分解整数

- 输入一个非负整数，正序输出它的每一位数字
- 输入：13425
- 输出：1 3 4 2 5

分解整数输出

```c
int x;
scanf("%d", &x);

do {
    int d = x % 10;
    printf("%d ", d);
    x /= 10;
} while ( x > 0 );
```

- 还没解决结尾的空格问题!

分解整数输出

```c
int x;
scanf("%d", &x);

do {
    int d = x % 10;
    printf("%d", d);
    if ( x>9 ) {
        printf(" ");
    }
    x /= 10;
} while ( x > 0 );
printf("\n");
```

- 但是是逆序的!

分解整数输出

```c
int x;
scanf("%d", &x);

x = 13425;
int mask = 10000;
do {
    int d = x / mask;
    printf("%d", d);
    if ( mask > 9 ) {
        printf(" ");
    }
    x %= mask;
    mask /= 10;
} while ( mask>0 );

printf("\n");
```

- 如果能有这么一个mask

分解整数输出

```
x = 12345;
int mask = 10000;
int n=0;
do {
    x /= 10;
    n++;
} while ( x > 0 );
printf("n=%d\n", n);
```

- 计算x的位数

分解整数输出

```
x = 12345;
int mask = 10000;
int n=0;
do {
    x /= 10;
    n++;
} while ( x > 0 );
printf("n=%d\n", n);
mask = pow(10,n);
printf("mask=%d\n", mask);
```

- pow?
  - #include <math.h>
  - pow是浮点运算，慢

分解整数输出

```
x = 12345;
int mask = 1;
do {
     x /= 10;
     mask *=10;
} while ( x > 0 );
printf("mask=%d\n", mask);
```

- 直接算mask
  - mask=100000?
  - 因为第一轮mask就是10了
- 怎么办?

分解整数输出

```
x = 12345;
int mask = 1;
do {
    x /= 10;
    mask *=10;
} while ( x > 9 );
```

- 改变循环的条件，让它少做一轮

分解整数输出

```
x = 12345;
int mask = 1;
do {
    x /= 10;
    mask *=10;
} while ( x > 9 );
```

- 改变循环的条件，让它少做一轮
- 但是最后的结果为什么不对?

```
mask=10000
00001[Finished in 0.1s]
```

分解整数输出

```
int x;
scanf("%d", &x);

x = 13425;
int mask = 1;
int t = x;
do {
    t/=10;
    mask *= 10;
} while (t>9);
printf("mask=%d\n", mask);
do {
    int d = x / mask;
    printf("%d", d);
    if ( mask > 9 ) {
        printf(" ");
    }
    x %= mask;
    mask /= 10;
} while ( mask>0 );
```

- 因为x在第一个循环中被改变了
- 需要用另外的变量代替x做计算

求最大公约数

- 输入两个数a和b，输出它们的最大公约数
- 输入：12 18
- 输出：6

```c
int a,b;
int min;

scanf("%d %d", &a, &b);
if ( a<b ) {
    min = a;
} else {
    min = b;
}
int ret = 0;
int i;
for ( i = 1; i < min; i++ ) {
    if ( a%i == 0 ) {
        if ( b%i == 0 ) {
            ret = i;
        }
    }
}
printf("%d和%d的最大公约数是%d.\n", a, b, ret);
```

枚举

1. 设t为2；
2. 如果u和v都能被t整除，则记下这个t
3. t加1后重复第2步，直到t等于u或v；
4. 那么，曾经记下的最大的可以同时整除u和v的t就是gcd

辗转相除法

1. 如果b等于0，计算结束，a就是最大公约数；
2. 否则，计算a除以b的余数，让a等于b，而b等于那个余数；
3. 回到第一步。

辗转相除法

```
int a,b;
int t;

scanf("%d %d", &a, &b);
int origa = a;
int origb = b;
while ( b != 0 ) {
    t = a%b;
    a = b;
    b = t;
}
printf("%d和%d的最大公约数是%d.\n", origa, origb, a);
```

数据类型

C是有类型的语言

- C语言的变量，必须：
  - 在使用前定义，并且
  - 确定类型
- C以后的语言向两个方向发展：
  - C++/Java更强调类型，对类型的检查更严格
  - JavaScript、Python、PHP不看重类型，甚至不需要事先定义

类型安全

- 支持强类型的观点认为明确的类型有助于尽早发现程序中的简单错误
- 反对强类型的观点认为过于强调类型迫使程序员面对底层、实现而非事务逻辑
- 总的来说，早期语言强调类型，面向底层的语言强调类型
- C语言需要类型，但是对类型的安全检查并不足够

C语言的类型

- 整数
  - char、short、int、long、long long
- 浮点数
  - float、double、long double
- 逻辑
  - bool
- 指针
- 自定义类型

蓝色的是C99的类型

类型有何不同

- 类型名称：int、long、double
- 输入输出时的格式化：%d、%ld、%lf
- 所表达的数的范围：char < short < int < float < double
- 内存中所占据的大小：1个字节到16个字节
- 内存中的表达形式：二进制数（补码）、编码

sizeof

- 是一个运算符，给出某个类型或变量在内存中所占据的字节数
  - sizeof(int)
  - sizeof(i)

sizeof

- 是静态运算符，它的结果在编译时刻就决定了
- 不要在sizeof的括号里做运算，这些运算不会做的

整数

整数

- char
- short
- int
- long
- long long

整数

- char：1字节（8比特）
- short：2字节
- int：取决于编译器（CPU），通常的意义是“1个字”
- long：取决于编译器（CPU），通常的意义是“1个字”
- long long：8字节

*整数的内部表达

- 计算机内部一切都是二进制
  - 18 —> 00010010
  - 0 —> 00000000
  - -18 —> ?

*如何表示负数

- 十进制用“-”来表示负数，在做计算的时候
  - 加减是做相反的运算
  - 乘除时当作正数，计算完毕后对结果的符号取反

*二进制负数

- 1个字节可以表达的数：
  - 00000000 — 11111111 (0-255)
- 三种方案：
  1. 仿照十进制，有一个特殊的标志表示负数
  2. 取中间的数为0，如1000000表示0，比它小的是负数，比它大的是正数
  3. 补码

*补码

- 考虑-1，我们希望-1 + 1 —> 0。如何能做到?
  - 0 —> 00000000
  - 11111111 + 00000001 —> 100000000
- 因为0 - 1 —> -1，所以，-1 =
  - (1)00000000 – 00000001 —> 11111111
  - 11111111被当作纯二进制看待时，是255，被当作补码看待时是-1
- 同理，对于-a，其补码就是0-a，实际是2^n - a，n是这种类型的位数

补码的意义就是拿补码和原码可以加出一个溢出的“零”

数的范围

- 对于一个字节（8位），可以表达的是：
  - 00000000 - 11111111
- 其中
  - 00000000 —> 0
  - 11111111 ~ 10000000 —> -1 ~ -128
  - 00000001 ~ 01111111 —> 1 ~ 127

整数的范围

- char：1字节：-128 ~ 127
- short：2字节：-32768 ~ 32767
- int：取决于编译器（CPU），通常的意义是“1个字”
- long：4字节
- long long：8字节

unsigned

- 在整形类型前加上unsigned使得它们成为无符号的整数
- 内部的二进制表达没变，变的是如何看待它们
  - 如何输出
- 11111111
  - 对于char，是-1
  - 对于unsigned char，是255

unsigned

- 如果一个字面量常数想要表达自己是unsigned，可以在后面加u或U
  - 255U
- 用l或L表示long(long)
- *unsigned的初衷并非扩展数能表达的范围，而是为了做纯二进制运算，主要是为了移位

整数越界

- 整数是以纯二进制方式进行计算的，所以：
  - 11111111 + 1 —> 100000000 —> 0
  - 01111111 + 1 —> 10000000 —> -128
  - 10000000 - 1 —> 01111111 —> 127

整数的输入输出

- 只有两种形式：int或long long
  - %d：int
  - %u：unsigned
  - %lld：long long
  - %llu：unsigned long long

8进制和16进制

- 一个以0开始的数字字面量是8进制
- 一个以0x开始的数字字面量是16进制
- %o用于8进制，%x用于16进制

- 8进制和16进制只是如何把数字表达为字符串，与内部如何表达数字无关

* 8进制和16进制

- 16进制很适合表达二进制数据，因为4位二进制正好是一个16进制位
- 8进制的一位数字正好表达3位二进制
  - 因为早期计算机的字长是12的倍数，而非8

选择整数类型

- 为什么整数要有那么多种?
  - 为了准确表达内存，做底层程序的需要
- 没有特殊需要，就选择int
  - 现在的CPU的字长普遍是32位或64位，一次内存读写就是一个int，一次计算也是一个int，选择更短的类型不会更快，甚至可能更慢
  - * 现代的编译器一般会设计内存对齐，所以更短的类型实际在内存中有可能也占据一个int的大小（虽然sizeof告诉你更小）
- unsigned与否只是输出的不同，内部计算是一样的

浮点数

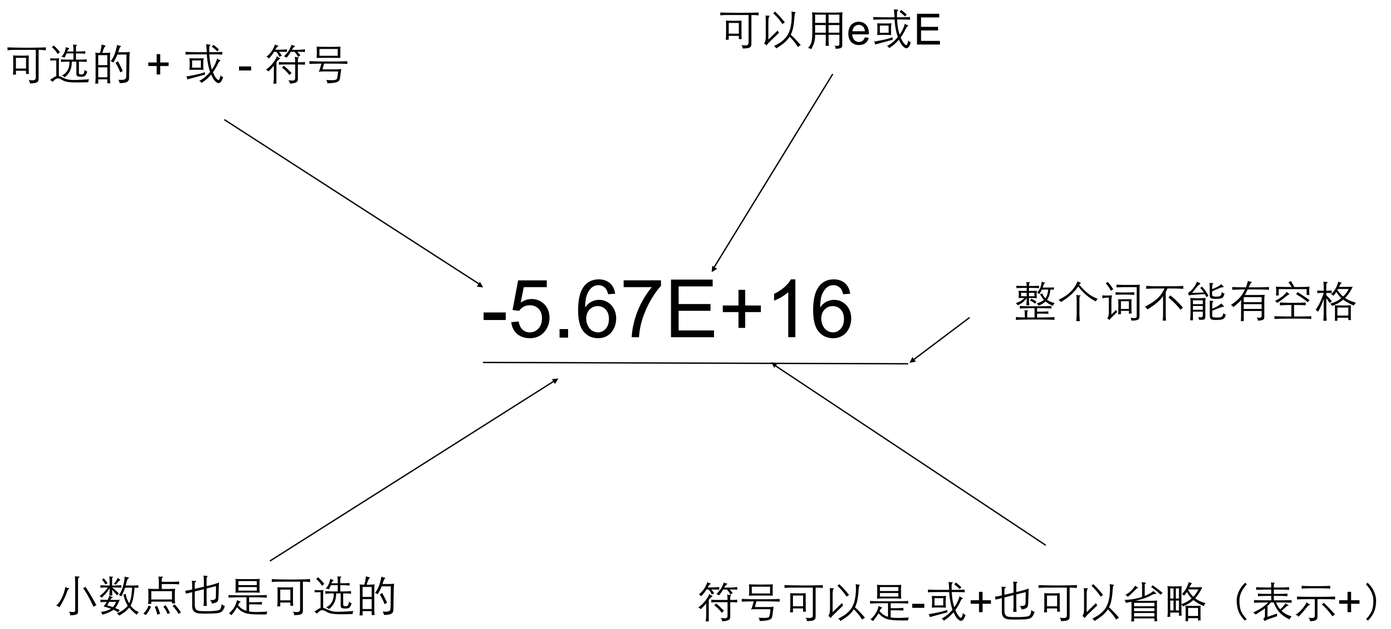
浮点类型

类型	字长	范围	有效数字
float	32	±(1.20x10 0,±inf,nan	7
double	64	±(2.2x10 0,±inf,nan	15

浮点的输入输出

类型	scanf	printf
float	%f	%f, %e
double	%lf	%f,%e

科学计数法



输出精度

- 在%和f之间加上.n可以指定输出小数点后几位，这样的输出是做4舍5入的
  - printf("%.3f\n", -0.0049);
  - printf("%.30f\n", -0.0049);
  - printf("%.3f\n", -0.00049);

超过范围的浮点数

- printf输出inf表示超过范围的浮点数：$\pm\infty$
- printf输出nan表示不存在的浮点数

浮点运算的精度

```c
float a, b, c;

a = 1.345f;
b = 1.123f;
c = a + b;
if (c == 2.468)
    printf("相等\n");
else
    printf("不相等！ c=%.10f，或%f\n",c,c);
```

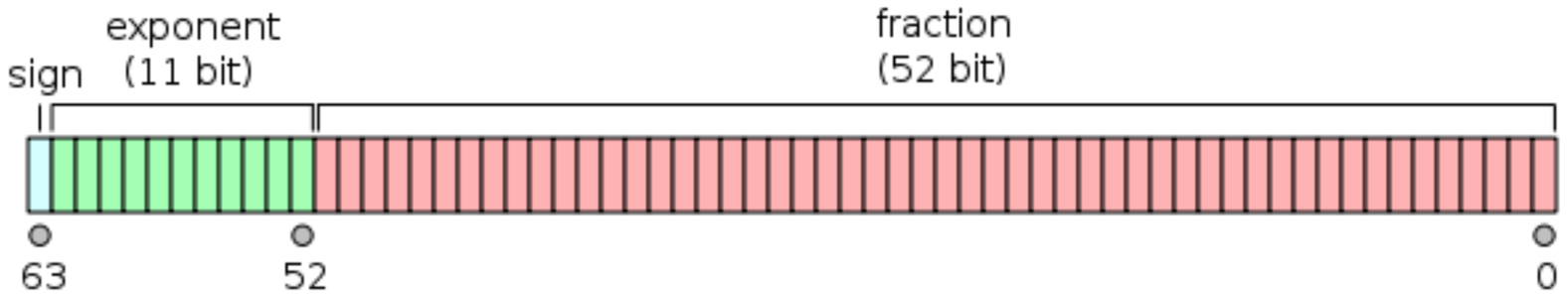
- 带小数点的字面量是double而非float
- float需要用f或F后缀来表明身份

Android计算器低级错误？都是二进制惹的祸！

- f1 == f2可能失败
- fabs(f1-f2) < 1e-12

http://www.guokr.com/article/27173/

* 浮点数的内部表达



- 浮点数在计算时是由专用的硬件部件实现的
- 计算double和float所用的部件是一样的

图片来源：wikipedia.org

选择浮点类型

- 如果没有特殊需要，只使用double
- 现代CPU能直接对double做硬件运算，性能不会比float差，在64位的机器上，数据存储的速度也不比float慢

字符

字符类型

- char是一种整数，也是一种特殊的类型：字符。这是因为：
  - 用单引号表示的字符字面量：'a', '1'
  - ''也是一个字符
  - printf和scanf里用%c来输入输出字符

字符的输入输出

- 如何输入'1'这个字符给char c?
  - scanf("%c", &c);—>1
  - scanf("%d", &i); c=i; —>49
- '1'的ASCII编码是49，所以当c==49时，它代表'1'
  - printf("%i %c\n", c,c);
- 一个49各自表述!

混合输入

- 有何不同?
  - scanf("%d %c", &i, &c);
  - scanf("%d%c", &i, &c);

字符计算

```c
char c = 'A';
c++;
printf("%c\n", c);
```

```c
int i = 'Z' - 'A';
printf("%d\n", i);
```

- 一个字符加一个数字得到ASCII码表中那个数之后的字符
- 两个字符的减，得到它们在表中的距离

大小写转换

- 字母在ASCII表中是顺序排列的
- 大写字母和小写字母是分开排列的，并不在一起
- ‘a’-‘A’可以得到两段之间的距离，于是
  - a+’a’-‘A”可以把一个大写字母变成小写字母，而
  - a+’A’-‘a’可以把一个小写字母变成大写字母

逃逸字符

- 用来表达无法印出来的控制字符或特殊字

```
printf("请分别输入身高的英尺和英寸，"
    "如输入\"5 7\"表示5英尺7英寸: ");
```

个字符

逃逸字符

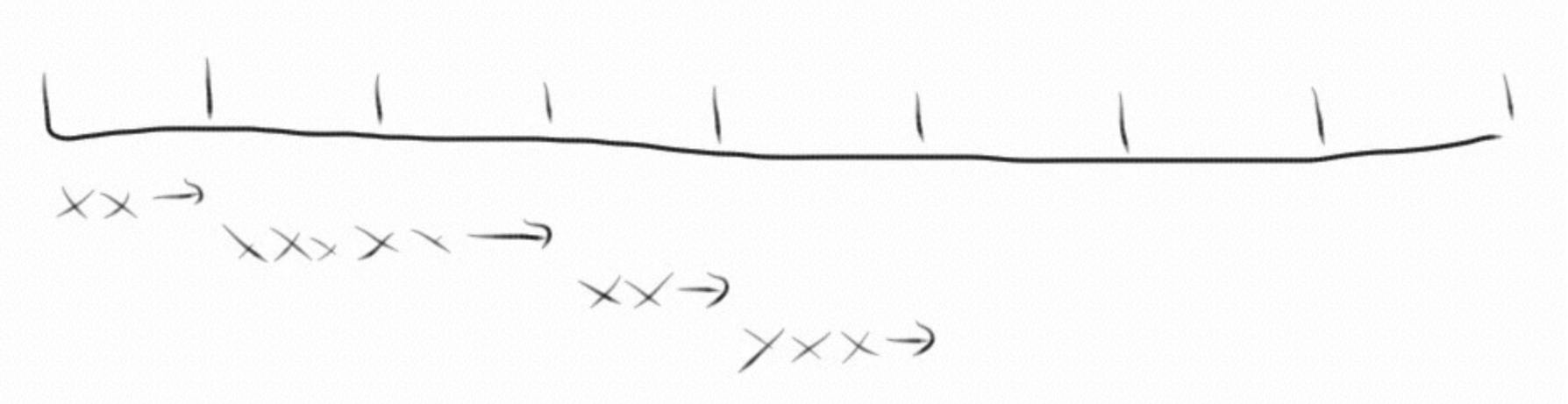
字符	意义	字符	意义
\b	回退一格	\"	双引号
\t	到下一个表格位	\'	单引号
\n	换行	\\	反斜杠本身
\r	回车		

回车换行

- 源自打字机的动作

回车换行

制表位



- 每行的固定位置
- 一个\t使得输出从下一个制表位开始
- 用\t才能使得上下两行对齐

逻辑类型

bool

- #include <stdbool.h>
- 之后就可以使用bool和true、false

bool的运算

- bool实际上还是以int的手段实现的，所以可以当作int来计算
- 也只能当作int来输入输出

类型转换

自动类型转换

- 当运算符的两边出现不一致的类型时，会自动转换成较大的类型
  - 大的意思是能表达的数的范围更大
  - char —> short —> int —> long —> long long
  - int —> float —> double

自动类型转换

- 对于printf，任何小于int的类型会被转换成int；float会被转换成double
- 但是scanf不会，要输入short，需要%hd

强制类型转换

- 要把一个量强制转换成另一个类型（通常是较小的类型），需要：
  - (类型)值
- 比如：
  - (int)10.2
  - (short)32
- 注意这时候的安全性，小的变量不总能表达大的量
  - (short)32768

- 只是从那个变量计算出了一个新的类型的值，它并不改变那个变量，无论是值还是类型都不改变

强制类型转换

```
double a = 1.0;
double b = 2.0;
int i = (int)a / b;
```

```
int i = (int)(a / b);
```

- 强制类型转换的优先级高于四则运算

```
int a = 5;
int b = 6;
double d = (double)(a / b);
```

逻辑运算

逻辑运算

- 逻辑运算是对逻辑量进行的运算，结果只有0或1
- 逻辑量是关系运算或逻辑运算的结果

运算符	描述	示例	结果
!	逻辑非	!a	如果a是true结果就是false，如果a是false结果就是true
&&	逻辑与	a && b	如果a和b都是true，结果就是true；否则就是false
||	逻辑或	a || b	如果a和b有一个是true，结果为true；两个都是false，结果为false

TRY

```
x > 4 && x < 6
```

- 如果要表达数学中的区间，如：x ∈(4,6)或x ∈[4,6]，应该如何写C的表达式？

像 4 < x < 6这样的式子，不是C能正确计算的式子，因为4 < x的结果是一个逻辑值（0或1）

- 如何判断一个字符c是否是大写字母?
  - c >= ‘A’ && c<= ‘Z’

理解一下

- age > 20 && age < 30
- index <0 || index > 99
- ! age<20

优先级

- ! > && > ||
  - !done && (count > MAX)

优先级

优先级	运算符	结合性
1	()	从左到右
2	! + - ++ --	从右到左（单目的+和-）
3	* / %	从左到右
4	+ -	从左到右
5	< <= > >=	从左到右
6	== !=	从左到右
7	&&	从左到右
8	||	从左到右
9	= += -= *= /= %=	从右到左

```
for ( i = a; i<=a+3; i++ ) {
    for ( j=a; j<=a+3; j++ ) {
        for ( k=a; k<=a+3; k++ ) {
            if ( i!=j ) {
                if ( i!=k ) {
                    if ( j!=k ) {
                        cnt++;
                        printf("%d%d%d",i,j,k);
                        if ( cnt == 6 ) {
                            printf("\n");
                            cnt = 0;
                        } else {
                            printf(" ");
                        }
                    }
                }
            }
        }
    }
}
```

```
for ( i = a; i<=a+3; i++ ) {
    for ( j=a; j<=a+3; j++ ) {
        for ( k=a; k<=a+3; k++ ) {
            if ( i!=j && i!=k && j!=k ) {
                cnt++;
                printf("%d%d%d",i,j,k);
                if ( cnt == 6 ) {
                    printf("\n");
                    cnt = 0;
                } else {
                    printf(" ");
                }
            }
        }
    }
}
```

```
if ( gameover == 0 ) {
    if ( player2move == 2 ) {
        printf("Your turn\n");
    }
}
```

```
if ( gameover == 0 && player2move == 2 ) {
    printf("Your turn\n");
}
```

```
if ( gameover == 0 ) {
    if ( player2move == 2 ) {
        printf("Your turn\n");
    } else {
        printf("My turn\n");
    }
}
```

?

短路

- 逻辑运算是自左向右进行的，如果左边的结果已经能够决定结果了，就不会做右边的计算
  - a==6 && b==1
  - a==6 && b+=1
- 对于&&，左边是false时就不做右边了
- 对于||，左边是true时就不做右边了

不要把赋值，包括复合赋值组合进表达式！

条件运算

条件运算符

- count = (count > 20) ? count -10 : count +10;
- 条件、条件满足时的值和条件不满足时的值

```
if ( count >20 )
  count = count-10;
else
  count = count+10;
```

优先级

- 条件运算符的优先级高于赋值运算符，但是低于其他运算符

```
m<n ? x : a+5
a++ >= 1 && b-- > 2 ? a : b
x=3 * a > 5 ? 5 : 20
```

嵌套条件表达式

- count = (count > 20) ? (cout < 50) ? count - 10 : count - 5 : (count < 10) ? count + 10 : count + 5;
- 条件运算的嵌套
  - w < x ? x + w : x < y ? x : y

我们不希望你使用嵌套的条件表达式

逗号运算

逗号运算符

- 逗号用来连接两个表达式，并以其右边的表达式的值作为它的结果。逗号的优先级是所有的运算符中最低的，所以它两边的表达式会先计算；逗号的组合关系是自左向右，所以左边的表达式会先计算，而右边的表达式的值就留下来作为逗号运算的结果。

在for中使用,

- for (i=0, j=10; i<j; i++, j- -) ……

函数

素数求和

```c
scanf("%d %d", &m, &n);
// m=10,n=31;
if ( m==1 ) m=2;
for ( i=m; i<=n; i++ ) {
    int isPrime = 1;
    int k;
    for ( k=2; k<i-1; k++ ) {
        if ( i%k == 0 ) {
            isPrime = 0;
            break;
        }
    }
    if ( isPrime ) {
        sum += i;
        cnt++;
    }
}
printf("%d %d\n", cnt, sum);
```

```c
int isPrime(int i)
{
    int ret = 1;
    int k;
    for ( k=2; k<i-1; k++ ) {
        if ( i%k == 0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```

```c
scanf("%d %d", &m, &n);
// m=10,n=31;
if ( m==1 ) m=2;
for ( i=m; i<=n; i++ ) {
    if ( isPrime(i) ) {
        sum += i;
        cnt++;
    }
}
printf("%d %d\n", cnt, sum);
```

求和

- 求出1到10、20到30和35到45的三个和

求和

```
int i;
int sum;

for ( i=1,sum=0; i<=10; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n",
```

```
for ( i=20,sum=0; i<=30; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n", 20, 30, sum);
```

```
for ( i=35,sum=0; i<=45; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n", 35, 45, sum);
```

“代码复制”是程序质量不良的表现

- 三段几乎一模一样的代码!

求和函数

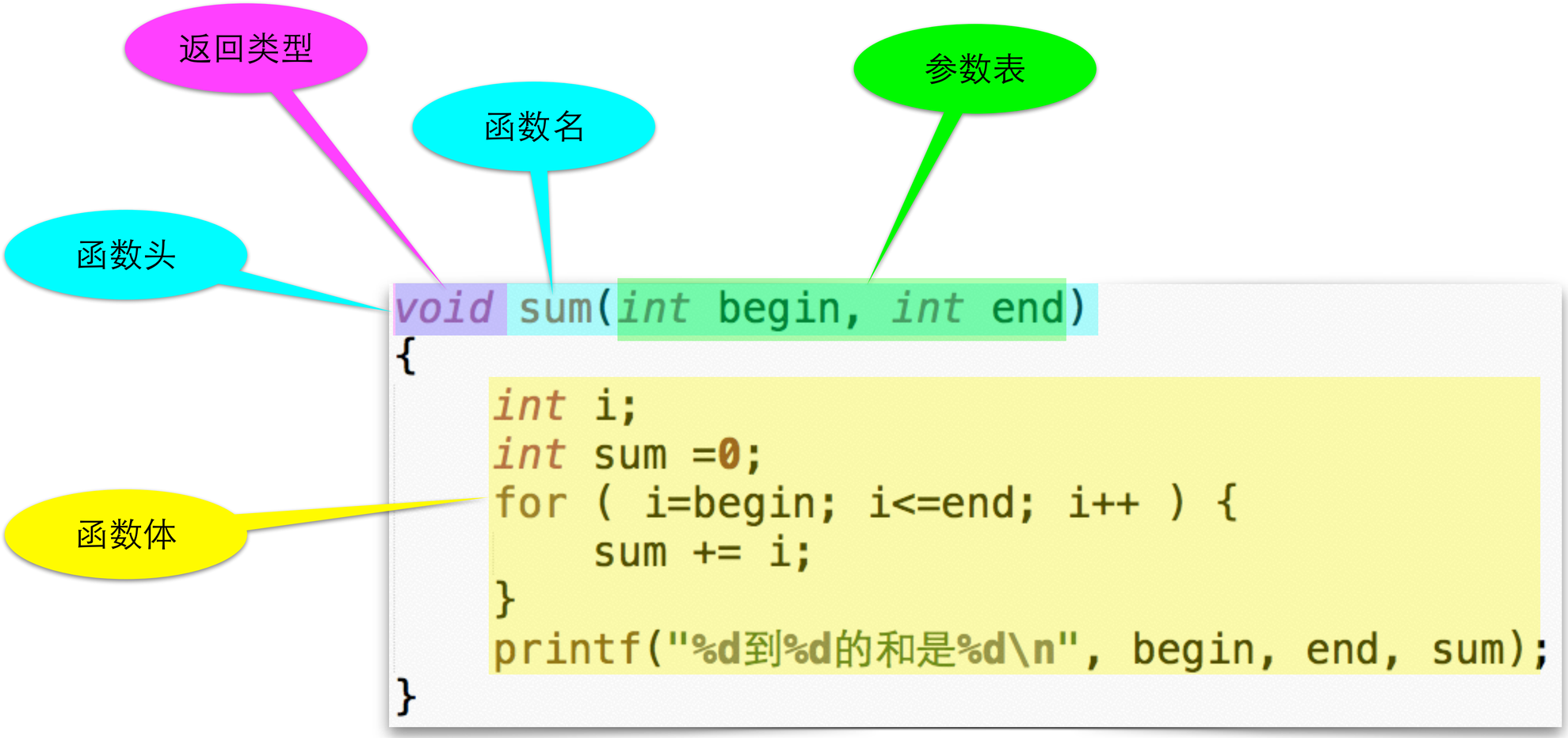
```c
int i;
int sum;

for ( i=1,sum=0; i<=10; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n", 1, 10, s

for ( i=20,sum=0; i<=30; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n", 20, 30,

for ( i=35,sum=0; i<=45; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n", 35, 45,
```

```c
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}

int main()
{
    sum(1,10);
    sum(20,30);
    sum(35,45);

    return 0;
}
```

什么是函数?

- 函数是一块代码，接收零个或多个参数，做一件事情，并返回零个或一个值
- 可以先想像成数学中的函数:
  - y = f(x)

函数定义


返回类型
函数名
参数表
函数头
函数体
void sum(int begin, int end)
{
int i;
int sum =0;
for (i=begin; i<=end; i++) {
sum += i;
}
printf("%d到%d的和是%d\n", begin, end, sum);
}

调用函数

- 函数名(参数值);
- ()起到了表示函数调用的重要作用
  - 即使没有参数也需要()
- 如果有参数，则需要给出正确的数量和顺序
- 这些值会被按照顺序依次用来初始化函数中的参数

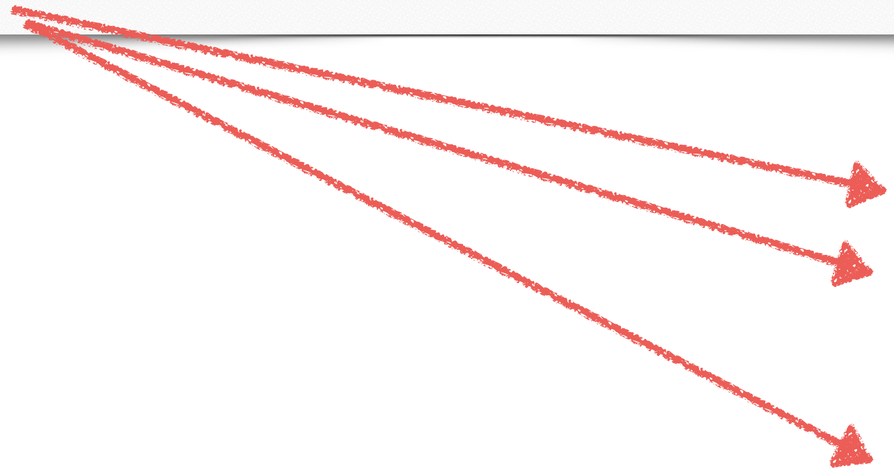
```
sum(1,10);
sum(20,30);
sum(35,45);
```

```
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}
```

函数返回

- 函数知道每一次是哪里调用它，会返回到正确的地方

```
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}
```



```
sum(1,10);
sum(20,30);
sum(35,45);

return 0;
```

从函数中返回值

```
int max(int a, int b)
{
    int ret;
    if ( a>b ) {
        ret = a;
    } else {
        ret = b;
    }

    return ret;
}
```

- **return**停止函数的执行，并送回一个值
  - return;
  - return 表达式;
- 一个函数里可以出现多个return语句

从函数中返回值

```c
int max(int a, int b)
{
    int ret;
    if ( a>b ) {
        ret = a;
    } else {
        ret = b;
    }

    return ret;
}
```

```c
int a,b,c;
a = 5;
b = 6;
c = max(10,12);
c = max(a,b);
c = max(c, 23);
c = max(max(c,a), 5);
printf("%d\n", max(a,b));
max(12,13);
```

- 可以赋值给变量
- 可以再传递给函数
- 甚至可以丢弃
  - 有的时候要的是副作用

没有返回值的函数

- void 函数名(参数表)
- 不能使用带值的return
  - 可以没有return
- 调用的时候不能做返回值的赋值

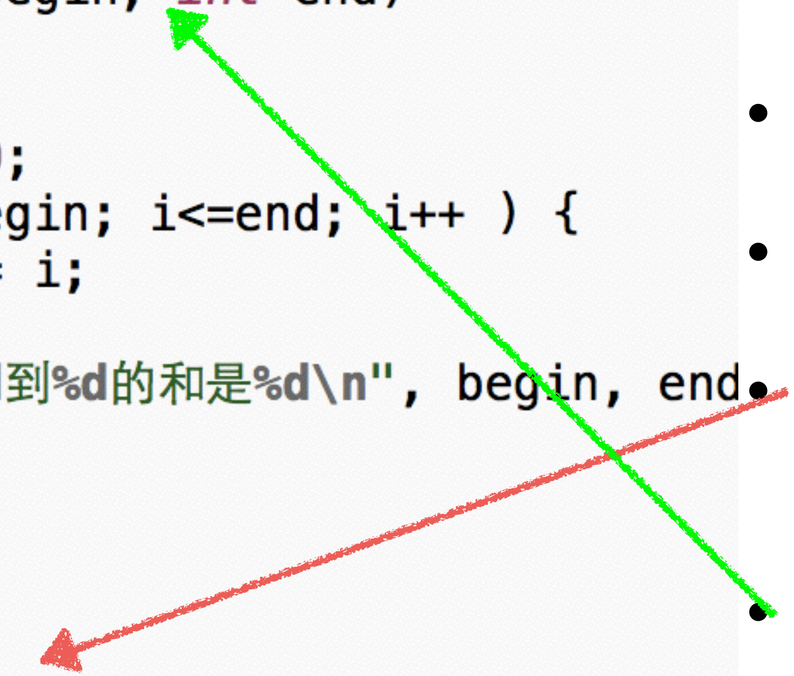
如果函数有返回值，则必须使用带值的return

```
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}
```

函数原型

函数先后关系

```c
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}

int main()
{
    sum(1,10);
    sum(20,30);
    sum(35,45);

    return 0;
}
```



- 像这样把sum()写在上面，是因为：
- C的编译器自上而下顺序分析你的代码
- 在看到sum(1,10)的时候，它需要知道sum()的样子
- 也就是sum()要几个参数，每个参数的类型如何，返回什么类型
- 这样它才能检查你对sum()的调用是否正确

如果不知道

- 也就是把要调用的函数放到下面了
- 旧标准会假设你所调用的函数所有的参数都是int，返回也是int
- 如果恰好不对...

```
    int a,b,c;
    a = 5;
    b = 6;
    c = max(10,12);
    printf("%d\n", c);
    max(12,13);

    return 0;
}

double max(double a, double b)
{
```

```
/Users/wengkai/cc/7.4.c:15:8: error: conflicting types for 'max'
double max(double a, double b)
       ^
```

函数原型

- 函数头，以分号“;”结尾，就构成了函数的原型
- 函数原型的目的是告诉编译器这个函数长什么样
  - 名称
  - 参数（数量及类型）
  - 返回类型
- 旧标准习惯把函数原型写在调用它的函数里面
- 现在一般写在调用它的函数前面
- 原型里可以不写参数的名字，但是一般仍然写上

函数原型

```
double max(double a, double b);

int main()
{
    int a,b,c;
    a = 5;
    b = 6;
    c = max(10,12);
    printf("%d\n", c);
    max(12,13);

    return 0;
}

double max(double a, double b)
```

根据原型判断

实际的函数头

参数传递

调用函数

- 如果函数有参数，调用函数时必须传递给它数量、类型正确的值
- 可以传递给函数的值是表达式的结果，这包括：
  - 字面量
  - 变量
  - 函数的返回值
  - 计算的结果

```
int a,b,c;
a = 5;
b = 6;
c = max(10,12);
c = max(a,b);
c = max(c, 23);
c = max(max(23,45), a);
c = max(23+45, b);
```

类型不匹配?

- 调用函数时给的值与参数的类型不匹配是C语言传统上最大的漏洞
- 编译器总是悄悄替你把类型转换好，但是这很可能不是你所期望的
- 后续的语言，C++/Java在这方面很严格

传过去的是什么?

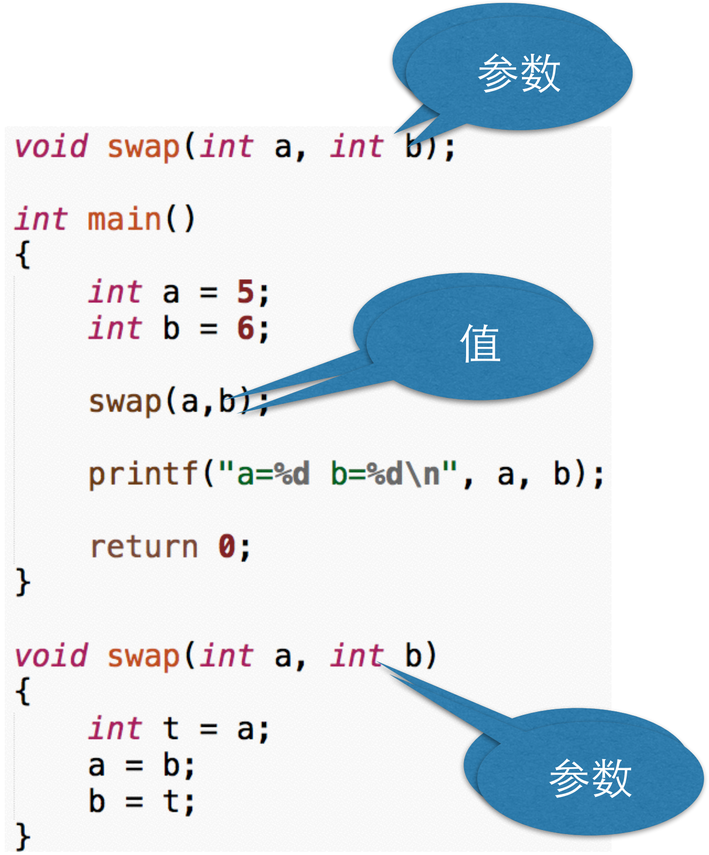
```c
void swap(int a, int b);

int main()
{
    int a = 5;
    int b = 6;

    swap(a,b);

    printf("a=%d b=%d\n", a, b);

    return 0;
}

void swap(int a, int b)
{
    int t = a;
    a = b;
    b = t;
}
```

C语言在调用函数时，永远只能传值给函数

- 这样的代码能交换a和b的值吗?

传值

- 每个函数有自己的变量空间，参数也位于这个独立的空间中，和其他函数没有关系
- 过去，对于函数参数表中的参数，叫做“形式参数”，调用函数时给的值，叫做“实际参数”
- 由于容易让初学者误会实际参数就是实际在函数中进行计算的参数，误会调用函数的时候把变量而不是值传进去了，所以我们不建议继续用这种古老的方式来称呼它们
- 我们认为，它们是参数和值的关系




```
void swap(int a, int b);

int main()
{
    int a = 5;
    int b = 6;

    swap(a,b);

    printf("a=%d b=%d\n", a, b);

    return 0;
}

void swap(int a, int b)
{
    int t = a;
    a = b;
    b = t;
}
```

本地变量

本地变量

- 函数的每次运行，就产生了一个独立的变量空间，在这个空间中的变量，是函数的这次运行所独有的，称作本地变量
- 定义在函数内部的变量就是本地变量
- 参数也是本地变量

变量的生存期和作用域

- 生存期：什么时候这个变量开始出现了，到什么时候它消亡了
- 作用域：在（代码的）什么范围内可以访问这个变量（这个变量可以起作用）
- 对于本地变量，这两个问题的答案是统一的：大括号内——块

本地变量的规则

- 本地变量是定义在块内的
  - 它可以是定义在函数的块内
  - 也可以定义在语句的块内
  - 甚至可以随便拉一对大括号来定义变量
- 程序运行进入这个块之前，其中的变量不存在，离开这个块，其中的变量就消失了
- 块外面定义的变量在里面仍然有效
- 块里面定义了和外面同名的变量则掩盖了外面的
- 不能在一个块内定义同名的变量
- 本地变量不会被默认初始化
- 参数在进入函数的时候被初始化了

其他细节

没有参数时

- void f(void);
- 还是
- void f();
  - 在传统C中，它表示f函数的参数表未知，并不表示没有参数

逗号运算符?

- 调用函数时的逗号和逗号运算符怎么区分?
- 调用函数时的圆括号里的逗号是标点符号，不是运算符
  - f(a,b)
  - f((a,b))

函数里的函数?

- C语言不允许函数嵌套定义

这是什么?

- int i,j,sum(int a, int b);
- return (i);

关于main

- int main()也是一个函数
- 要不要写成int main(void)?
- return的0有人看吗?
  - Windows：if errorlevel 1 …
  - Unix Bash：echo $?
  - Csh：echo $status

8.1-1 初试数组

- 如何写一个程序计算用户输入的数字的平均数?

```c
int x;
double sum = 0;
int cnt = 0;
scanf("%d", &x);
while ( x!= -1 ) {
    sum += x;
    cnt ++;
    scanf("%d", &x);
}
if ( cnt >0 ) {
    printf("%f\n", sum/cnt);
}
```

- 不需要记录输入的每一个数

- 如何写一个程序计算用户输入的数字的平均数，并输出所有大于平均数的数?
- 必须先记录每一个输入的数字，计算平均数之后，再检查记录下来的每一个数字，与平均数比较，决定是否输出

如何记录很多数?

- int num1,num2,num3…..?

数组

- int number[100];
- scanf(“%d”, &number[i]);

数组

```c
int x;
double sum = 0;
int cnt = 0;
int number[100];
scanf("%d", &x);
while ( x!= -1 ) {
    number[cnt] = x;
    sum += x;
    cnt ++;
    scanf("%d", &x);
}
if ( cnt >0 ) {
    int i;
    double average = sum / cnt;
    for ( i=0; i<cnt; i++ ) {
        if ( number[i] > average ) {
            printf("%d ", number[i]);
        }
    }
}
```

定义数组

对数组中的元素赋值

这个程序存在安全隐患，是什么？

使用数组中的元素

遍历数组

8.1-2 数组的定义和使用

定义数组

- <类型> 变量名称[元素数量];
  - int grades[100];
  - double weight[20];
- 元素数量必须是整数
- C99之前：元素数量必须是编译时刻确定的字面量

数组

- 是一种容器（放东西的东西），特点是:
  - 其中所有的元素具有相同的数据类型;
  - 一旦创建，不能改变大小
  - *（数组中的元素在内存中是连续依次排列的）

int a[10]

- 一个int的数组
- 10个单元：a[0],a[1],…,a[9]

- 每个单元就是一个int类型的变量
- 可以出现在赋值的左边或右边：
  - a[2] = a[1]+6;
- * 在赋值左边的叫做左值

数组的单元

- 数组的每个单元就是数组类型的一个变量
- 使用数组时放在[]中的数字叫做下标或索引，下标从0开始计数：
  - grades[0]
  - grades[99]
  - average[5]

有效的下标范围

- 编译器和运行环境都不会检查数组下标是否越界，无论是对数组单元做读还是写
- 一旦程序运行，越界的数组访问可能造成问题，导致程序崩溃
  - **segmentation fault**
- 但是也可能运气好，没造成严重的后果
- 所以这是程序员的责任来保证程序只使用有效的下标值：[0,数组的大小 – 1]

```c
int x;
double sum = 0;
int cnt = 0;
int number[100];
scanf("%d", &x);
while ( x!= -1 ) {
    number[cnt] = x;
    sum += x;
    cnt ++;
    scanf("%d", &x);
}
if ( cnt >0 ) {
    int i;
    double average = sum / cnt;
    for ( i=0; i<cnt; i++ ) {
        if ( number[i] > average ) {
            printf("%d ", number[i]);
        }
    }
}
```

- 这个程序是危险的，因为输入的数据可能超过100个

计算平均数

- 如果先让用户输入有多少数字要计算，可以用C99的新功能来实现

```c
int x;
double sum = 0;
int cnt;
printf("请输入数字的数量: ");
scanf("%d", &cnt);
if ( cnt>0 ) {
    int number[cnt];
    scanf("%d", &x);
    while ( x!= -1 ) {
        number[cnt] = x;
        sum += x;
        cnt ++;
        scanf("%d", &x);
    }
```

C99 ONLY!

长度为0的数组?

- int a[0];
- 可以存在，但是无用

8.1-3 数组的例子：投票统计

- 写一个程序，输入数量不确定的[0,9]范围内的整数，统计每一种数字出现的次数，输入-1表示结束

```c
const int number = 10;
int x;
int count[number];
int i;

for ( i=0; i<number; i++ ) {
    count[i] = 0;
}
scanf("%d", &x);
while ( x!= -1 ) {
    if ( x>=0 && x<=9 ) {
        count[x]++;
    }
    scanf("%d", &x);
}
for ( i=0; i<number; i++ ) {
    printf("%d:%d\n", i, count[i]);
}
```

数组的大小

C99

定义数组

初始化数组

数组参与运算

遍历数组输出

8.2-1 数组的运算

- 在一组给定的数据中，如何找出某个数据是否存在?

```
/**
找出key在数组a中的位置
@param key 要寻找的数字
@param a 要寻找的数组
@param length 数组a的长度
@return 如果找到，返回其在a中的位置；如果找不到则返回-1
*/
int search(int key, int a[], int length);
```

```
int main(void)
{
    int a[] = {2,4,6,7,1,3,5,9,11,13,23,14,32};
    int x;
    int loc;
    printf("请输入一个数字: ");
    scanf("%d", &x);
    loc=search(x, a, sizeof(a)/sizeof(a[0]));
    if ( loc != -1 ) {
        printf("%d在第%d个位置上\n", x, loc);
    } else {
        printf("%d不存在\n", x);
    }

    return 0;
}
```

```
int search(int key, int a[], int length)
{
    int ret = -1;
    int i;
    for ( i=0; i< length; i++ ) {
        if ( a[i] == key ) {
            ret = i;
            break;
        }
    }
    return ret;
}
```

数组的集成初始化

```
int a[] = {2,4,6,7,1,3,5,9,11,13,23,14,32};
```

- 直接用大括号给出数组的所有元素的初始值
- 不需要给出数组的大小，编译器替你数数

```
int b[20] = {2};
```

- 如果给出了数组的大小，但是后面的初始值数量不足，则其后的元素被初始化为0

集成初始化时的定位

C99 ONLY!

```
int a[10] = {
      [0] = 2, [2] = 3, 6 ,
};
```

- 用[n]在初始化数据中给出定位
- 没有定位的数据接在前面的位置后面
- 其他位置的值补零
- 也可以不给出数组大小，让编译器算
- 特别适合初始数据稀疏的数组

数组的大小

- sizeof给出整个数组所占据的内容的大小，单位是字节

```
sizeof(a)/sizeof(a[0])
```

- sizeof(a[0])给出数组中单个元素的大小，于是相除就得到了数组的单元个数
- 这样的代码，一旦修改数组中初始的数据，不需要修改遍历的代码

可扩展性!

数组的赋值

```
int a[] = {2,4,6,7,1,3,5,9,11,13,23,14,32};
int b[] = a;
```

- 数组变量本身不能被赋值
- 要把一个数组的所有元素交给另一个数组，必须采用遍历

```
for ( i=0; i<length; i++ ) {
    b[i] = a[i];
}
```

遍历数组

```
for ( i=0; i<length; i++ ) {
    b[i] = a[i];
}
```

```
for ( i=0; i<number; i++ ) {
    count[i] = 0;
}
```

```
for ( i=0; i< length; i++ ) {
    if ( a[i] == key ) {
        ret = i;
        break;
    }
}
```

```
for ( i=0; i<cnt; i++ ) {
    if ( number[i] > average ) {
        printf("%d ", number[i]);
    }
}
```

```
for ( i=0; i<number; i++ ) {
    printf("%d:%d\n", i, count[i]);
}
```

- 通常都是使用for循环，让循环变量i从0到<数组的长度，这样循环体内最大的i正好是数组最大的有效下标
- 常见错误是：
  - 循环结束条件是<=数组长度，或；
  - 离开循环后，继续用i的值来做数组元素的下标！

```c
/**
找出key在数组a中的位置
@param key 要寻找的数字
@param a 要寻找的数组
@param length 数组a的长度
@return 如果找到，返回其在a中的位置；如果找不到则返回-1
*/
int search(int key, int a[], int length);
```

```c
int main(void)
{
    int a[] = {2,4,6,7,1,3,5,9,11,13,23,14,32};
    int x;
    int loc;
    printf("请输入一个数字: ");
    scanf("%d", &x);
    loc=search(x, a, sizeof(a)/sizeof(a[0]));
    if ( loc != -1 ) {
        printf("%d在第%d个位置上\n", x, loc);
    } else {
        printf("%d不存在\n", x);
    }

    return 0;
}
```

```c
int search(int key, int a[], int length)
{
    int ret = -1;
    int i;
    for ( i=0; i< length; i++ ) {
        if ( a[i] == key ) {
            ret = i;
            break;
        }
    }
    return ret;
}
```

数组作为函数参数时，往往必须再用另一个参数来传入数组的大小

- 数组作为函数的参数时：
  - 不能在[]中给出数组的大小
  - 不能再利用sizeof来计算数组的元素个数！

8.2-2数组的例子：素数

判断素数

```c
int isPrime(int x);

int main(void)
{
    int x;
    scanf("%d", &x);
    if ( isPrime(x) ) {
        printf("%d是素数\n", x);
    } else {
        printf("%d不是素数\n", x);
    }
    return 0;
}
```

从2到x-1测试是否可以整除

```
int isPrime(int x)
{
    int ret = 1;
    int i;
    if ( x == 1 ) ret = 0;
    for ( i=2; i<x; i++ ) {
        if ( x % i == 0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```

- 对于n要循环n-1遍
  - 当n很大时就是n遍

去掉偶数后，从3到x-1，每次加2

```
int isPrime(int x)
{
    int ret = 1;
    int i;
    if ( x == 1 ||
         (x%2 == 0 && x!=2) )
        ret = 0;
    for ( i=3; i<x; i+=2 ) {
        if ( x % i == 0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```

- 如果x是偶数，立刻
- 否则要循环(n-3)/2+1遍
  - 当n很大时就是n/2遍

无须到x-1，到sqrt(x)就够了

```
int isPrime(int x)
{
    int ret = 1;
    int i;
    if ( x == 1 ||
         (x%2 == 0 && x!=2) )
        ret = 0;
    for ( i=3; i<sqrt(x); i+=2 ) {
        if ( x % i == 0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```

- 只需要循环sqrt(x)遍

sqrt

```
SQRT(3)                    BSD Library Functions Manual                    SQRT(3)

NAME
     sqrt -- square root function

SYNOPSIS
     #include <math.h>

     double
     sqrt(double x);

     long double
     sqrtl(long double x);

     float
     sqrtf(float x);

DESCRIPTION
     The sqrt() function compute the non-negative square root of x.
```

判断是否能被已知的且<x的素数整除

```c
int main(void)
{
    const int number = 100;
    int prime[number] = {2};
    int count = 1;
    int i = 3;
    while ( count < number ) {
        if ( isPrime(i, prime, count) ) {
            prime[count++] = i;
        }
        i++;
    }
    for ( i=0; i<number; i++ ) {
        printf("%d", prime[i]);
        if ( (i+1)%5 ) printf("\t");
        else printf("\n");
    }
    return 0;
}
```

```c
int isPrime(int x, int knownPrimes[], int numberOfKnownPrimes)
{
    int ret = 1;
    int i;
    for ( i=0; i<numberOfKnownPrimes; i++ ) {
        if ( x % knownPrimes[i] ==0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```

构造素数表

- 欲构造n以内的素数表

1. 令x为2
2. 将2x、3x、4x直至ax<n的数标记为非素数
3. 令x为下一个没有被标记为非素数的数，重复2；直到所有的数都已经尝试完毕

构造素数表

- 欲构造n以内（不含）的素数表

1. 开辟prime[n]，初始化其所有元素为1，prime[x]为1表示x是素数
2. 令x=2
3. 如果x是素数，则对于(i=2;x*i<n;i++)令prime[i*x]=0
4. 令x++，如果x<n，重复3，否则结束

构造素数表

```
const int maxNumber = 25;
int isPrime[maxNumber];
int i;
int x;
for ( i=0; i<maxNumber; i++ ) {
    isPrime[i] = 1;
}
for ( x=2; x<maxNumber; x++ ) {
    if ( isPrime[x] ) {
        for ( i=2; i*x<maxNumber; i++ ) {
            isPrime[i*x] = 0;
        }
    }
}
for ( i=2; i<maxNumber; i++ ) {
    if ( isPrime[i] ) {
        printf("%d\t", i);
    }
}
printf("\n");
```

- 算法不一定和人的思考方式相同

8.2-3 二维数组

二维数组

- int a[3][5];
- 通常理解为a是一个3行5列的矩阵

a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[0][4]
a[1][0]	a[1][1]	a[1][2]	a[1][3]	a[1][4]
a[2][0]	a[2][1]	a[2][2]	a[2][3]	a[2][4]

二维数组的遍历

```
for ( i=0; i<3; i++ ) {
    for ( j=0; j<5; j++ ) {
        a[i][j] = i*j;
    }
}
```

- a[i][j]是一个int
- 表示第i行第j列上的单元
  - a[i,j]是什么?

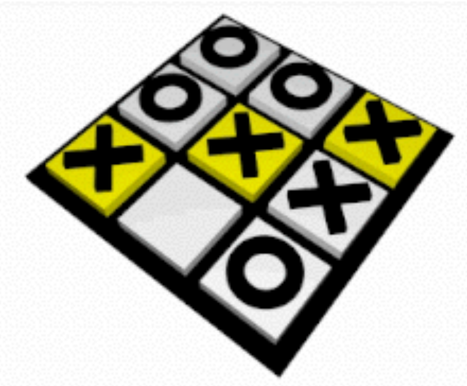
二维数组的初始化

```
int a[][5] = {
    {0,1,2,3,4},
    {2,3,4,5,6},
};
```

- 列数是必须给出的，行数可以由编译器来数
- 每行一个{}，逗号分隔
- 最后的逗号可以存在，有古老的传统
- 如果省略，表示补零
- 也可以用定位（ * C99 ONLY）

tic-tac-toe游戏

- 读入一个3X3的矩阵，矩阵中的数字为1表示该位置上有一个X，为0表示为O
- 程序判断这个矩阵中是否有获胜的一方，输出表示获胜一方的字符X或O，或输出无人获胜



本页图片来源：wikipedia.org

读入矩阵

```
const int size = 3;
int board[size][size];
int i,j;
int numOfX;
int numOfO;
int result = -1;     //  -1:没人赢，1:X赢，0:O赢

//  读入矩阵
for ( i=0; i<size; i++ ) {
    for ( j=0; j<size; j++ ) {
        scanf("%d", &board[i][j]);
    }
}
```

检查行

```
//  检查行
for ( i=0; i<size && result == -1; i++ ) {
    numOf0 = numOfX = 0;
    for ( j=0; j<size; j++ ) {
        if ( board[i][j] == 1 ) {
            numOfX ++;
        } else {
            numOf0 ++;
        }
    }
    if ( numOf0 == size ) {
        result = 0;
    } else if (numOfX == size ) {
        result = 1;
    }
}
```

检查列

```
if ( result == -1 ) {
    for ( j=0; j<size && result == -1; j++ ) {
        numOf0 = numOfX = 0;
        for ( i=0; i<size; i++ ) {
            if ( board[i][j] == 1 ) {
                numOfX ++;
            } else {
                numOf0 ++;
            }
        }
        if ( numOf0 == size ) {
            result = 0;
        } else if (numOfX == size ) {
            result = 1;
        }
    }
}
```

行和列?

```
//  检查行
for ( i=0; i<size && result == -1; i++ ) {
    numOf0 = numOfX = 0;
    for ( j=0; j<size; j++ ) {
        if ( board[i][j] == 1 ) {
            numOfX ++;
        } else {
            numOf0 ++;
        }
    }
    if ( numOf0 == size ) {
        result = 0;
    } else if (numOfX == size ) {
        result = 1;
    }
}
```

```
if ( result == -1 ) {
    for ( j=0; j<size && result == -1; j++ ) {
        numOf0 = numOfX = 0;
        for ( i=0; i<size; i++ ) {
            if ( board[i][j] == 1 ) {
                numOfX ++;
            } else {
                numOf0 ++;
            }
        }
        if ( numOf0 == size ) {
            result = 0;
        } else if (numOfX == size ) {
            result = 1;
        }
    }
}
```

能否用一个两重循环来检查行和列?

检查对角线

```
numOf0 = numOfX = 0;
for ( i=0; i<size; i++ ) {
    if ( board[i][i] == 1 ) {
        numOfX ++;
    } else {
        numOf0 ++;
    }
}
if ( numOf0 == size ) {
    result = 0;
} else if (numOfX == size ) {
    result = 1;
```

```
numOf0 = numOfX = 0;
for ( i=0; i<size; i++ ) {
    if ( board[i][size-i-1] == 1 ) {
        numOfX ++;
    } else {
        numOf0 ++;
    }
}
```

9.1-1 取地址运算

运算符 &

- scanf(“%d”, &i);里的&
- 获得变量的地址，它的操作数必须是变量
  - int i; printf(“%x”,&i);
- 地址的大小是否与int相同取决于编译器
  - int i; printf(“%p”,&i);

&不能取的地址

- &不能对没有地址的东西取地址
  - &(a+b)?
  - &(a++)?
  - &(++a)?

试试这些&

- 变量的地址
- 相邻的变量的地址
- &的结果的sizeof
- 数组的地址
- 数组单元的地址
- 相邻的数组单元的地址

9.1-2 指针

scanf

- 如果能够将取得的变量的地址传递给一个函数，能否通过这个地址在那个函数内访问这个变量?
  - scanf(“%d”, &i);
- scanf()的原型应该是怎样的？我们需要一个参数能保存别的变量的地址，如何表达能够保存地址的变量?

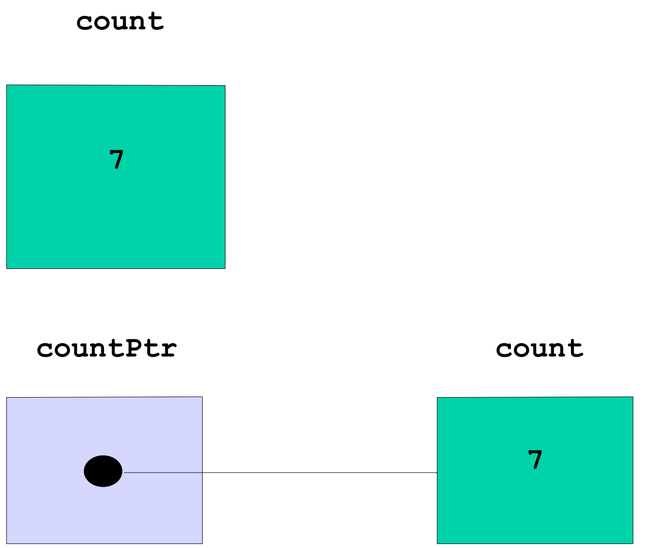
指针

- 就是保存地址的变量

```
int i;
int* p = &i;
int* p,q;
int *p,q;
```

指针变量

- 变量的值是内存的地址
  - 普通变量的值是实际的值
  - 指针变量的值是具有实际值的变量的地址



作为参数的指针

- void f(int *p);
- 在被调用的时候得到了某个变量的地址：
  - int i=0; f(&i);
- 在函数里面可以通过这个指针访问外面的这个i

访问那个地址上的变量*

- *是一个单目运算符，用来访问指针的值所表示的地址上的变量
- 可以做右值也可以做左值
  - int k = *p;
  - *p = k+1;

* 左值之所以叫左值

- 是因为出现在赋值号左边的不是变量，而是值，是表达式计算的结果：
  - a[0] = 2;
  - *p = 3;
- 是特殊的值，所以叫做左值

指针的运算符&*

- 互相反作用
  - *&yptr -> * (&yptr) -> * (yptr的地址)-> 得到那个地址上的变量 -> yptr
  - &*yptr -> &(*yptr) -> &(y) -> 得到y的地址，也就是yptr -> yptr

传入地址

- 为什么
  - int i; scanf(“%d”, i);
- 编译没有报错?

指针应用场景一

- 交换两个变量的值

```
void swap(int *pa, int *pb)
{
    int t = *pa;
    *pa = *pb;
    *pb = t;
}
```

指针应用场景二

- 函数返回多个值，某些值就只能通过指针返回
  - 传入的参数实际上是需要保存带回的结果的变量

指针应用场景二b

- 函数返回运算的状态，结果通过指针返回
- 常用的套路是让函数返回特殊的不属于有效范围内的值来表示出错:
  - -1或0（在文件操作会看到大量的例子）
- 但是当任何数值都是有效的可能结果时，就得分开返回了
  - 后续的语言（C++,Java）采用了异常机制来解决这个问题

指针最常见的错误

- 定义了指针变量，还没有指向任何变量，就开始使用指针

9.1-3 指针与数组

传入函数的数组成了什么?

```
int isPrime(int x, int knownPrimes[], int numberOfKnownPrimes)
{
    int ret = 1;
    int i;
    for ( i=0; i<numberOfKnownPrimes; i++ ) {
        if ( x % knownPrimes[i] ==0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```

- 函数参数表中的数组实际上是指针
  - sizeof(a) == sizeof(int*)
  - 但是可以用数组的运算符[]进行运算

数组参数

- 以下四种函数原型是等价的:
  - int sum(int *ar, int n);
  - int sum(int *, int);
  - int sum(int ar[], int n);
  - int sum(int [], int);

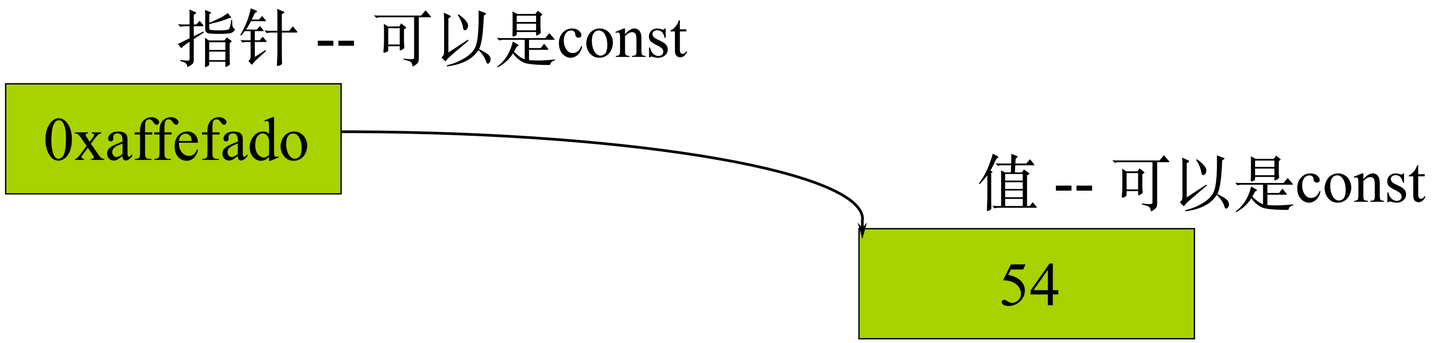
数组变量是特殊的指针

- 数组变量本身表达地址，所以
  - int a[10]; int*p=a; // 无需用&取地址
  - 但是数组的单元表达的是变量，需要用&取地址
  - a == &a[0]
- []运算符可以对数组做，也可以对指针做：
  - p[0] <==> a[0]
- *运算符可以对指针做，也可以对数组做：
  - *a = 25;
- 数组变量是const的指针，所以不能被赋值
  - int a[] <==> int * const a=….

* 9.1-4 指针与const

C99 ONLY!

指针与const



指针是const

- 表示一旦得到了某个变量的地址，不能再指向其他变量
  - int * const q = &i; // q 是 const
  - *q = 26; // OK
  - q++; // ERROR

所指是const

- 表示不能通过这个指针去修改那个变量（并不能使得那个变量成为const）
  - const int *p = &i;
  - *p = 26; // ERROR! (*p) 是 const
  - i = 26; //OK
  - p = &j; //OK

这些是啥意思?

```
int i;
const int* p1 = &i;
int const* p2 = &i;
int *const p3 = &i;
```

判断哪个被const了的标志是const在*的前面还是后面

转换

- 总是可以把一个非const的值转换成const的

```
void f(const int* x);
int a = 15;
f(&a); // ok
const int b = a;

f(&b); // ok
b = a + 1; // Error!
```

- 当要传递的参数的类型比地址大的时候，这是常用的手段：既能用比较少的字节数传递值给参数，又能避免函数对外面的变量的修改

const数组

- const int a[] = {1,2,3,4,5,6,};
- 数组变量已经是const的指针了，这里的const表明数组的每个单元都是const int
- 所以必须通过初始化进行赋值

保护数组值

- 因为把数组传入函数时传递的是地址，所以那个函数内部可以修改数组的值
- 为了保护数组不被函数破坏，可以设置参数为const
  - int sum(const int a[], int length);

9.2-1 指针是可计算的

1+1=2?

- 给一个指针加1表示要让指针指向下一个变量

  int a[10];

  int *p = a;

  *(p+1) —> a[1]

- 如果指针不是指向一片连续分配的空间，如数组，则这种运算没有意义

指针计算

- 这些算术运算可以对指针做:
  - 给指针加、减一个整数(+, +=, -, -=)
  - 递增递减(++/—)
  - 两个指针相减

*p++

- 取出p所指的那个数据来，完事之后顺便把p移到下一个位置去
- *的优先级虽然高，但是没有++高
- 常用于数组类的连续空间操作
- 在某些CPU上，这可以直接被翻译成一条汇编指令

指针比较

- <, <=, ==, >, >=, != 都可以对指针做
- 比较它们在内存中的地址
- 数组中的单元的地址肯定是线性递增的

0地址

- 当然你的内存中有0地址，但是0地址通常是个不能随便碰的地址
- 所以你的指针不应该具有0值
- 因此可以用0地址来表示特殊的事情：
  - 返回的指针是无效的
  - 指针没有被真正初始化（先初始化为0）
- NULL是一个预定定义的符号，表示0地址
  - 有的编译器不愿意你用0来表示0地址

指针的类型

- 无论指向什么类型，所有的指针的大小都是一样的，因为都是地址
- 但是指向不同类型的指针是不能直接互相赋值的
- 这是为了避免用错指针

指针的类型转换

- void* 表示不知道指向什么东西的指针
  - 计算时与char*相同（但不相通）
- 指针也可以转换类型
  - int *p = &i; void*q = (void*)p;
- 这并没有改变p所指的变量的类型，而是让后人用不同的眼光通过p看它所指的变量
  - 我不再当你是int啦，我认为你就是个void!

用指针来做什么

- 需要传入较大的数据时用作参数
- 传入数组后对数组做操作
- 函数返回不止一个结果
  - 需要用函数来修改不止一个变量
- 动态申请的内存...

9.2-2 动态内存分配

输入数据

- 如果输入数据时，先告诉你个数，然后再输入，要记录每个数据
- C99可以用变量做数组定义的大小，C99之前呢?
- int *a = (int*)malloc(n*sizeof(int));

malloc

```
#include <stdlib.h>

void* malloc(size_t size);
```

- 向malloc申请的空间的大小是以字节为单位的
- 返回的结果是void*，需要类型转换为自己需要的类型
  - (int*)malloc(n*sizeof(int))

没空间了?

- 如果申请失败则返回0，或者叫做NULL
- 你的系统能给你多大的空间?

free()

- 把申请得来的空间还给“系统”
- 申请过的空间，最终都应该要还
  - 混出来的，迟早都是要还的
- 只能还申请来的空间的首地址
- free(0)?

常见问题

- 申请了没free—>长时间运行内存逐渐下降
  - 新手：忘了
  - 老手：找不到合适的free的时机
- free过了再free
- 地址变过了，直接去free

以下内容不制作视频

9.2-3 函数间传递指针

好的模式

- 如果程序中要用到动态分配的内存，并且会在函数之间传递，不要让函数申请内存后返回给调用者
- 因为十有八九调用者会忘了free，或找不到合适的时机来free
- 好的模式是让调用者自己申请，传地址进函数，函数再返回这个地址出来

```c
int* init(int a[], int length);
int* print(int a[], int length);

int main()
{
    const int MAX_SIZE = 1000;
    int size;
    do {
        printf("输入数量(0,1000)：");
        scanf("%d", &size);
    } while (size>0 && size<MAX_SIZE);
    int* a = (int*)malloc(size*sizeof(int));
    print(init(a, size), size);
    free(a);

    return 0;
}
```

```c
int* init(int a[], int length)
{
    int i;
    for ( i=0; i<length; i++ ) {
        a[i] = i;
    }
    return a;
}

int* print(int a[], int length)
{
    int i;
    for ( i=0; i<length; i++ ) {
        printf("%d\t", a[i]);
    }
    printf("\n");
    return a;
}
```

除非函数的作用就是分配空间，否则不要在函数中malloc然后传出去用

在
同一个地方malloc和
free

函数返回指针?

- 返回指针没问题，关键是谁的地址?
  - 本地变量（包括参数）？函数离开后这些变量就不存在了，指针所指的是不能用的内存
  - 传入的指针？没问题
  - 动态申请的内存？没问题
  - 全局变量—>以后再解释

函数返回数组?

- 如果一个函数的返回类型是数组，那么它实际返回的也是数组的地址
- 如果这个数组是这个函数的本地变量，那么回到调用函数那里，这个数组就不存在了
- 所以只能返回
  - 传入的参数：实际就是在调用者那里
  - 全局变量或动态分配的内存

和返回指针是一样的!

字符串

字符数组

- char word[] = {‘H’, ‘e’, ‘l’, ‘l’, ‘o’,‘!’};

word[0]	H
word[1]	e
word[2]	
word[3]	l
word[4]	o
word[5]	!

这不是C语言的字符串，因为不能用字符串的方式做计算

字符串

- char word[] = {‘H’, ‘e’, ‘l’, ‘l’, ‘o’,‘!’,’\0’};

word[0]	H
word[1]	e
word[2]	l
word[3]	l
word[4]	o
word[5]	!
word[6]	\0

字符串

- 以0（整数0）结尾的一串字符
  - 0或'\0'是一样的，但是和'0'不同
- 0标志字符串的结束，但它不是字符串的一部分
  - 计算字符串长度的时候不包含这个0
- 字符串以数组的形式存在，以数组或指针的形式访问
  - 更多的是以指针的形式
- string.h 里有很多处理字符串的函数

字符串变量

- char *str = “Hello”;
- char word[] = “Hello”;
- char line[10] = “Hello”;

字符串常量

- “Hello”
- ”Hello” 会被编译器变成一个字符数组放在某处，这个数组的长度是6，结尾还有表示结束的0
- 两个相邻的字符串常量会被自动连接起来
- 行末的\表示下一行还是这个字符串常量

字符串

- C语言的字符串是以字符数组的形态存在的
  - 不能用运算符对字符串做运算
  - 通过数组的方式可以遍历字符串
- 唯一特殊的地方是字符串字面量可以用来初始化字符数组
- 以及标准库提供了一系列字符串函数

字符串常量

```
char* s = "Hello, world!";
```

- s 是一个指针，初始化为指向一个字符串常量
  - 由于这个常量所在的地方，所以实际上s是 `const char* s` ，但是由于历史的原因，编译器接受不带const的写法
  - 但是试图对s所指的字符串做写入会导致严重的后果
- 如果需要修改字符串，应该用数组:

```
char s[] = "Hello, world!";
```

指针还是数组?

- char *str = “Hello”;
- char word[] = “Hello”;
  - 数组：这个字符串在这里
    - 作为本地变量空间自动被回收
  - 指针：这个字符串不知道在哪里
    - 处理参数
    - 动态分配空间

- 如果要构造一个字符串—>数组
- 如果要处理一个字符串—>指针

char*是字符串?

- 字符串可以表达为char*的形式
- char*不一定是字符串
  - 本意是指向字符的指针，可能指向的是字符的数组（就像int*一样）
  - 只有它所指的字符数组有结尾的0，才能说它所指的是字符串

字符串运算

字符串赋值?

- char *t = “title”;
- char *s;
- s = t;
- 并没有产生新的字符串，只是让指针s指向了t所指的字符串，对s的任何操作就是对t做的

字符串输入输出

- char string[8];
- scanf(“%s”, string);
- printf(“%s”, string);

- scanf读入一个单词（到空格、tab或回车为止）
- scanf是不安全的，因为不知道要读入的内容的长度

安全的输入

- char string[8];
- scanf(“%7s”, string);

- 在%和s之间的数字表示最多允许读入的字符的数量，这个数字应该比数组的大小小一
  - 下一次scanf从哪里开始?

常见错误

- char *string;
- scanf(“%s”, string);
- 以为char*是字符串类型，定义了一个字符串类型的变量string就可以直接使用了
  - 由于没有对string初始化为0，所以不一定每次运行都出错

空字符串

- char buffer[100]=””;
  - 这是一个空的字符串，buffer[0] == ‘\0’
- char buffer[] = “”;
  - 这个数组的长度只有1!

字符串数组

- char **a
  - a是一个指针，指向另一个指针，那个指针指向一个字符（串）
- char a[][]
  - a是一个二维数组，第二个维度的大小不知道，不能编译
- char a[][10]
  - a是一个二维数组，a[x]是一个char[10]
- char *a[]
  - a是一个一维数组，a[x]是一个char*

```
printf("请输入月份: ");
int month;
scanf("%d", &month);
switch ( month ) {
      case 1: printf("January\n"); break;
      case 2: printf("February\n"); break;
      case 3: printf("March\n"); break;
      case 4: printf("April\n"); break;
      case 5: printf("May\n"); break;
      case 6: printf("June\n"); break;
      case 7: printf("July\n"); break;
      case 8: printf("August\n"); break;
      case 9: printf("September\n"); break;
      case 10:printf("October\n"); break;
      case 11:printf("November\n"); break;
      case 12:printf("December\n"); break;
}
```

今后可以用数组来做

程序参数

- int main(int argc, char const *argv[])
- argv[0]是命令本身
  - 当使用Unix的符号链接时，反映符号链接的名字

字符输入输出

putchar

- int putchar(int c);
- 向标准输出写一个字符
- 返回写了几个字符，EOF（-1）表示写失败

getchar

- int getchar(void);
- 从标准输入读入一个字符
- 返回类型是int是为了返回EOF（-1）
  - Windows—>Ctrl-Z
  - Unix—>Ctrl-D

标准库中的字符串函数

string.h

- strlen
- strcmp
- strcpy
- strcat
- strchr
- strstr

strlen

- size_t strlen(const char *s);
- 返回s的字符串长度（不包括结尾的0）

strcmp

- int strcmp(const char *s1, const char *s2);
- 比较两个字符串，返回：
  - 0:s1==s2
  - >0:s1>s2
  - <0:s1<s2

strcpy

- char * strcpy(char *restrict dst, const char *restrict src);
- 把src的字符串拷贝到dst
  - restrict表明src和dst不重叠（C99）
- 返回dst
  - 为了能链起代码来

复制一个字符串

```
char *dst = (char*)malloc(strlen(src)+1);

strcpy(dst, src);
```

strcat

- char * strcat(char *restrict s1, const char *restrict s2);
- 把s2拷贝到s1的后面，接成一个长的字符串
- 返回s1
- s1必须具有足够的空间

安全问题

- strcpy和strcat都可能出现安全问题
  - 如果目的地没有足够的空间?

安全版本

- char * strncpy(char *restrict dst, const char *restrict src, size_t n);
- char * strncat(char *restrict s1, const char *restrict s2, size_t n);
- int strncmp(const char *s1, const char *s2, size_t n);

字符串中找字符

- char * strchr(const char *s, int c);
- char * strrchr(const char *s, int c);
- 返回NULL表示没有找到

- 如何寻找第2个?

字符串中找字符串

- char * strstr(const char *s1, const char *s2);
- char * strcasestr(const char *s1, const char *s2);

枚举

常量符号化

```c
#include <stdio.h>

const int red = 0;
const int yellow = 1;
const int green = 2;

int main(int argc, char const *argv[])
{
    int color = -1;
    char *colorName = NULL;

    printf("输入你喜欢的颜色的代码: ");
    scanf("%d", &color);
    switch ( color ) {
    case red: colorName = "red"; break;
    case yellow: colorName = "yellow"; break;
    case green: colorName = "green"; break;
    default: colorName= "unknown"; break;
    }
    printf("你喜欢的颜色是%s\n", colorName);

    return 0;
}
```

- 用符号而不是具体的数字来表示程序中的数字

枚举

```c
#include <stdio.h>

enum COLOR {RED, YELLOW, GREEN};

int main(int argc, char const *argv[])
{
    int color = -1;
    char *colorName = NULL;

    printf("输入你喜欢的颜色的代码: ");
    scanf("%d", &color);
    switch ( color ) {
    case RED: colorName = "red"; break;
    case YELLOW: colorName = "yellow"; break;
    case GREEN: colorName = "green"; break;
    default: colorName= "unknown"; break;
    }
    printf("你喜欢的颜色是%s\n", colorName);

    return 0;
}
```

- 用枚举而不是定义独立的const int变量

枚举

- 枚举是一种用户定义的数据类型，它用关键字 enum 以如下语法来声明：

  enum 枚举类型名字 {名字0, …, 名字n} ;

- 枚举类型名字通常并不真的使用，要用的是在大括号里的名字，因为它们就是就是常量符号，它们的类型是int，值则依次从0到n。如：

  enum colors { red, yellow, green } ;

- 就创建了三个常量，red的值是0，yellow是1，而green是2。

- 当需要一些可以排列起来的常量值时，定义枚举的意义就是给了这些常量值名字。

```c
#include <stdio.h>

enum color { red, yellow, green};

void f(enum color c);

int main(void)
{
    enum color t = red;

    scanf("%d", &t);
    f(t);

    return 0;
}

void f(enum color c)
{
    printf("%d\n", c);
}
```

- 枚举量可以作为值
- 枚举类型可以跟上enum作为类型
- 但是实际上是以整数来做内部计算和外部输入输出的

套路：自动计数的枚举

```c
#include <stdio.h>

enum COLOR {RED, YELLOW, GREEN, NumCOLORS};

int main(int argc, char const *argv[])
{
    int color = -1;
    char *ColorNames[NumCOLORS] = {
        "red", "yellow", "green",
    };
    char *colorName = NULL;

    printf("输入你喜欢的颜色的代码: ");
    scanf("%d", &color);
    if ( color >=0 && color < NumCOLORS ) {
        colorName = ColorNames[color];
    } else {
        colorName= "unknown";
    }
    printf("你喜欢的颜色是%s\n", colorName);

    return 0;
}
```

- 这样需要遍历所有的枚举量或者需要建立一个用枚举量做下标的数组的时候就很方便了

枚举量

- 声明枚举量的时候可以指定值
  - enum COLOR { RED=1, YELLOW, GREEN = 5};

```c
#include <stdio.h>

enum COLOR {RED=1, YELLOW, GREEN=5, NumCOLORS};

int main(int argc, char const *argv[])
{
    printf("code for GREEN is %d\n", GREEN);

    return 0;
}
```

枚举只是int

```c
#include <stdio.h>

enum COLOR {RED=1, YELLOW, GREEN=5, NumCOLORS};

int main(int argc, char const *argv[])
{
    enum COLOR color = 0;

    printf("code for GREEN is %d\n", GREEN);
    printf("and color is %d\n", color);

    return 0;
}
```

- 即使给枚举类型的变量赋不存在的整数值也没有任何warning或error

枚举

- 虽然枚举类型可以当作类型使用，但是实际上很(bu)少(hao)用
- 如果有意义上排比的名字，用枚举比const int方便
- 枚举比宏（macro）好，因为枚举有int类型

结构

声明结构类型

```c
#include <stdio.h>

int main(int argc, char const *argv[])
{
    struct date {
        int month;
        int day;
        int year;
    };

    struct date today;

    today.month = 07;
    today.day = 31;
    today.year = 2014;

    printf("Today's date is %i-%i-%i.\n",
        today.year,today.month,today.day);

    return 0;
}
```

初学者最常见的
错误：漏了这个分号！

在函数内/外?

```c
#include <stdio.h>

struct date {
    int month;
    int day;
    int year;
};

int main(int argc, char const *argv[])
{
    struct date today;

    today.month = 07;
    today.day = 31;
    today.year = 2014;

    printf("Today's date is %i-%i-%i.\n",
        today.year,today.month,today.day);

    return 0;
}
```

- 和本地变量一样，在函数内部声明的结构类型只能在函数内部使用
- 所以通常在函数外部声明结构类型，这样就可以被多个函数所使用了

声明结构的形式

```
struct point {
    int x;
    int y;
};

struct point p1, p2;
```

p1 和 p2 都是point
里面有x和y的值

```
struct {
    int x;
    int y;
} p1, p2;
```

p1 和 p2都是一种
无名结构，里面有
x和y

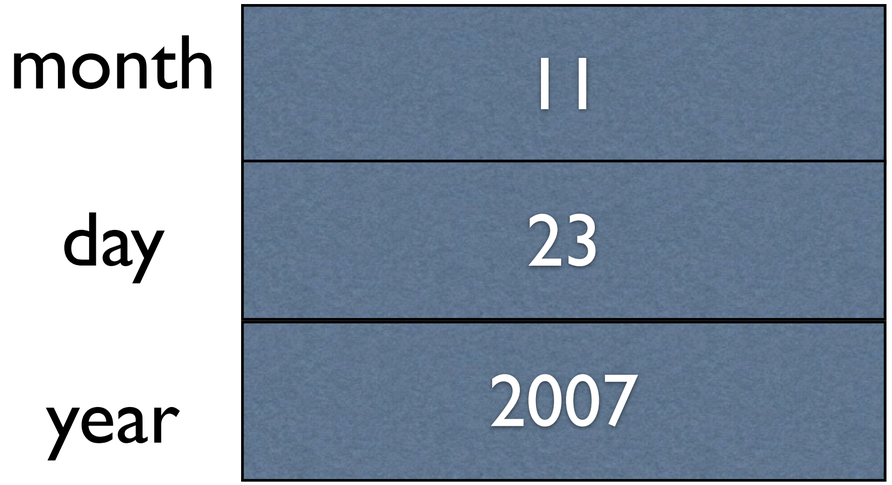
```
struct  point {
    int x;
    int y;
} p1, p2;
```

p1和p2都是point
里面有x和y的值t

对于第一和第三种形式，都声明了结构point。但是第二种形式没有声明point，只是定义了两个变量

结构变量

```
struct date today;
today.month=06;
today.day=19;
today.year=2005;
```



结构的初始化

```c
#include <stdio.h>

struct date {
    int month;
    int day;
    int year;
};

int main(int argc, char const *argv[])
{
    struct date today = {07,31,2014};
    struct date thismonth = {.month=7, .year=2014};

    printf("Today's date is %i-%i-%i.\n",
        today.year,today.month,today.day);
    printf("This month is %i-%i-%i.\n",
        thismonth.year,thismonth.month,thismonth.day);

    return 0;
}
```

结构成员

- 结构和数组有点像
- 数组用[]运算符和下标访问其成员
  - a[0] = 10;
- 结构用.运算符和名字访问其成员
  - today.day
  - student.firstName
  - p1.x
  - p1.y

结构运算

- 要访问整个结构，直接用结构变量的名字
- 对于整个结构，可以做赋值、取地址，也可以传递给函数参数
  - p1 = (struct point){5, 10}; // 相当于p1.x = 5; p1.y = 10;
  - p1 = p2; // 相当于p1.x = p2.x; p1.y = p2.y;

数组无法做这两种运算！

复合字面量

- today = (struct date) {9,25,2004};
- today = (struct date) {.month=9, .day=25, year=2004};

结构指针

- 和数组不同，结构变量的名字并不是结构变量的地址，必须使用&运算符
- struct date *pDate = &today;

结构与函数

结构作为函数参数

```
int numberOfDays(struct date d)
```

- 整个结构可以作为参数的值传入函数
- 这时候是在函数内新建一个结构变量，并复制调用者的结构的值
- 也可以返回一个结构
- 这与数组完全不同

输入结构

- 没有直接的方式可以一次scanf一个结构
- 如果我们打算写一个函数来读入结构
  - —>
- 但是读入的结构如何送回来呢?
- 记住C在函数调用时是传值的
  - 所以函数中的p与main中的y是不同的
  - 在函数读入了p的数值之后，没有任何东西回到main，所以y还是 {0, 0}

```
#include <stdio.h>

struct point  {
     int x;
     int y;  };

void getStruct(struct point);
void output(struct point);
void main( )    {
     struct point y = {0, 0};
     getStruct(y);
     output(y);  }

void getStruct(struct point p)   {
     scanf("%d", &p.x);
     scanf("%d", &p.y);
     printf("%d, %d", p.x, p.y);  }

void output(struct point p)   {
     printf("%d, %d", p.x, p.y);  }
```

解决的方案

- 之前的方案，把一个结构传入了函数，然后在函数中操作，但是没有返回回去
  - 问题在于传入函数的是外面那个结构的克隆体，而不是指针
    - 传入结构和传入数组是不同的
- 在这个输入函数中，完全可以创建一个临时的结构变量，然后把这个结构返回给调用者

```
void main( )
{
      struct point y = {0, 0};
      y = inputPoint( );
      output(y);
}
```

```
struct point inputPoint( )
{
      struct point temp;
      scanf("%d", &temp.x);
      scanf("%d", &temp.y);
      return temp;
}
```

也可以把y的地址传给函数，函数的参数类型是指向一个结构的指针。
不过那样的话，访问结构的成员的方式需要做出调整。

结构指针作为参数

- K & R 说过 (p. 131)
  - “If a large structure is to be passed to a function, it is generally more efficient to pass a pointer than to copy the whole structure”

指向结构的指针

```
struct date {
    int month;
    int day;
    int year;
} myday;

struct date *p = &myday;

(*p).month = 12;
p->month = 12;
```

- 用->表示指针所指的结构变量中的成员

结构指针参数

```
void main( )
{
    struct point y = {0, 0};
    inputPoint(&y);
    output(y);
}
```

```
struct point* inputPoint(struct point *p)
{
        scanf("%d", &(p->x));
        scanf("%d", &(p->y);
        return p;
}
```

- 好处是传入传出只是一个指针的大小
- 如果需要保护传入的结构不被函数修改
  - const struct point *p
- 返回传入的指针是一种套路

结构中的结构

结构数组

```
struct date dates[100];
struct date dates[] = {
    {4,5,2005},{2,4,2005}};
```

结构中的结构

```
struct dateAndTime {
    struct date sdate;
    struct time stime;
};
```

嵌套的结构

```
struct point {
      int x;
      int y;
 };
struct rectangle {
      struct point pt1;
      struct point pt2;
};
```

如果有变量

```
   struct rectangle r;
```

就可以有：

```
    r.pt1.x、r.pt1.y,
    r.pt2.x 和 r.pt2.y
```

如果有变量定义：

```
    struct rectangle r, *rp;
    rp = &r;
```

那么下面的四种形式是等价的：

```
    r.pt1.x
    rp->pt1.x
    (r.pt1).x
    (rp->pt1).x
```

但是没有rp->pt1->x （因为pt1不是指针）

结构中的结构的数组

```
#include <stdio.h>

struct point{
    int x;
    int y;
};

struct rectangle {
    struct point p1;
    struct point p2;
};

void printRect(struct rectangle r)
{
    printf("<%d, %d> to <%d, %d>\n", r.p1.x, r.p1.y, r.p2.x, r.p2.y);
}

int main(int argc, char const *argv[])
{
    int i;
    struct rectangle rects[ ] = {{{1, 2}, {3, 4}}, {{5, 6}, {7, 8}}}; // 2 rectangles
    for(i=0;i<2;i++) printRect(rects[i]);
}
```

typedef

自定义数据类型 (typedef)

- C语言提供了一个叫做 ***typedef*** 的功能来声明一个已有的数据类型的新名字。比如：

```
typedef  int  Length;
```

使得 ***Length*** 成为 ***int*** 类型的别名。

- 这样， ***Length*** 这个名字就可以代替int出现在变量定义和参数声明的地方了：

```
Length   a, b, len ;
Length   numbers[10] ;
```

Typedef

声明新的类型的名字

- 新的名字是某种类型的别名
- 改善了程序的可读性

```
typedef long int64_t;
typedef struct ADate {
  int month;
  int day;
  int year;
} Date;

int64_t i = 100000000000;
Date d = {9, 1, 2005};
```

重载已有的类型名字
新名字的含义更清晰
具有可移植性

简化了复杂的名字

typedef

```
typedef struct {
    int month;
    int day;
    int year;
 } Date;
```

typedef

```
typedef  int Length; // Length就等价于int类型

typedef *char[10] Strings;     // Strings 是10个字符串的数组的类型

typedef struct node {
   int data;
   struct node *next;
 } aNode;
```

或

```
typedef struct node aNode; // 这样用aNode 就可以代替 struct node
```

联合

联合

选择:

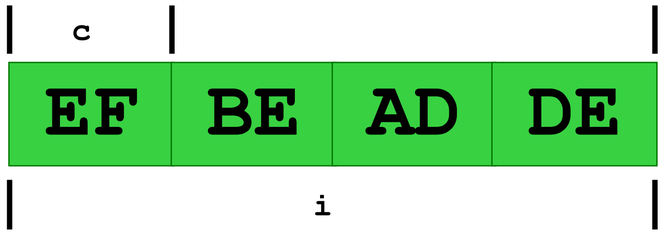
成员是

- 一个 `int i` 还是
- 一个 `char c`

```
union AnElt {
   int   i;
   char  c;
} elt1, elt2;

elt1.i = 4;
elt2.c = 'a';
elt2.i = 0xDEADBEEF;
```

`sizeof(union …) =`

`sizeof(每个成员)的最大值`



联合

- 存储
  - 所有的成员共享一个空间
  - 同一时间只有一个成员是有效的
  - union的大小是其最大的成员
- 初始化
  - 对第一个成员做初始化

联合

union自己并不知道当时其中哪个成员是有效的

```
union AnElt {
   int   i;
   char  c;
} elt1, elt2;


elt1.i = 4;
elt2.c = 'a';
elt2.i = 0xDEADBEEF;


如果 (elt1 当前是char) …
```

?

程序怎么能知道当时elt1和elt2里面到底是int还是char?

?

最好的答案：另一个变量来表达这个事情

union的用处

```c
#include <stdio.h>

typedef union {
    int i;
    char ch[sizeof(int)];
} CHI;

int main(int argc, char const *argv[])
{
    CHI chi;
    int i;
    chi.i = 1234;
    for ( i=0; i<sizeof(int); i++ ) {
        printf("%02hhX", chi.ch[i]);
    }
    printf("\n");

    return 0;
}
```

这个结果表明我们所用的CPU是小端的

全局变量

全局变量

- 定义在函数外面的变量是全局变量
- 全局变量具有全局的生存期和作用域
  - 它们与任何函数都无关
  - 在任何函数内部都可以使用它们

全局变量初始化

- 没有做初始化的全局变量会得到0值
  - 指针会得到NULL值
- 只能用编译时刻已知的值来初始化全局变量
- 它们的初始化发生在main函数之前

被隐藏的全局变量

- 如果函数内部存在与全局变量同名的变量，则全局变量被隐藏

静态本地变量

- 在本地变量定义时加上static修饰符就成为静态本地变量
- 当函数离开的时候，静态本地变量会继续存在并保持其值
- 静态本地变量的初始化只会在第一次进入这个函数时做，以后进入函数时会保持上次离开时的值

静态本地变量

- 静态本地变量实际上是特殊的全局变量
- 它们位于相同的内存区域
- 静态本地变量具有全局的生存期，函数内的局部作用域
  - static在这里的意思是局部作用域（本地可访问）

*返回指针的函数

- 返回本地变量的地址是危险的
- 返回全局变量或静态本地变量的地址是安全的
- 返回在函数内malloc的内存是安全的，但是容易造成问题
- 最好的做法是返回传入的指针

tips

- 不要使用全局变量来在函数间传递参数和结果
- 尽量避免使用全局变量
  - 丰田汽车的案子
- * 使用全局变量和静态本地变量的函数是线程不安全的

宏

编译预处理指令

- #开头的是编译预处理指令
- 它们不是C语言的成分，但是C语言程序离不开它们
- #define用来定义一个宏

#define

- #define <名字> <值>
- 注意没有结尾的分号，因为不是C的语句
- 名字必须是一个单词，值可以是各种东西
- 在C语言的编译器开始编译之前，编译预处理程序（cpp）会把程序中的名字换成值
  - 完全的文本替换
- gcc —save-temps

宏

- 如果一个宏的值中有其他的宏的名字，也是会被替换的
- 如果一个宏的值超过一行，最后一行之前的行末需要加\
- 宏的值后面出现的注释不会被当作宏的值的一部分

没有值的宏

- #define _DEBUG
- 这类宏是用于条件编译的，后面有其他的编译预处理指令来检查这个宏是否已经被定义过了

预定义的宏

- __LINE__
- __FILE__
- __DATE__
- __TIME__
- __STDC__

带参数的宏

像函数的宏

- #define cube(x) ((x)*(x)*(x))
- 宏可以带参数

错误定义的宏

- #define RADTODEG(x) (x * 57.29578)
- #define RADTODEG(x) (x) * 57.29578

带参数的宏的原则

- 一切都要括号
  - 整个值要括号
  - 参数出现的每个地方都要括号
- #define RADTODEG(x) ((x) * 57.29578)

带参数的宏

- 可以带多个参数
  - #define MIN(a,b) ((a)>(b)?(b):(a))
- 也可以组合（嵌套）使用其他宏

分号?

```
#define PRETTY_PRINT(msg) printf(msg);


if (n < 10)
    PRETTY_PRINT("n is less than 10");
else
    PRETTY_PRINT("n is at least 10");
```

带参数的宏

- 在大型程序的代码中使用非常普遍
- 可以非常复杂，如“产生”函数
  - 在#和##这两个运算符的帮助下
- 存在中西方文化差异
- 部分宏会被inline函数替代

其他编译预处理指令

- 条件编译
- error
- …

大程序

多个.c文件

- main()里的代码太长了适合分成几个函数
- 一个源代码文件太长了适合分成几个文件
- 两个独立的源代码文件不能编译形成可执行的程序

编译单元

- 一个.c文件是一个编译单元
- 编译器每次编译只处理一个编译单元

项目

- 在Dev C++中新建一个项目，然后把几个源代码文件加入进去
- 对于项目，Dev C++的编译会把一个项目中所有的源代码文件都编译后，链接起来
- 有的IDE有分开的编译和构建两个按钮，前者是对单个源代码文件编译，后者是对整个项目做链接

头文件

函数原型

- 如果不给出函数原型，编译器会猜测你所调用的函数的所有参数都是int，返回类型也是int
- 编译器在编译的时候只看当前的一个编译单元，它不会去看同一个项目中的其他编译单元以找出那个函数的原型
- 如果你的函数并非如此，程序链接的时候不会出错
- 但是执行的时候就不对了
- 所以需要在调用函数的地方给出函数的原型，以告诉编译器那个函数究竟长什么样

头文件

- 把函数原型放到一个头文件（以.h结尾）中，在需要调用这个函数的源代码文件（.c文件）中#include这个头文件，就能让编译器在编译的时候知道函数的原型

#include

- #include是一个编译预处理指令，和宏一样，在编译之前就处理了
- 它把那个文件的全部文本内容原封不动地插入到它所在的地方
  - 所以也不是一定要在.c文件的最前面#include

“”还是<>

- #include有两种形式来指出要插入的文件
  - “”要求编译器首先在当前目录（.c文件所在的目录）寻找这个文件，如果没有，到编译器指定的目录去找
  - <>让编译器只在指定的目录去找
- 编译器自己知道自己的标准库的头文件在哪里
- 环境变量和编译器命令行参数也可以指定寻找头文件的目录

#include的误区

- #include不是用来引入库的
- stdio.h里只有printf的原型，printf的代码在另外的地方，某个.lib(Windows)或.a(Unix)中
- 现在的C语言编译器默认会引入所有的标准库
- #include <stdio.h>只是为了让编译器知道printf函数的原型，保证你调用时给出的参数值是正确的类型

头文件

- 在使用和定义这个函数的地方都应该#include这个头文件
- 一般的做法就是任何.c都有对应的同名的.h，把所有对外公开的函数的原型和全局变量的声明都放进去

不对外公开的函数

- 在函数前面加上static就使得它成为只能在所在的编译单元中被使用的函数
- 在全局变量前面加上static就使得它成为只能在所在的编译单元中被使用的全局变量

声明

变量的声明

- int i;是变量的定义
- extern int i;是变量的声明

声明和定义

- 声明是不产生代码的东西
  - 函数原型
  - 变量声明
  - 结构声明
  - 宏声明
  - 枚举声明
  - 类型声明
  - inline函数
- 定义是产生代码的东西

头文件

- 只有声明可以被放在头文件中
  - 是规则不是法律
- 否则会造成一个项目中多个编译单元里有重名的实体
  - * 某些编译器允许几个编译单元中存在同名的函数，或者用weak修饰符来强调这种存在

重复声明

- 同一个编译单元里，同名的结构不能被重复声明
- 如果你的头文件里有结构的声明，很难这个头文件不会在一个编译单元里被#include多次
- 所以需要“标准头文件结构”

标准头文件结构

```
#ifndef __LIST_HEAD__
#define __LIST_HEAD__

#include "node.h"

typedef struct _list {
    Node* head;
    Node* tail;
} List;

#endif
```

- 运用条件编译和宏，保证这个头文件在一个编译单元中只会被#include一次
- #pragma once也能起到相同的作用，但是不是所有的编译器都支持

* 前向声明

```
#ifndef __LIST_HEAD__
#define __LIST_HEAD__

struct Node;

typedef struct _list {
    struct Node* head;
    struct Node* tail;
} List;

#endif
```

- 因为在这个地方不需要具体知道Node是怎样的，所以可以用struct Node来告诉编译器Node是一个结构

格式化输入输出

格式化的输入输出

- printf
  - %[flags][width][.prec][hlL]type
- scanf
  - %[flag]type

%[flags][width][.prec][hlL]type

Flag	含义
-	左对齐
+	在前面放 + 或 -
(space)	正数留空
0	0填充

%[flags][width][.prec][hlL]type

width 或 prec	含义
number	最小字符数
*	下一个参数是字符数
.number	小数点后的位数
.*	下一个参数是小数点后的位数

%[flags][width][.prec][hlL]type

类型修饰	含义
hh	单个字节
h	short
l	long
ll	long long
L	long double

%[flags][width][.prec][hlL]type

type	用于	type	用于
i 或 d	int	g	float
u	unsigned int	G	float
o	八进制	a 或 A	十六进制浮点
x	十六进制	c	char
X	字母大写的十六进制	s	字符串
f 或 F	float, 6	p	指针
e 或 E	指数	n	读入/写出的个数

scanf：%[flag]type

flag	含义	flag	含义
*	跳过	l	long, double
数字	最大字符数	ll	long long
hh	char	L	long double
h	short		

scanf：%[flag]type

type	用于	type	用于
d	int	s	字符串（单词）
i	整数，可能为十六进制或八进制	[...]	所允许的字符
u	unsigned int	p	指针
o	八进制		
x	十六进制		
a,e,f,g	float		
c	char		

[^.]

- //$GPRMC,004319.00,A,3016.98468,N,12006.39211,E,0.047,,130909,,,D*79
- scanf("%*[^,],%[^,],%[^,],%[^,],%[^,],%[^,],%[^,],%[^,],%[^,],%[^,]",sTime,sAV,sLati,&sNW,sLong,&sEW,sSpeed,sAngle,sDate);

printf和scanf的返回值

- 读入的项目数
- 输出的字符数

- 在要求严格的程序中，应该判断每次调用scanf或printf的返回值，从而了解程序运行中是否存在问题

文件

文件输入输出

- 用>和<做重定向

输入结束

- getchar读到了EOF
- scanf返回小于要求读的数量

FILE

- FILE* fopen(const char * restrict path, const char * restrict mode);
- int fclose(FILE *stream);
- fscanf(FILE*, ...)
- fprintf(FILE*, ...)

打开文件的标准代码

```
FILE* fp = fopen(“file”,“r”);
if ( fp ) {
    fscanf(fp,...);
    fclose(fp);
} else {
    ...
}
```

fopen

r	打开只读
r+	打开读写，从文件头开始
w	打开只写。如果不存在则新建，如果存在则清空
w+	打开读写。如果不存在则新建，如果存在则清空
a	打开追加。如果不存在则新建，如果存在则从文件尾开始
..x	只新建，如果文件已存在则不能打开

二进制文件

二进制文件

- 其实所有的文件最终都是二进制的
- 文本文件无非是用最简单的方式可以读写的文件
  - more、tail
  - cat
  - vi
- 而二进制文件是需要专门的程序来读写的文件
- 文本文件的输入输出是格式化，可能经过转码

文本 vs 二进制

- Unix喜欢用文本文件来做数据存储和程序配置
  - 交互式终端的出现使得人们喜欢用文本和计算机“talk”
  - Unix的shell提供了一些读写文本的小程序
- Windows喜欢用二进制文件
  - DOS是草根文化，并不继承和熟悉Unix文化
  - PC刚开始的时候能力有限，DOS的能力更有限，二进制更接近底层

文本 vs 二进制

- 文本的优势是方便人类读写，而且跨平台
- 文本的缺点是程序输入输出要经过格式化，开销大
- 二进制的缺点是人类读写困难，而且不跨平台
  - int的大小不一致，大小端的问题...
- 二进制的优点是程序读写快

程序为什么要文件

- 配置
  - Unix用文本，Windows用注册表
- 数据
  - 稍微有点量的数据都放数据库了
- 媒体
  - 这个只能是二进制的
- 现实是，程序通过第三方库来读写文件，很少直接读写二进制文件了

二进制读写

- size_t fread(void *restrict ptr, size_t size, size_t nitems, FILE *restrict stream);
- size_t fwrite(const void *restrict ptr, size_t size, size_t nitems, FILE *restrict stream);
- 注意FILE指针是最后一个参数
- 返回的是成功读写的字节数

为什么nitem?

- 因为二进制文件的读写一般都是通过对一个结构变量的操作来进行的
- 于是nitem就是用来说明这次读写几个结构变量!

在文件中定位

- long ftell(FILE *stream);
- int fseek(FILE *stream, long offset, int whence);
  - SEEK_SET：从头开始
  - SEEK_CUR：从当前位置开始
  - SEEK_END：从尾开始（倒过来）

可移植性

- 这样的二进制文件不具有可移植性
  - 在int为32位的机器上写成的数据文件无法直接在int为64位的机器上正确读出
- 解决方案之一是放弃使用int，而是typedef具有明确大小的类型
- 更好的方案是用文本

位运算

按位运算

- C有这些按位运算的运算符：

- **&** 按位的与
- **|** 按位的或
- **~** 按位取反
- **^** 按位的异或
- **<<** 左移
- **>>** 右移

按位与 &

- 如果 $(x)_i == 1$ 并且 $(y)_i == 1$，那么 $(x \ \& \ y)_i = 1$
- 否则的话 $(x \ \& \ y)_i = 0$
- 按位与常用于两种应用：
  - 让某一位或某些位为0：`x & 0xFE`
  - 取一个数中的一段：`x & 0xFF`

按位或 |

- 如果 $(x)_i$ == 1 或 $(y)_i$ == 1，那么 $(x \mid y)_i$ = 1
- 否则的话， $(x \mid y)_i$ == 0
- 按位或常用于两种应用：
  - 使得一位或几个位为1：x | 0x01
  - 把两个数拼起来：0x00FF | 0xFF00

按位取反 ~

- $(\sim x)_i = 1 - (x)_i$

- **把1位变0，0位变1**

- 想得到全部位为1的数：~0

- 7的二进制是0111，x | 7使得低3位为1，而

- x & ~7，就使得低3位为0

逻辑运算vs按位运算

- 对于逻辑运算，它只看到两个值：0和1
- 可以认为逻辑运算相当于把所有非0值都变成1，然后做按位运算
  - 5 & 4 —>4而 5 && 4 —> 1 & 1 —> 1
  - 5 | 4 —> 5而 5 || 4 —> 1 | 1 —> 1
  - ~4 —> 3而 !4 —> !1 —> 0

按位异或^

- 如果 $(x)_i == (y)_i$，那么 $(x \hat{} y)_i = 0$
- 否则的话，$(x \hat{} y)_i == 1$
- 如果两个位相等，那么结果为0；不相等，结果为1
- 如果x和y相等，那么x ^ y的结果为0
- 对一个变量用同一个值异或两次，等于什么也没做
  - x ^ y ^ y —> x

左移 <<

- **`i << j`**
- i中所有的位向左移动j个位置，而右边填入0
- 所有小于int的类型，移位以int的方式来做，结果是int
- **`x <<= 1`** 等价于 **`x *= 2`**
- **`x <<= n`** 等价于 **`x *= 2`**^{n}.

右移 >>

- **`i >> j`**
- i中所有的位向右移j位
- 所有小于int的类型，移位以int的方式来做，结果是int
- 对于unsigned的类型，左边填入0
- 对于signed的类型，左边填入原来的最高位（保持符号不变）
- **`x >>= 1`** 等价于 **`x /= 2`**
- **`x >>= n`** 等价于 **`x /= 2`**^{n}.

no zuo no die

- 移位的位数不要用负数，这是没有定义的行为
  - `x << -2  //!!NO!!`

输出一个数的二进制

```c
#include <stdio.h>

int main(int argc, char const *argv[])
{
    int number;
    scanf("%d", &number);
    unsigned mask = 1u<<31;
    for ( ; mask ; mask >>=1 ) {
        printf("%d", number & mask?1:0);
    }
    printf("\n");

    return 0;
}
```

MCU的SFR

Table 70: UART0 Line Control Register (U0LCR - 0xE000C00C)

U0LCR	Function	Description	Reset Value
1:0	Word Length Select	00: 5 bit character length 01: 6 bit character length 10: 7 bit character length 11: 8 bit character length	0
2	Stop Bit Select	0: 1 stop bit 1: 2 stop bits (1.5 if U0LCR[1:0]=00)	0
3	Parity Enable	0: Disable parity generation and checking 1: Enable parity generation and checking	0
5:4	Parity Select	00: Odd parity 01: Even parity 10: Forced “1” stick parity 11: Forced “0” stick parity	0
6	Break Control	0: Disable break transmission 1: Enable break transmission. Output pin UART0 TxD is forced to logic 0 when U0LCR6 is active high.	0
7	Divisor Latch Access Bit	0: Disable access to Divisor Latches 1: Enable access to Divisor Latches	0

MCU的SFR

Table 70: UART0 Line Control Register (U0LCR - 0xE000C00C)

U0LCR	Function	Description	Reset Value
1:0	Word Length Select	00: 5 bit character length 01: 6 bit character length 10: 7 bit character length 11: 8 bit character length	0
2	Stop Bit Select	0: 1 stop bit 1: 2 stop bits (1.5 if U0LCR[1:0]=00)	0
3	Parity Enable	0: Disable parity generation and checking 1: Enable parity generation and checking	0
5:4	Parity Select	00: Odd parity 01: Even parity 10: Forced “1” stick parity 11: Forced “0” stick parity	0
6	Break Control	0: Disable break transmission 1: Enable break transmission. Output pin UART0 TxD is forced to logic 0 when U0LCR6 is active high.	0
7	Divisor Latch Access Bit	0: Disable access to Divisor Latches 1: Enable access to Divisor Latches	0

- const unsigned int SBS = 1u << 2;
- const unsigned int PE = 1u << 3;
- U0LCR |= SBS | PE;
- U0LCR &= ~SBS;
- U0LCR &= ~(SBS | PE);

位段

- 把一个int的若干位组合成一个结构

```
struct {
    unsigned int leading : 3;
    unsigned int FLAG1: 1;
    unsigned int FLAG2: 1;
    int trailing: 11;
};
```

位段

- 可以直接用位段的成员名称来访问
  - 比移位、与、或还方便
- 编译器会安排其中的位的排列，不具有可移植性
- 当所需的位超过一个int时会采用多个int